7

Arrays



© 1992-2007 Pearson Education, Inc. All rights reserved.

Now go, write it before them in a table, and note it in a book.

— Isaiah 30:8

To go beyond is as wrong as to fall short.

— Confucius

Begin at the beginning,… and go on till you come to the end: then stop.

— Lewis Carroll



© 1992-2007 Pearson Education, Inc. All rights reserved.

OBJECTIVES

In this chapter you will learn:

- What arrays are.
- To use arrays to store data in and retrieve data from lists and tables of values.
- To declare an array, initialize an array and refer to individual elements of an array.
- To use the enhanced `for` statement to iterate through arrays.
- To pass arrays to methods.
- To declare and manipulate multidimensional arrays.
- To write methods that use variable-length argument lists.
- To read command-line arguments into a program.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

7.1 Introduction

7.2 Arrays

7.3 Declaring and Creating Arrays

7.4 Examples Using Arrays

7.5 Case Study: Card Shuffling and Dealing Simulation

7.6 Enhanced `for` Statement

7.7 Passing Arrays to Methods

7.8 Case Study: Class `GradeBook` Using an Array to Store Grades

7.9 Multidimensional Arrays

7.10 Case Study: Class `GradeBook` Using a Two-Dimensional Array

7.11 Variable-Length Argument Lists

7.12 Using Command-Line Arguments

7.13 (Optional) GUI and Graphics Case Study: Drawing Arcs

7.14 (Optional) Software Engineering Case Study: Collaboration Among Objects

7.15 Wrap-Up



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.1 Introduction

- **Arrays**
  - **Data structures**
  - **Related data items of same type**
  - **Remain same size once created**
    - **Fixed-length entries**



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.2 Arrays

- **Array**
  - **Group of variables**
    - **Have same type**
  - **Reference type**



© 1992-2007 Pearson Education, Inc. All rights reserved.

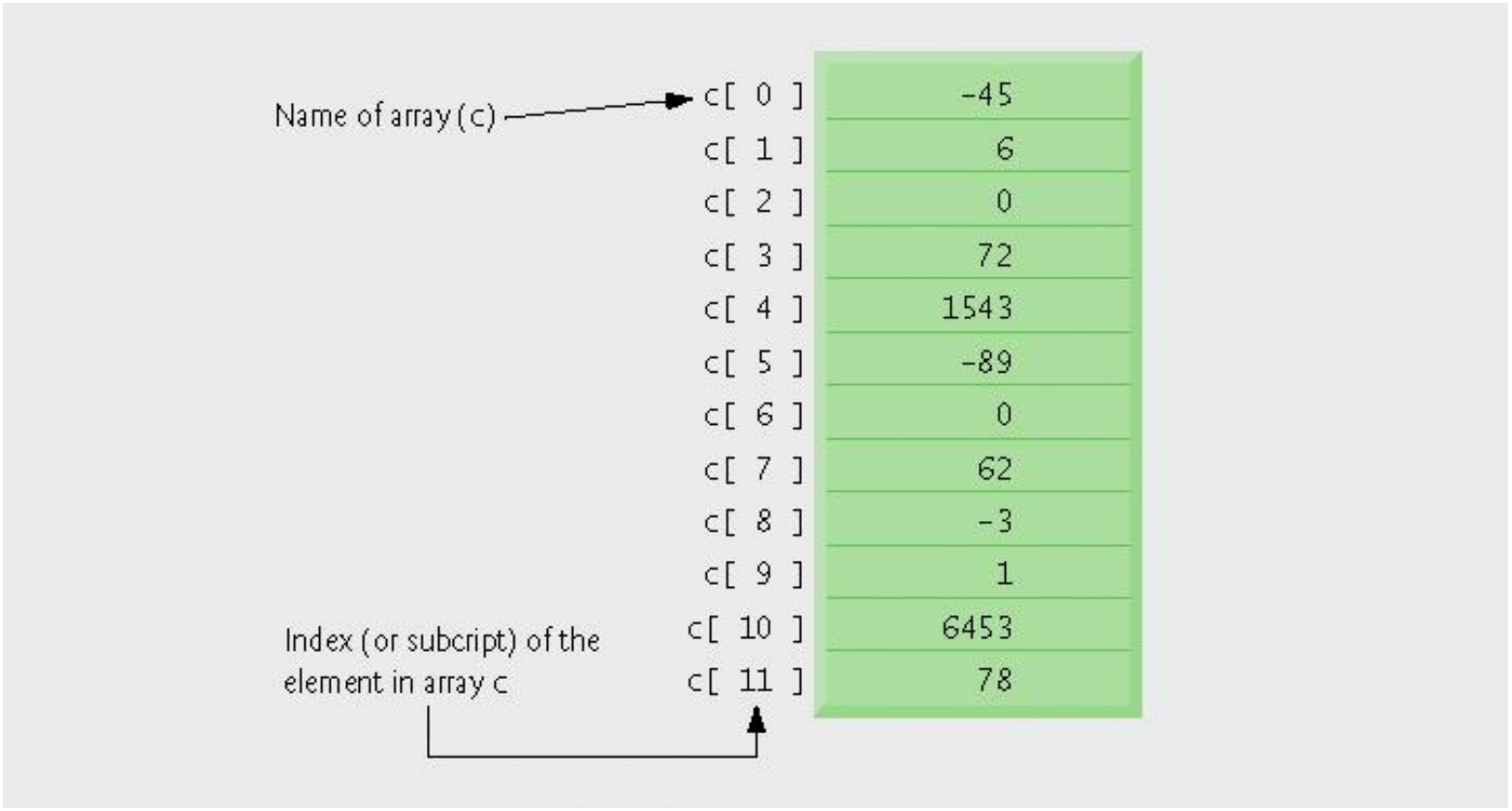


Fig. 7.1 | A 12-element array.

© 1992-2007 Pearson Education, Inc. All rights reserved.

7.2 Arrays (Cont.)

- **Index**
  - **Also called subscript**
  - **Position number in square brackets**
  - **Must be positive integer or integer expression**
  - **First element has index zero**

```
a = 5;
b = 6;
c[ a + b ] += 2;
```

- **Adds `2` to `c[ 11 ]`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 7.1

Using a value of type `long` as an array index results in a compilation error. An index must be an `int` value or a value of a type that can be promoted to int—namely, `byte`, `short` or `char`, but not `long`.



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.2 Arrays (Cont.)

- **Examine array `c`**
  - **`c` is the array *name***
  - **`c.length` accesses array `c`'s *length***
  - **`c` has 12 *elements* (`c[0]`, `c[1]`, … `c[11]`)**
    - **The *value* of `c[0]` is `-45`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.3 Declaring and Creating Arrays

- **Declaring and Creating arrays**
  - **Arrays are objects that occupy memory**
  - **Created dynamically with keyword new**

```
int c[] = new int[ 12 ];
```

    - **Equivalent to**

```
int c[];  // declare array variable
c = new int[ 12 ]; // create array
```

  - **We can create arrays of objects too**

```
String b[] = new String[ 100 ];
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 7.2

In an array declaration, specifying the number of elements in the square brackets of the declaration (`e.g., int c[ 12 ];`) is a syntax error.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Good Programming Practice 7.1

For readability, declare only one variable per declaration. Keep each declaration on a separate line, and include a comment describing the variable being declared.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 7.3

Declaring multiple array variables in a single declaration can lead to subtle errors. Consider the declaration `int[] a, b, c;`. If `a`, `b` and `c` should be declared as array variables, then this declaration is correct—placing square brackets directly following the type indicates that all the identifiers in the declaration are array variables. However, if only `a` is intended to be an array variable, and `b` and `c` are intended to be individual int variables, then this declaration is incorrect—the declaration `int a[], b, c;` would achieve the desired result.



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.4 Examples Using Arrays

- **Declaring arrays**
- **Creating arrays**
- **Initializing arrays**
- **Manipulating array elements**



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.4 Examples Using Arrays

- **Creating and initializing an array**
  - **Declare array**
  - **Create array**
  - **Initialize array elements**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 7.2: InitArray.j
2  // Creating an array.
3
4  public class InitArray
5  {
6     public static void main( String
7     {
8        int array[]; // declare array named array
9
10       array = new int[ 10 ]; // create the space for array
11
12       System.out.printf( "%s%8s\n", "Index", "Value" ); // column headings
13
14       // output each
15       for ( int counter = 0; counter < array.length; counter++ )
16          System.out.printf( "%5d%8d\n", counter, array[ counter ] );
17    } // end main
18 } // end class InitArray
```

Declare `array` as an array of `int`s

Create 10 `int`s for `array`; each `int` is initialized to 0 by default

`array.length` returns length of `array`

Each `int` is initialized to 0 by default

`array[counter]` returns `int` associated with index in `array`

```
Index   Value
    0       0
    1       0
    2       0
    3       0
    4       0
    5       0
    6       0
    7       0
    8       0
```

served.

7.4 Examples Using Arrays (Cont.)

- **Using an array initializer**
  - Use *initializer list*
    - Items enclosed in braces ({})
    - Items in list separated by commas

```
int n[] = { 10, 20, 30, 40, 50 };
```

      - Creates a five-element array
      - Index values of 0, 1, 2, 3, 4
  - Do not need keyword `new`



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Declare `array` as an array of `int`s

Compiler uses initializer list to allocate array

```
1  // Fig. 7.3: InitArray.java
2  // Initializing the elements of an array with a
3
4  public class InitArray
5  {
6     public static void main( String args[] )
7     {
8        // initializer list specifies the value for each element
9        int array[] = { 32, 27, 64, 18, 95, 14, 90, 70, 60, 37 };
10
11       System.out.printf( "%s%8s\n", "Index", "Value" ); // column headings
12
13       // output each array element's value
14       for ( int counter = 0; counter < array.length; counter++ )
15          System.out.printf( "%5d%8d\n", counter, array[ counter ] );
16    } // end main
17 } // end class InitArray
```

```
Index   Value
    0      32
    1      27
    2      64
    3      18
    4      95
    5      14
    6      90
    7      70
    8      60
    9      37
```

ights reserved.

7.4 Examples Using Arrays (Cont.)

- **Calculating a value to store in each array element**
  - **Initialize elements of 10-element array to even integers**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 7.4: InitArray.java
2  // Calculating values to be placed i
3
4  public class InitArray
5  {
6     public static void main( String args[] )
7     {
8        final int ARRAY_LENGTH = 10; // declare constant
9        int array[] = new int[ ARRAY_LENGTH ]; // create array
10
11       // calculate value for each array element
12       for ( int counter = 0; counter < array.length; counter++ )
13          array[ counter ] = 2 + 2 * counter;
14
15       System.out.printf( "%s%8s\n",                    // column headings
16
17       // output each array element'
18       for ( int counter = 0; counter < array.length; counter++ )
19          System.out.printf( "%5d%8d\n", counter, array[ counter ] );
20    } // end main
21 } // end class InitArray
```

Declare constant variable ARRAY_LENGTH using the `final` modifier

Declare and create `array` that contains 10 `int`s

Use `array` index to assign array value

```
Index   Value
    0       2
    1       4
    2       6
    3       8
    4      10
    5      12
    6      14
    7      16
    8      18
    9      20
```

Good Programming Practice 7.2

Constant variables also are called named constants or read-only variables. Such variables often make programs more readable than programs that use literal values (e.g., 10)—a named constant such as ARRAY_LENGTH clearly indicates its purpose, whereas a literal value could have different meanings based on the context in which it is used.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 7.4

Assigning a value to a constant after the variable has been initialized is a compilation error.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 7.5

Attempting to use a constant before it is initialized is a compilation error.



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.4 Examples Using Arrays (Cont.)

- **Summing the elements of an array**
  - **Array elements can represent a series of values**
    - **We can sum these values**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Declare `array` with initializer list

Sum all `array` values

```
// Fig. 7.5: SumArray.java
// Computing the sum of the elements of an array.

public class SumArray
{
   public static void main( String args[] )
   {
      int array[] = { 87, 68, 94, 100,  87 };
      int total = 0;

      // add each element's value to total
      for ( int counter = 0; counter < array.length; counter++ )
         total += array[ counter ];

      System.out.printf( "Total of array elements: %d\n",   total );
   } // end main
} // end class SumArray
```

```
tal of array elements: 849
```

© 1992-2007 Pearson Education, Inc. All rights reserved.

7.4 Examples Using Arrays (Cont.)

- **Using bar charts to display array data graphically**
  - **Present data in graphical manner**
    - **E.g., bar chart**
  - **Examine the distribution of grades**



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 7.6: BarChart.java
2  // Bar chart printing program.
3
4  public class BarChart
5  {
6     public static void main( String args[] )
7     {
8        int array[] = { 0, 0, 0, 0, 0, 0, 1, 2, 4, 2, 1 };
9
10       System.out.println( "Grade distribution:" );
11
12       // for each array element, output a bar of the chart
13       for ( int counter = 0; counter < array.length; counter++ )
14       {
15          // output bar label ( "00-09: ", ..., "90-99: ", "100: " )
16          if ( counter == 10 )
17             System.out.printf( "%5d: ", 100 );
18          else
19             System.out.printf( "%02d-%02d: ",
20                counter * 10, counter * 10 + 9 );
21
22          // print bar of asterisks
23          for ( int stars = 0; stars < array[ counter ]; stars++ )
24             System.out.print( "*" );
25
26          System.out.println(); // start a new line of output
27       } // end outer for
28    } // end main
29 } // end class BarChart
```

Declare `array` with initializer list

Use the `0` flag to display one-digit grade with a leading 0

For each `array` element, print associated number of asterisks

Outline

BarChart.java

(1 of 2)

Line 8
Declare array with initializer list

Line 19
Use the `0` flag to display one-digit grade with a leading 0



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
Grade distribution:
00-09:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: *
70-79: **
80-89: ****
90-99: **
  100: *
```

Outline

BarChart.java

(2 of 2)

Program output



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.4 Examples Using Arrays (Cont.)

- **Using the elements of an array as counters**
  - **Use a series of counter variables to summarize data**



© 1992-2007 Pearson Education, Inc. All rights reserved.

```java
// Fig. 7.7: RollDie.java
// Roll a six-sided die 6000 times.
import java.util.Random;

public class RollDie
{
   public static void main( String args[] )
   {
      Random randomNumbers = new Random(); // random number generator
      int frequency[] = new int[ 7 ]; // array of frequency counters

      // roll die 6000 times; use die value as frequency ind
      for ( int roll = 1; roll <= 6000; roll++ )
         ++frequency[ 1 + randomNumbers.nextInt( 6 ) ];

      System.out.printf( "%s%10s\n"

      // output each array element's value
      for ( int face = 1; face < frequency.length; face++ )
         System.out.printf( "%4d%10d\n", face, frequency[ face ] );
   } // end main
} // end class RollDie
```

```
Face Frequency
   1       988
   2       963
   3      1018
   4      1041
   5       978
   6      1012
```

Declare `frequency` as array of 7 `int`s

Generate 6000 random integers in range 1-6

Increment `frequency` values at index associated with random number

Outline

Die.java

Line 10
Declare
frequency as
of 7 ints

13-14
Generate 6000
random integers
in range 1-6

Line 14
Increment
frequency values
at index
associated with
random number

Program output



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.4 Examples Using Arrays (Cont.)

- **Using arrays to analyze survey results**
  - **40 students rate the quality of food**
    - **1–10 Rating scale: 1 means awful, 10 means excellent**
  - **Place 40 responses in array of integers**
  - **Summarize results**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 7.8: StudentPoll.java
2  // Poll analysis program.
3
4  public class StudentPoll
5  {
6     public static void main( String args[] )
7     {
8        // array of survey responses
9        int responses[] = { 1, 2, 6, 4, 8, 5, 9, 7, 8, 10,
10          10, 3, 8, 2, 7, 6, 5, 7, 6, 8, 6, 7, 5, 6, 6, 5
11          4, 8, 6, 8, 10 };
12       int frequency[] = new int[ 11 ]; // array of frequency counters
13
14       // for each answer, select responses element and use that value
15       // as frequency index to determine element to increment
16       for ( int answer = 0; answer < responses.length; answer++ )
17          ++frequency[ responses[ answer ] ];
18
19       System.out.printf( "%s%10s", "Rating", "Frequency" );
20
21       // output each array element's value
22       for ( int rating = 1; rating < frequency.length; rating++ )
23          System.out.printf( "%d%10d", rating, frequency[ rating ] );
24    } // end main
25 } // end class StudentPoll
```

Declare `responses` as array to store 40 responses

Declare `frequency` as array of 11 `int` and ignore the first element

For each response, increment `frequency` values at index associated with that response

StudentPoll.java

(1 of 2)

ponses as array to store 40 responses

Line 12 Declare frequency as array of 11 int re the ement

-17 response, increment frequency values at index associated with that response



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
Rating Frequency
     1         2
     2         2
     3         2
     4         2
     5         5
     6        11
     7         5
     8         7
     9         1
    10         3
```

Outline

StudentPoll.java

(2 of 2)

Program output



© 1992-2007 Pearson Education, Inc. All rights reserved.

Error-Prevention Tip 7.1

An exception indicates that an error has occurred in a program. A programmer often can write code to recover from an exception and continue program execution, rather than abnormally terminating the program. When a program attempts to access an element outside the array bounds, an `ArrayIndexOutOfBoundsException` occurs. Exception handling is discussed in Chapter 13.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Error-Prevention Tip 7.2

When writing code to loop through an array, ensure that the array index is always greater than or equal to 0 and less than the length of the array. The loop-continuation condition should prevent the accessing of elements outside this range.



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.5 Case Study: Card Shuffling and Dealing Simulation

- **Program simulates card shuffling and dealing**
  - **Use random number generation**
  - **Use an array of reference type elements to represent cards**
  - **Three classes**
    - **Card**
      - **Represents a playing card**
    - **DeckOfCards**
      - **Represents a deck of 52 playing cards**
    - **DeckOfCardsTest**
      - **Demonstrates card shuffling and dealing**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Card.java

Lines 17-20

```
1  // Fig. 7.9: Card.java
2  // Card class represents a playing card.
3
4  public class Card
5  {
6     private String face; // face of card ("Ace", "Deuce", ...)
7     private String suit; // suit of card ("Hearts", "Diamonds", ...)
8
9     // two-argument constructor initializes card's face and suit
10    public Card( String cardFace, String cardSuit )
11    {
12       face = cardFace; // initialize face of card
13       suit = cardSuit; // initialize suit of card
14    } // end two-argument Card constructor
15
16    // return String representation of Card
17    public String toString()
18    {
19       return face + " of " + suit;
20    } // end method toString
21 } // end class Card
```

Return the string representation of a card



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 7.10: DeckOfCards.java
2  // DeckOfCards class represents a deck of playing cards.
3  import java.util.Random;
4
5  public class DeckOfCards
6  {
7     private Card deck[]; // array of Card objects
8     private int currentCard; // index of next Card to be dea
9     private final int NUMBER_OF_CARDS = 52; // constant number of Cards
10    private Random randomNumbers; // random number generator
11
12    // constructor fills deck of Cards
13    public DeckOfCards()
14    {
15       String faces[] = { "Ace", "Deuce
16          "Seven", "Eight", "Nine", "Te
17       String suits[] = { "Hearts", "Di
18
19       deck = new Card[ NUMBER_OF_CARDS ]; // create array of Card objects
20       currentCard = 0; // set currentCard so first Card dealt is deck[ 0 ]
21       randomNumbers = new Random(); // create random nu
22
23       // populate deck with Card objects
24       for ( int count = 0; count < deck.length; count++ )
25          deck[ count ] =
26             new Card( faces[ count % 13 ], suits[ count / 13 ] );
27    } // end DeckOfCards constructor
```

Declare `deck` as array to store `Card` objects

Constant `NUMBER_OF_CARDS` indicates the number of Cards in the deck

Declare and initialize `faces` with `String`s that represent the face of card

Declare and initialize `suits` with `String`s that represent the suit of card

Fill the `deck` array with `Card`s

(1 of 2)

Line 7

Line 9

Lines 15-16

Line 17

Lines 24-26



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

DeckOfCards.java

(2 of 2)

```
28
29    // shuffle deck of Cards with one-pass algorithm
30    public void shuffle()
31    {
32       // after shuffling, dealing should start at deck[ 0 ] again
33       currentCard = 0; // reinitialize currentCard
34
35       // for each Card, pick another random Card and swap them
36       for ( int first = 0; first < deck.length; first++ )
37       {
38          // select a random number between 0 and 51
39          int second =  randomNumbers.nextInt( NUMBER_OF_CARDS );
40
41          // swap current Card with randomly selected Card
42          Card temp = deck[ first ];
43          deck[ first ] = deck[ second ];
44          deck[ second ] = temp;
45       } // end for
46    } // end method shuffle
47
48    // deal one Card
49    public Card dealCard()
50    {
51       // determine whether Cards remain to be dealt
52       if ( currentCard < deck.length )
53          return deck[ currentCard++ ]; // return current Card in array
54       else
55          return null; // return null to indicate that all Cards were dealt
56    } // end method dealCard
57 } // end class DeckOfCards
```

Swap current `Card` with randomly selected `Card`

Lines 42-44

Determine whether `deck` is empty

Line 52



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

DeckOfCardsTest.java

(1 of 2)

```
1  // Fig. 7.11: DeckOfCardsTest.java
2  // Card shuffling and dealing application.
3
4  public class DeckOfCardsTest
5  {
6     // execute application
7     public static void main( String args[] )
8     {
9        DeckOfCards myDeckOfCards = new DeckOfCards();
10       myDeckOfCards.shuffle(); // place Cards in random order
11
12       // print all 52 Cards in the order in which they are dealt
13       for ( int i = 0; i < 13; i++ )
14       {
15          // deal and print 4 Cards
16          System.out.printf( "%-20s%-20s%-20s%-20s\n",
17             myDeckOfCards.dealCard(), myDeckOfCards.dealCard(),
18             myDeckOfCards.dealCard(), myDeckOfCards.dealCard() );
19       } // end for
20    } // end main
21 } // end class DeckOfCardsTest
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
Six of Spades       Eight of Spades     Six of Clubs        Nine of Hearts
Queen of Hearts     Seven of Clubs      Nine of Spades      King of Hearts
Three of Diamonds   Deuce of Clubs      Ace of Hearts       Ten of Spades
Four of Spades      Ace of Clubs        Seven of Diamonds   Four of Hearts
Three of Clubs      Deuce of Hearts     Five of Spades      Jack of Diamonds
King of Clubs       Ten of Hearts       Three of Hearts     Six of Diamonds
Queen of Clubs      Eight of Diamonds   Deuce of Diamonds   Ten of Diamonds
Three of Spades     King of Diamonds    Nine of Clubs       Six of Hearts
Ace of Spades       Four of Diamonds    Seven of Hearts     Eight of Clubs
Deuce of Spades     Eight of Hearts     Five of Hearts      Queen of Spades
Jack of Hearts      Seven of Spades     Four of Clubs       Nine of Diamonds
Ace of Diamonds     Queen of Diamonds   Five of Clubs       King of Spades
Five of Diamonds    Ten of Clubs        Jack of Spades      Jack of Clubs
```

Outline

DeckOfCardsTest

.java

(2 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.6 Enhanced `for` Statement

- **Enhanced `for` statement**
  - **Iterates through elements of an array or a collection without using a counter**
  - **Syntax**

```
for ( parameter : arrayName )
   statement
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 7.12: EnhancedForTest.java
2  // Using enhanced for statement to total integers in an array.
3
4  public class EnhancedForTest
5  {
6     public static void main( String
7     {
8        int array[] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
9        int total = 0;
10
11       // add each element's value to total
12       for ( int number : array )
13          total += number;
14
15       System.out.printf( "Total of array elements: %d\n", total );
16    } // end main
17 } // end class EnhancedForTest
```

For each iteration, assign the next element of `array` to `int` variable `number`, then add it to `total`

```
Total of array elements: 849
```

© 1992-2007 Pearson Education, Inc. All rights reserved.

7.6 Enhanced `for` Statement (Cont.)

- **Lines 12-13 are equivalent to**

```
for ( int counter = 0; counter < array.length; counter++ )
    total += array[ counter ];
```

- **Usage**
  - **Can access array elements**
  - **Cannot modify array elements**
  - **Cannot access the counter indicating the index**



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.7 Passing Arrays to Methods

- To pass array argument to a method
  - Specify array name without brackets
    - Array `hourlyTemperatures` is declared as

```
int hourlyTemperatures = new int[ 24 ];
```

    - The method call

```
modifyArray( hourlyTemperatures );
```

    - Passes array `hourlyTemperatures` to method `modifyArray`



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 7.13: PassArray.java
2  // Passing arrays and individual array elements to methods.
3
4  public class PassArray
5  {
6     // main creates array and calls modifyArray and modifyElement
7     public static void main( String args[] )
8     {
9        int array[] = { 1, 2, 3, 4, 5 };
10
11       System.out.println(
12          "Effects of passing reference to entire array:\n" +
13          "The values of the original array are:" );
14
15       // output original array elements
16       for ( int value : array )
17          System.out.printf( "   %d", value );
18
19       modifyArray( array ); // pass array reference
20       System.out.println( "\n\nThe values of the modified array are:" );
21
22       // output modified array elements
23       for ( int value : array )
24          System.out.printf( "   %d", value );
25
26       System.out.printf(
27          "\n\nEffects of passing array element value:\n" +
28          "array[3] before modifyElement: %d\n", array[ 3 ] );
```

Declare 5-int array with initializer list

Pass entire array to method modifyArray

ava

reserved.

```
34
35    // multiply each element of an array by 2
36    public static void modifyAr
37    {
38       for ( int counter = 0; counter < array2.lengt
39          array2[ counter ] *= 2;
40    } // end method modifyArray
41
42    // multiply argument by 2
43    public static void modifyElement
44    {
45       element *= 2;
46       System.out.printf(
47          "Value of element in modifyElement: %d\n", element );
48    } // end method modifyElement
49 } // end class PassArray
```

Pass array element `array[3]` to method `modifyElement`

Method `modifyArray` manipulates the array directly

Method `modifyElement` manipulates a primitive's copy

```
Effects of passing reference to entire array:
The values of the original array are:
   1   2   3   4   5

The values of the modified array are:
   2   4   6   8   10

Effects of passing array element value:
array[3] before modifyElement: 8
Value of element in modifyElement: 16
array[3] after modifyElement: 8
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.7 Passing Arrays to Methods (Cont.)

- **Notes on passing arguments to methods**
  - **Two ways to pass arguments to methods**
    - **Pass-by-value**
      - **Copy of argument's value is passed to called method**
      - **Every primitive type is passed-by-value**
    - **Pass-by-reference**
      - **Caller gives called method direct access to caller's data**
      - **Called method can manipulate this data**
      - **Improved performance over pass-by-value**
      - **Every object is passed-by-reference**
        - **Arrays are objects**
        - **Therefore, arrays are passed by reference**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Performance Tip 7.1

Passing arrays by reference makes sense for performance reasons. If arrays were passed by value, a copy of each element would be passed. For large, frequently passed arrays, this would waste time and consume considerable storage for the copies of the arrays.



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.8 Case Study: Class GradeBook Using an Array to Store Grades

- **Further evolve class GradeBook**
- **Class GradeBook**
  - **Represents a grade book that stores and analyzes grades**
  - **Does not maintain individual grade values**
  - **Repeat calculations require reentering the same grades**
    - **Can be solved by storing grades in an array**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

(1 of 5)

Line 7

Line 13

```
1  // Fig. 7.14: GradeBook.java
2  // Grade book using an array to store test grades.
3
4  public class GradeBook
5  {
6     private String courseName; // name of course this GradeBook represents
7     private int grades[]; // array of student grades
8
9     // two-argument constructor initializes courseName
10    public GradeBook( String name, int gradesArray[] )
11    {
12       courseName = name; // initialize courseName
13       grades = gradesArray; // store grades
14    } // end two-argument GradeBook constructor
15
16    // method to set the course name
17    public void setCourseName( String name )
18    {
19       courseName = name; // store the course name
20    } // end method setCourseName
21
22    // method to retrieve the course name
23    public String getCourseName()
24    {
25       return courseName;
26    } // end method getCourseName
27
```

Declare array `grades` to store individual grades

Assign the array's reference to instance variable `grades`



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

(2 of 5)

```
28    // display a welcome message to the GradeBook user
29    public void displayMessage()
30    {
31       // getCourseName gets the name of the course
32       System.out.printf( "Welcome to the grade book for\n%s!\n\n",
33          getCourseName() );
34    } // end method displayMessage
35
36    // perform various operations on the data
37    public void processGrades()
38    {
39       // output grades array
40       outputGrades();
41
42       // call method getAverage to calculate the average grade
43       System.out.printf( "\nClass average is %.2f\n", getAverage() );
44
45       // call methods getMinimum and getMaximum
46       System.out.printf( "Lowest grade is %d\nHighest grade is %d\n\n",
47          getMinimum(), getMaximum() );
48
49       // call outputBarChart to print grade distribution chart
50       outputBarChart();
51    } // end method processGrades
52
53    // find minimum grade
54    public int getMinimum()
55    {
56       int lowGrade = grades[ 0 ]; // assume grades[ 0 ] is smallest
57
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
58       // loop through grades array
59       for ( int grade : grades )
60       {
61          // if grade lower than lowGrade, assign it to lowGrade
62          if ( grade < lowGrade )
63             lowGrade = grade; // new lowest grade
64       } // end for
65
66       return lowGrade; // return lowest grade
67    } // end method getMinimum
68
69    // find maximum grade
70    public int getMaximum()
71    {
72       int highGrade = grades[ 0 ]; // assume grades[ 0 ] is largest
73
74       // loop through grades array
75       for ( int grade : grades )
76       {
77          // if grade greater than highGrade, assign it to highGrade
78          if ( grade > highGrade )
79             highGrade = grade; // new highest grade
80       } // end for
81
82       return highGrade; // return highest grade
83    } // end method getMaximum
84
```

Loop through `grades` to find the lowest grade

Loop through `grades` to find the highest grade

Outline

`radeBook.java`

(3 of 5)

Lines 59-64

Lines 75-80



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

(4 of 5)

Lines 91-92

Lines 107-108

```
85   // determine average grade for test
86   public double getAverage()
87   {
88      int total = 0; // initialize total
89
90      // sum grades for one student
91      for ( int grade : grades )
92         total += grade;
93
94      // return average of grades
95      return (double) total / grades.length;
96   } // end method getAverage
97
98   // output bar chart displaying grade distribution
99   public void outputBarChart()
100    {
101       System.out.println( "Grade distribution:" );
102
103       // stores frequency of grades in each range of 10 grades
104       int frequency[] = new int[ 11 ];
105
106       // for each grade, increment the appropriate frequency
107       for ( int grade : grades )
108          ++frequency[ grade / 10 ];
109
```

Loop through `grades` to sum grades for one student

Loop through `grades` to calculate frequency



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

(5 of 5)

Lines 134-136

```
110      // for each grade frequency, print bar in chart
111      for ( int count = 0; count < frequency.length; count++ )
112      {
113         // output bar label ( "00-09: ", ..., "90-99: ", "100: " )
114         if ( count == 10 )
115            System.out.printf( "%5d: ", 100 );
116         else
117            System.out.printf( "%02d-%02d: ",
118               count * 10, count * 10 + 9  );
119
120         // print bar of asterisks
121         for ( int stars = 0; stars < frequency[ count ]; stars++ )
122            System.out.print( "*" );
123
124         System.out.println(); // start a new line of output
125      } // end outer for
126   } // end method outputBarChart
127
128   // output the contents of the grades array
129   public void outputGrades()
130   {
131      System.out.println( "The grades are:\n" );
132
133      // output each student's grade
134      for ( int student = 0; student < grades.length; student++ )
135         System.out.printf( "Student %2d: %3d\n",
136            student + 1, grades[ student ] );
137   } // end method outputGrades
138 } // end class GradeBook
```

Loop through `grades` to display each grade



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 7.1

A test harness (or test application) is responsible for creating an object of the class being tested and providing it with data. This data could come from any of several sources. Test data can be placed directly into an array with an array initializer, it can come from the user at the keyboard, it can come from a file (as you will see in Chapter 14), or it can come from a network (as you will see in Chapter 24). After passing this data to the class's constructor to instantiate the object, the test harness should call upon the object to test its methods and manipulate its data. Gathering data in the test harness like this allows the class to manipulate data from several sources.



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 7.15: GradeBookTest.java
2  // Creates GradeBook object using an array of grades.
3
4  public class GradeBookTest
5  {
6     // main method begins program execution
7     public static void main( String args[] )
8     {
9        // array of student grades
10       int gradesArray[] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
11
12       GradeBook myGradeBook = new GradeBook(
13          "CS101 Introduction to Java Programming", gradesArray );
14       myGradeBook.displayMessage();
15       myGradeBook.processGrades();
16    } // end main
17 } // end class GradeBookTest
```

Declare and initialize `gradesArray` with 10 elements

Pass `gradesArray` to `GradeBook` constructor

Outline

`eBookTest`

`.java`

(1 of 2)

Line 10

Line 13



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
Welcome to the grade book for
CS101 Introduction to Java Programming!

The grades are:

Student  1:  87
Student  2:  68
Student  3:  94
Student  4: 100
Student  5:  83
Student  6:  78
Student  7:  85
Student  8:  91
Student  9:  76
Student 10:  87

Class average is 84.90
Lowest grade is 68
Highest grade is 100

Grade distribution:
00-09:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: *
70-79: **
80-89: ****
90-99: **
  100: *
```

Outline

GradeBookTest

.java

(2 of 2)

Program output



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.9 Multidimensional Arrays

- **Multidimensional arrays**
  - **Tables with rows and columns**
    - **Two-dimensional array**
    - **m-by-n array**



© 1992-2007 Pearson Education, Inc. All rights reserved.

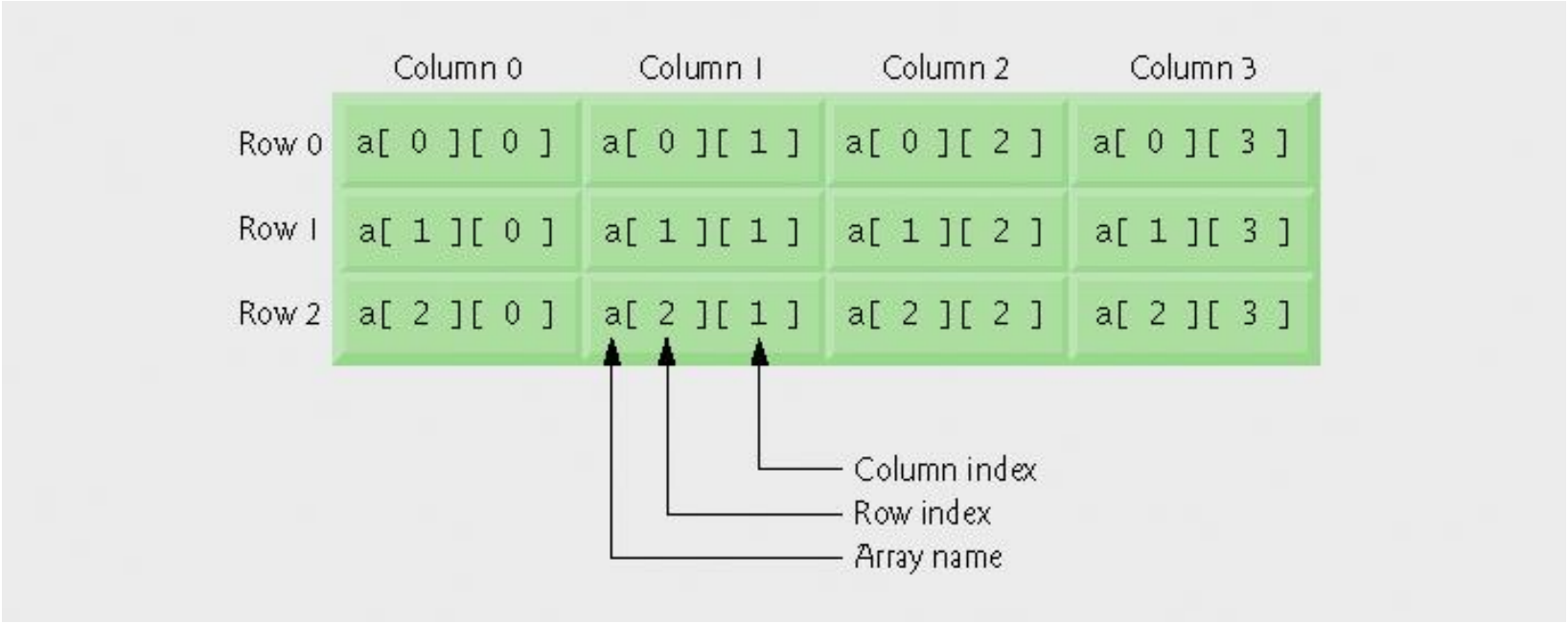


Fig. 7.16 | Two-dimensional array with three rows and four columns.

© 1992-2007 Pearson Education, Inc. All rights reserved.

7.9 Multidimensional Arrays (Cont.)

- **Arrays of one-dimensional array**
  - **Declaring two-dimensional array `b[2][2]`**

```
int b[][] = { { 1, 2 }, { 3, 4 } };
```

    - **`1` and `2` initialize `b[0][0]` and `b[0][1]`**
    - **`3` and `4` initialize `b[1][0]` and `b[1][1]`**

```
int b[][] = { { 1, 2 }, { 3, 4, 5 } };
```

    - **row `0` contains elements `1` and `2`**
    - **row `1` contains elements `3`, `4` and `5`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.9 Multidimensional Arrays (Cont.)

- **Two-dimensional arrays with rows of different lengths**
  - **Lengths of rows in array are not required to be the same**
    - **E.g.,** `int b[][] = { { 1, 2 }, { 3, 4, 5 } };`



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.9 Multidimensional Arrays (Cont.)

- **Creating two-dimensional arrays with array-creation expressions**
  - **3-by-4 array**

```
int b[][];
b = new int[ 3 ][ 4 ];
```

  - **Rows can have different number of columns**

```
int b[][];

b = new int[ 2 ][ ];   // create 2 rows
b[ 0 ] = new int[ 5 ]; // create 5 columns for row 0
b[ 1 ] = new int[ 3 ]; // create 3 columns for row 1
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

InitArray.java

(1 of 2)

Line 9

Line 10

```
1  // Fig. 7.17: InitArray.java
2  // Initializing two-dimensional arrays.
3
4  public class InitArray
5  {
6     // create and output two-dimensional arrays
7     public static void main( String args[] )
8     {
9        int array1[][] = { { 1, 2, 3 }, { 4, 5, 6 } };
10       int array2[][] = { { 1, 2 }, { 3 }, { 4, 5, 6 } };
11
12       System.out.println( "Values in array1 by row are" );
13       outputArray( array1 ); // displays array1 by row
14
15       System.out.println( "\nValues in array2 by row are" );
16       outputArray( array2 ); // displays array2 by row
17    } // end main
18
```

Use nested array initializers to initialize `array1`

Use nested array initializers of different lengths to initialize `array2`



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
19    // output rows and columns of a two-dimensional array
20    public static void outputArray( int array[][] )
21    {
22       // loop through array's rows
23       for ( int row = 0; row < array.length; row++ )
24       {
25          // loop through columns of current row
26          for ( int column = 0; column < array[ row ].length; column++ )
27             System.out.printf( "%d  ", array[ row ][ column ] );
28
29          System.out.println(); // start new line of output
30       } // end outer for
31    } // end method outputArray
32 } // end class InitArray
```

`array[row].length` returns number of columns associated with `row` subscript

Use double-bracket notation to access two-dimensional array values

```
Values in array1 by row are
1  2  3
4  5  6

Values in array2 by row are
1  2
3
4  5  6
```

InitArray.java

(2 of 2)

Line 26

Line 27

Program output



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.9 Multidimensional Arrays (Cont.)

- **Common multidimensional-array manipulations performed with `for` statements**
  - **Many common array manipulations use `for` statements**

E.g.,

```
for ( int column = 0; column < a[ 2 ].length; column++ )
      a[ 2 ][ column ] = 0;
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.10 Case Study: Class GradeBook Using a Two-Dimensional Array

- Class `GradeBook`
  - One-dimensional array
    - Store student grades on a single exam
  - Two-dimensional array
    - Store grades for a single student and for the class as a whole



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

(1 of 7)

Line 7

Line 10

```
1  // Fig. 7.18: GradeBook.java
2  // Grade book using a two-dimensional array to store grades.
3
4  public class GradeBook
5  {
6     private String courseName; // name of course this grade book represents
7     private int grades[][]; // two-dimensional array of student grades
8
9     // two-argument constructor initializes courseName and grades array
10    public GradeBook( String name, int gradesArray[][] )
11    {
12       courseName = name; // initialize courseName
13       grades = gradesArray; // store grades
14    } // end two-argument GradeBook constructor
15
16    // method to set the course name
17    public void setCourseName( String name )
18    {
19       courseName = name; // store the course name
20    } // end method setCourseName
21
22    // method to retrieve the course name
23    public String getCourseName()
24    {
25       return courseName;
26    } // end method getCourseName
27
```

Declare two-dimensional array grades

`GradeBook` constructor accepts a `String` and a two-dimensional array



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

(2 of 7)

```
28    // display a welcome message to the GradeBook user
29    public void displayMessage()
30    {
31       // getCourseName gets the name of the course
32       System.out.printf( "Welcome to the grade book for\n%s!\n\n",
33          getCourseName() );
34    } // end method displayMessage
35
36    // perform various operations on the data
37    public void processGrades()
38    {
39       // output grades array
40       outputGrades();
41
42       // call methods getMinimum and getMaximum
43       System.out.printf( "\n%s %d\n%s %d\n\n",
44          "Lowest grade in the grade book is", getMinimum(),
45          "Highest grade in the grade book is", getMaximum() );
46
47       // output grade distribution chart of all grades on all tests
48       outputBarChart();
49    } // end method processGrades
50
51    // find minimum grade
52    public int getMinimum()
53    {
54       // assume first element of grades array is smallest
55       int lowGrade = grades[ 0 ][ 0 ];
56
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
57       // loop through rows of grades array
58       for ( int studentGrades[] : grades )
59       {
60          // loop through columns of current row
61          for ( int grade : studentGrades
62          {
63             // if grade less than lowGra
64             if ( grade < lowGrade )
65                lowGrade = grade;
66          } // end inner for
67       } // end outer for
68
69       return lowGrade; // return lowest grade
70    } // end method getMinimum
71
72    // find maximum grade
73    public int getMaximum()
74    {
75       // assume first element of grades array is largest
76       int highGrade = grades[ 0 ][ 0 ];
77
```

Loop through rows of `grades` to find the lowest grade of any student

Outline

`GradeBook.java`

(3 of 7)

Lines 58-67



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
78       // loop through rows of grades array
79       for ( int studentGrades[] : grades )
80       {
81          // loop through columns of current row
82          for ( int grade : studentGrad
83          {
84             // if grade greater than h
85             if ( grade > highGrade )
86                highGrade = grade;
87          } // end inner for
88       } // end outer for
89
90       return highGrade; // return highest grade
91    } // end method getMaximum
92
93    // determine average grade for particular set of grades
94    public double getAverage( int setOfGrades[] )
95    {
96       int total = 0; // initialize total
97
98       // sum grades for one student
99       for ( int grade : setOfGrades )
100         total += grade;
101
102      // return average of grades
103      return (double) total / setOfGrades.length;
104   } // end method getAverage
105
```

Loop through rows of `grades` to find the highest grade of any student

Calculate a particular student's semester average

Outline

`GradeBook.java`

(4 of 7)

Lines 79-88

Lines 94-104



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

(5 of 7)

Lines 115-119

```
106    // output bar chart displaying overall grade distribution
107    public void outputBarChart()
108    {
109       System.out.println( "Overall grade distribution:" );
110
111       // stores frequency of grades in each range of 10 grades
112       int frequency[] = new int[ 11 ];
113
114       // for each grade in GradeBook, increment the appropriate frequency
115       for ( int studentGrades[] : grades )
116       {
117          for ( int grade : studentGrades )
118             ++frequency[ grade / 10 ];
119       } // end outer for
120
121       // for each grade frequency, print bar in chart
122       for ( int count = 0; count < frequency.length; count++ )
123       {
124          // output bar label ( "00-09: ", ..., "90-99: ", "100: " )
125          if ( count == 10 )
126             System.out.printf( "%5d: ", 100 );
127          else
128             System.out.printf( "%02d-%02d: ",
129                count * 10, count * 10 + 9  );
130
131          // print bar of asterisks
132          for ( int stars = 0; stars < frequency[ count ]; stars++ )
133             System.out.print( "*" );
```

Calculate the distribution of all student grades



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

(6 of 7)

```
134
135          System.out.println(); // start a new line of output
136       } // end outer for
137    } // end method outputBarChart
138
139    // output the contents of the grades array
140    public void outputGrades()
141    {
142       System.out.println( "The grades are:\n" );
143       System.out.print( "            " ); // align column heads
144
145       // create a column heading for each of the tests
146       for ( int test = 0; test < grades[ 0 ].length; test++ )
147          System.out.printf( "Test %d  ", test + 1 );
148
149       System.out.println( "Average" ); // student average column heading
150
151       // create rows/columns of text representing array grades
152       for ( int student = 0; student < grades.length; student++ )
153       {
154          System.out.printf( "Student %2d", student + 1 );
155
156          for ( int test : grades[ student ] ) // output student's grades
157             System.out.printf( "%8d", test );
158
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
159          // call method getAverage to calculate student's average grade;
160          // pass row of grades as the argument to getAverage
161          double average = getAverage( grades[ student ] );
162          System.out.printf( "%9.2f\n", average );
163       } // end outer for
164    } // end method outputGrades
165 } // end class GradeBook
```

Outline

GradeBook.java

(7 of 7)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 7.19: GradeBookTest.java
2  // Creates GradeBook object using a two-dimensional array of grades.
3
4  public class GradeBookTest
5  {
6     // main method begins program execution
7     public static void main( String args[] )
8     {
9        // two-dimensional array of student grades
10       int gradesArray[][] = { { 87, 96, 70 },
11                               { 68, 87, 90 },
12                               { 94, 100, 90 },
13                               { 100, 81, 82 },
14                               { 83, 65, 85 },
15                               { 78, 87, 65 },
16                               { 85, 75, 83 },
17                               { 91, 94, 100 },
18                               { 76, 72, 84 },
19                               { 87, 93, 73 } };
20
21       GradeBook myGradeBook = new GradeB
22          "CS101 Introduction to Java Pro
23       myGradeBook.displayMessage();
24       myGradeBook.processGrades();
25    } // end main
26 } // end class GradeBookTest
```

Declare `gradesArray` as 10-by-3 array

Each row represents a student; each column represents an exam grade

.java

(1 of 2)

Lines 10-19



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
Welcome to the grade book for
CS101 Introduction to Java Programming!

The grades are:

            Test 1  Test 2  Test 3  Average
Student  1      87      96      70    84.33
Student  2      68      87      90    81.67
Student  3      94     100      90    94.67
Student  4     100      81      82    87.67
Student  5      83      65      85    77.67
Student  6      78      87      65    76.67
Student  7      85      75      83    81.00
Student  8      91      94     100    95.00
Student  9      76      72      84    77.33
Student 10      87      93      73    84.33

Lowest grade in the grade book is 65
Highest grade in the grade book is 100

Overall grade distribution:
00-09:
10-19:
20-29:
```

```
30-39:
40-49:
50-59:
60-69: ***
70-79: ******
80-89: ***********
90-99: *******
  100: ***
```

Outline

GradeBookTest

.java

(2 of 2)

Program output



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.11 Variable-Length Argument Lists

- **Variable-length argument lists**
  - **Unspecified number of arguments**
  - **Use ellipsis (...) in method's parameter list**
    - **Can occur only once in parameter list**
    - **Must be placed at the end of parameter list**
  - **Array whose elements are all of the same type**



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 7.20: VarargsTest.java
2  // Using variable-length argument lists.
3
4  public class VarargsTest
5  {
6     // calculate average
7     public static double average( double... numbers )
8     {
9        double total = 0.0; // initialize total
10
11       // calculate total using the enha
12       for ( double d : numbers )
13          total += d;
14
15       return total / numbe
16    } // end method average
17
18    public static void main( String args[] )
19    {
20       double d1 = 10.0;
21       double d2 = 20.0;
22       double d3 = 30.0;
23       double d4 = 40.0;
24
```

Method `average` receives a variable length sequence of `double`s

Calculate the total of the `double`s in the array

Access `numbers.length` to obtain the size of the `numbers` array

Outline

`VarargsTest.java`

(1 of 2)

Line 7

Lines 12-13

Line 15



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
25       System.out.printf( "d1 = %.1f\nd2 = %.1f\nd3 = %.1f\nd4 = %.1f\n\n",
26          d1, d2, d3, d4 );
27
28       System.out.printf( "Average of d1 and d2 is %.1f\n",
29          average( d1, d2 ) );
30       System.out.printf( "Average of d1, d2 and d
31          average( d1, d2, d3 ) );
32       System.out.printf( "Average of d1, d2, d3 and d4 is %.1f\n",
33          average( d1, d2, d3, d4 ) );
34    } // end main
35 } // end class VarargsTest
```

```
d1 = 10.0
d2 = 20.0
d3 = 30.0
d4 = 40.0

Average of d1 and d2 is 15.0
Average of d1, d2 and d3 is 20.0
Average of d1, d2, d3 and d4 is 25.0
```

Invoke method average with two arguments

Invoke method average with three arguments

Invoke method average with four arguments

Outline

VarargsTest.java

2)

Line 29

Line 31

Line 33

Program output



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 7.6

Placing an ellipsis in the middle of a method parameter list is a syntax error. An ellipsis may be placed only at the end of the parameter list.



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.12 Using Command-Line Arguments

- **Command-line arguments**
  - **Pass arguments from the command line**
    - `String args[]`
  - **Appear after the class name in the `java` command**
    - `java MyClass a b`
  - **Number of arguments passed in from command line**
    - `args.length`
  - **First command-line argument**
    - `args[ 0 ]`



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

InitArray.java

(1 of 2)

Line 6

Line 9

Line 16

Lines 20-21

Lines 24-25

```
1  // Fig. 7.21: InitArray.java
2  // Using command-line arguments to initialize an array.
3
4  public class InitArray
5  {
6     public static void main( String args[] )
7     {
8        // check number of command-line arguments
9        if ( args.length != 3 )
10          System.out.println(
11             "Error: Please re-enter                    " +
12             "an array                                  );
13       else
14       {
15          // get array size from first command-line argument
16          int arrayLength = Integer.parseInt( args[ 0 ] );
17          int array[] = new int[ arrayLength ]; // create array
18
19          // get initial value and increment from co
20          int initialValue = Integer.parseInt( args[
21          int increment = Integer.parseInt( args[ 2 ] );
22
23          // calculate value for each array element
24          for ( int counter = 0; counter < array.le
25             array[ counter ] = initialValue + incr
26
27          System.out.printf( "%s%8s\n", "Index", "Value" );
28
```

Array `args` stores command-line arguments

Check number of arguments passed in from the command line

Obtain first command-line argument

Obtain second and third command-line arguments

Calculate the value for each array element based on command-line arguments

© 1992-2007 Pearson Education, Inc. All rights reserved.

```
29          // display array index and value
30          for ( int counter = 0; counter < array.length; counter++ )
31             System.out.printf( "%5d%8d\n", counter, array[ counter ] );
32       } // end else
33    } // end main
34 } // end class InitArray
```

```
java InitArray
Error: Please re-enter the entire command, including
an array size, initial value and increment.
```

Missing command-line arguments

```
java Ini
Index   Value
    0       0
    1       4
    2       8
    3      12
    4      16
```

Three command-line arguments are 5, 0 and 4

```
java InitArray 10 1 2
Index   Value
    0       1
    1       3
    2       5
    3       7
    4       9
    5      11
    6      13
    7      15
    8      17
    9      19
```

Three command-line arguments are 10, 1 and 2

Outline

InitArray.java

(2 of 2)

Program output



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.13 (Optional) GUI and Graphics Case Study: Drawing Arcs

- **Draw rainbow**
  - **Use arrays**
  - **Use repetition statement**
  - **Use `Graphics` method `fillArc`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

DrawRainbow

.java

(1 of 2)

Line 22

```
1  // Fig. 7.22: DrawRainbow.java
2  // Demonstrates using colors in an array.
3  import java.awt.Color;
4  import java.awt.Graphics;
5  import javax.swing.JPanel;
6
7  public class DrawRainbow extends JPanel
8  {
9     // Define indigo and violet
10    final Color VIOLET = new Color( 128, 0, 128 );
11    final Color INDIGO = new Color( 75, 0, 130 );
12
13    // colors to use in the rainbow, starting from the innermost
14    // The two white entries result in an empty arc in the center
15    private Color colors[] =
16       { Color.WHITE, Color.WHITE, VIOLET, INDIGO, Color.BLUE,
17         Color.GREEN, Color.YELLOW, Color.ORANGE, Color.RED };
18
19    // constructor
20    public DrawRainbow()
21    {
22       setBackground( Color.WHITE ); // set the background to white
23    } // end DrawRainbow constructor
24
25    // draws a rainbow using
26    public void paintComponent( Graphics g )
27    {
28       super.paintComponent( g );
29
30       int radius = 20; // radius of an arch
```

Set component background to white



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
31
32       // draw the rainbow near the bottom-center
33       int centerX = getWidth() / 2;
34       int centerY = getHeight() - 10;
35
36       // draws filled arcs starting with the outermost
37       for ( int counter = colors.length; counter > 0; counter-- )
38       {
39          // set the color for the current arc
40          g.setColor( colors[ counter - 1 ] );
41
42          // fill the arc from 0 to 180 degrees
43          g.fillArc( centerX - counter * radius,
44             centerY - counter * radius,
45             counter * radius * 2, counter * radius * 2, 0, 180 );
46       } // end for
47    } // end method paintComponent
48 } // end class DrawRainbow
```

Draw a filled semicircle

Outline

DrawRainbow

.java

(2 of 2)

Lines 43-45

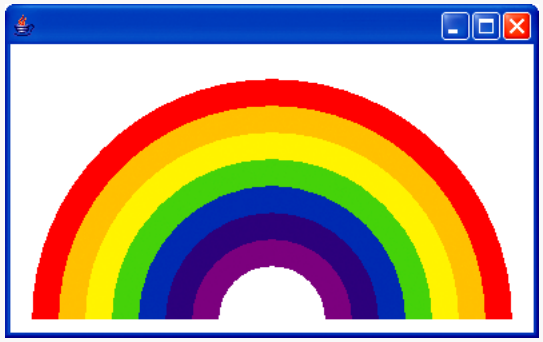


© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

DrawRainbowTest.java

```
1  // Fig. 7.23: DrawRainbowTest.java
2  // Test application to display a rainbow.
3  import javax.swing.JFrame;
4
5  public class DrawRainbowTest
6  {
7     public static void main( String args[] )
8     {
9        DrawRainbow panel = new DrawRainbow();
10       JFrame application = new JFrame();
11
12       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
13       application.add( panel );
14       application.setSize( 400, 250 );
15       application.setVisible( true );
16    } // end main
17 } // end class DrawRainbowTest
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

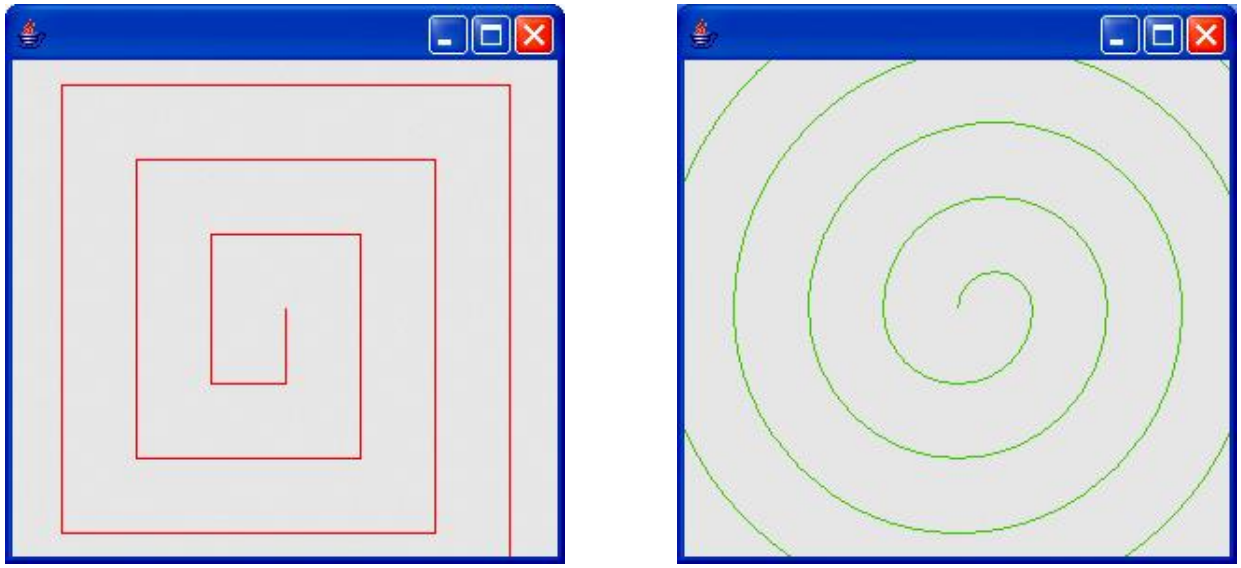


Fig. 7.24 | Drawing a spiral using `drawLine` (left) and `drawArc` (right).

© 1992-2007 Pearson Education, Inc. All rights reserved.

7.14 (Optional) Software Engineering Case Study: Collaboration Among Objects

- **Collaborations**
  - **When objects communicate to accomplish task**
    - **Accomplished by invoking operations (methods)**
  - **One object sends a message to another object**



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.14 (Optional) Software Engineering Case Study (Cont.)

- **Identifying the collaborations in a system**
  - **Read requirements document to find**
    - **What ATM should do to authenticate a use**
    - **What ATM should do to perform transactions**
  - **For each action, decide**
    - **Which objects must interact**
      - **Sending object**
      - **Receiving object**



© 1992-2007 Pearson Education, Inc. All rights reserved.

An object of class…	sends the message…	to an object of class…
ATM	displayMessage	Screen
	getInput	Keypad
	authenticateUser	BankDatabase
	execute	BalanceInquiry
	execute	Withdrawal
	Execute	Deposit
BalanceInquiry	getAvailableBalance	BankDatabase
	getTotalBalance	BankDatabase
	displayMessage	Screen
Withdrawal	displayMessage	Screen
	getInput	Keypad
	getAvailableBalance	BankDatabase
	isSufficientCashAvailable	CashDispenser
	debit	BankDatabase
	dispenseCash	CashDispenser
Deposit	displayMessage	Screen
	getInput	Keypad
	isEnvelopeReceived	DepositSlot
	Credit	BankDatabase
BankDatabase	validatePIN	Account
	getAvailableBalance	Account
	getTotalBalance	Account
	debit	Account
	Credit	Account

Fig. 7.25 | Collaborations in the ATM system.



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.14 (Optional) Software Engineering Case Study (Cont.)

- **Interaction Diagrams**
  - **Used to model interactions in the UML**
  - **Communication diagrams**
    - **Also called collaboration diagrams**
    - **Emphasize which objects participate in collaborations**
  - **Sequence diagrams**
    - **Emphasize when messages are sent between objects**



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.14 (Optional) Software Engineering Case Study (Cont.)

- **Communication diagrams**
  - **Objects**
    - **Modeled as rectangles**
    - **Contain names in the form `objectName : className`**
  - **Objects are connected with solid lines**
  - **Messages are passed alone these lines in the direction shown by arrows**
  - **Name of message appears next to the arrow**



© 1992-2007 Pearson Education, Inc. All rights reserved.



Fig. 7.26 | Communication diagram of the ATM executing a balance inquiry.



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.14 (Optional) Software Engineering Case Study (Cont.)

- **Sequence of messages in a communication diagram**
  - **Appear to the left of a message name**
  - **Indicate the order in which the message is passed**
  - **Process in numerical order from least to greatest**



© 1992-2007 Pearson Education, Inc. All rights reserved.

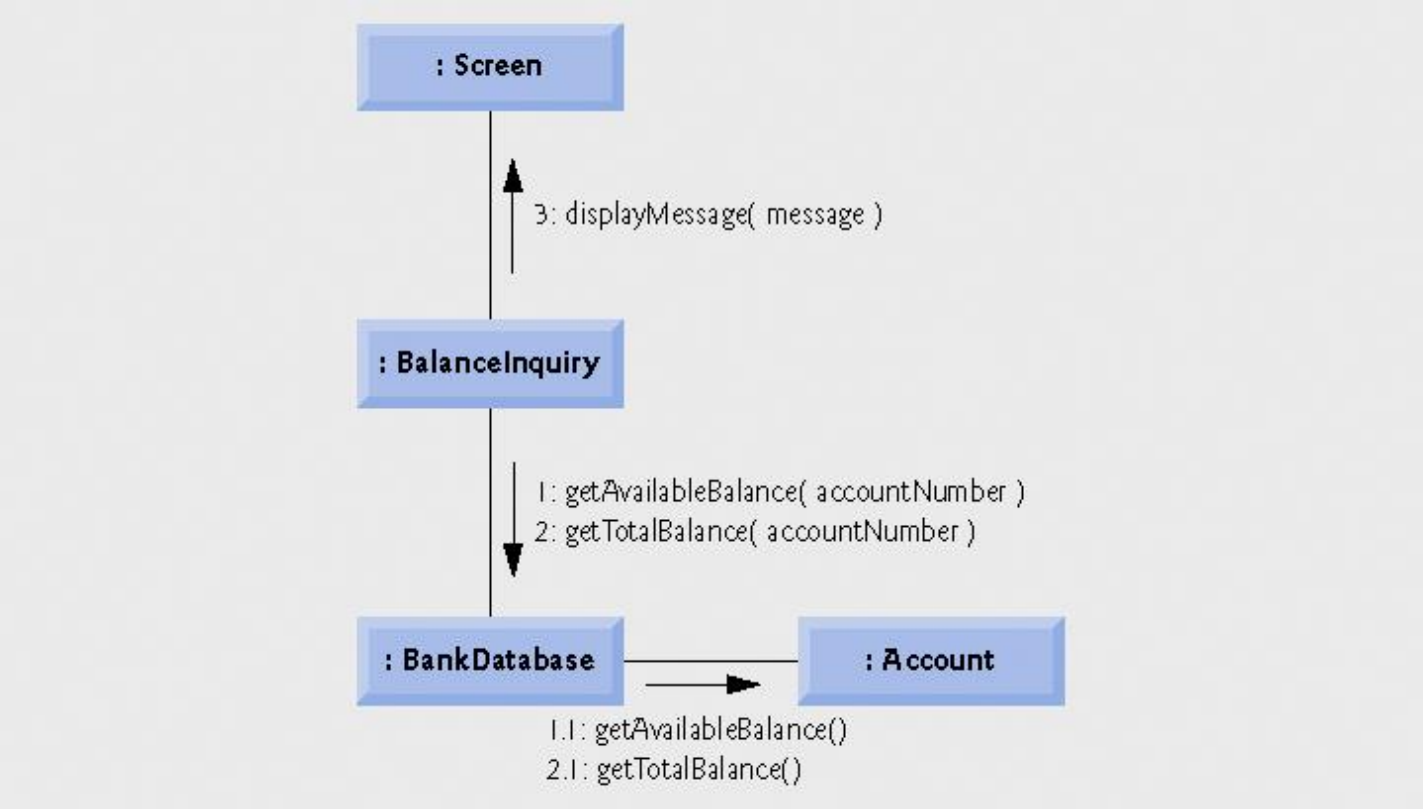


Fig. 7.27 | Communication diagram for executing a balance inquiry.



© 1992-2007 Pearson Education, Inc. All rights reserved.

7.14 (Optional) Software Engineering Case Study (Cont.)

- **Sequence diagrams**
  - **Help model the timing of collaborations**
  - **Lifeline**
    - **Dotted line extending down from an object's rectangle**
      - **Represents the progression of time**
  - **Activation**
    - **Thin vertical rectangle**
      - **Indicates that an object is executing**



© 1992-2007 Pearson Education, Inc. All rights reserved.

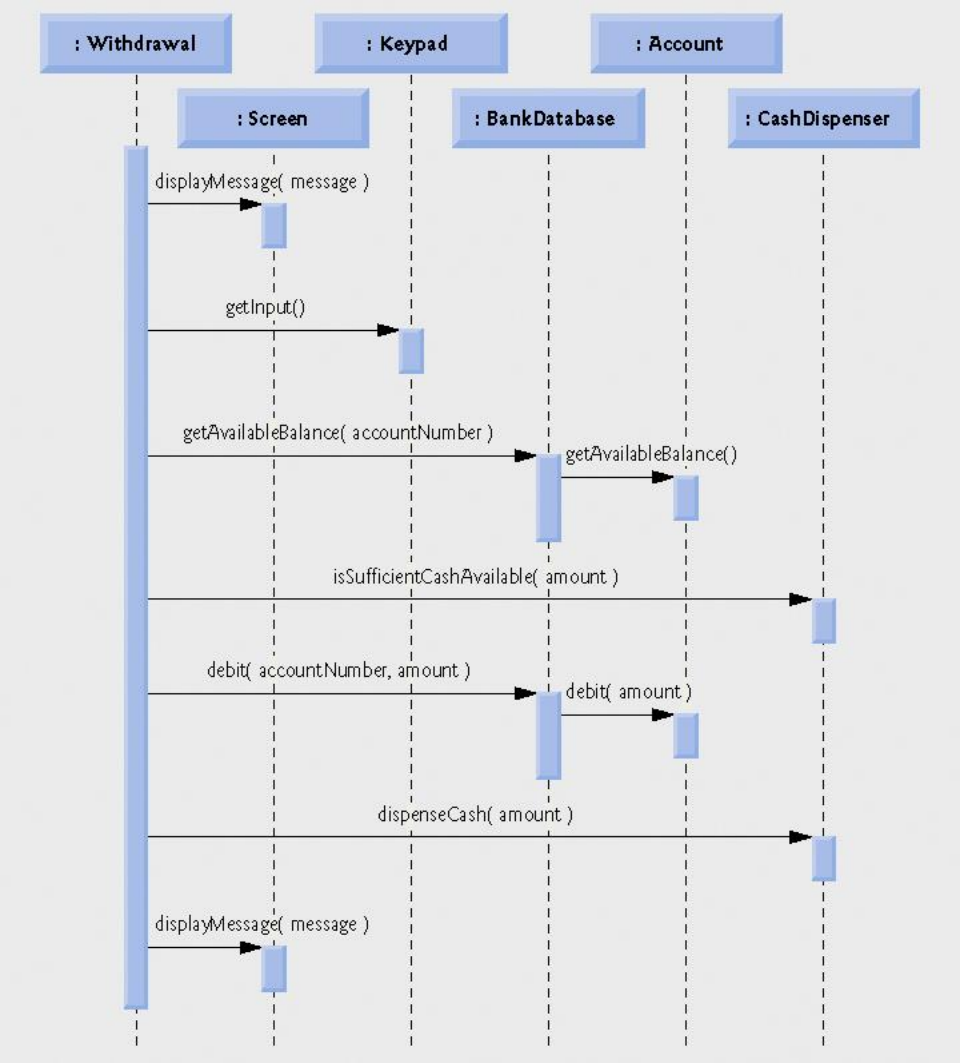


Fig. 7.28 | Sequence diagram that models a `Withdrawal` executing.

© 1992-2007 Pearson Education, Inc. All rights reserved.

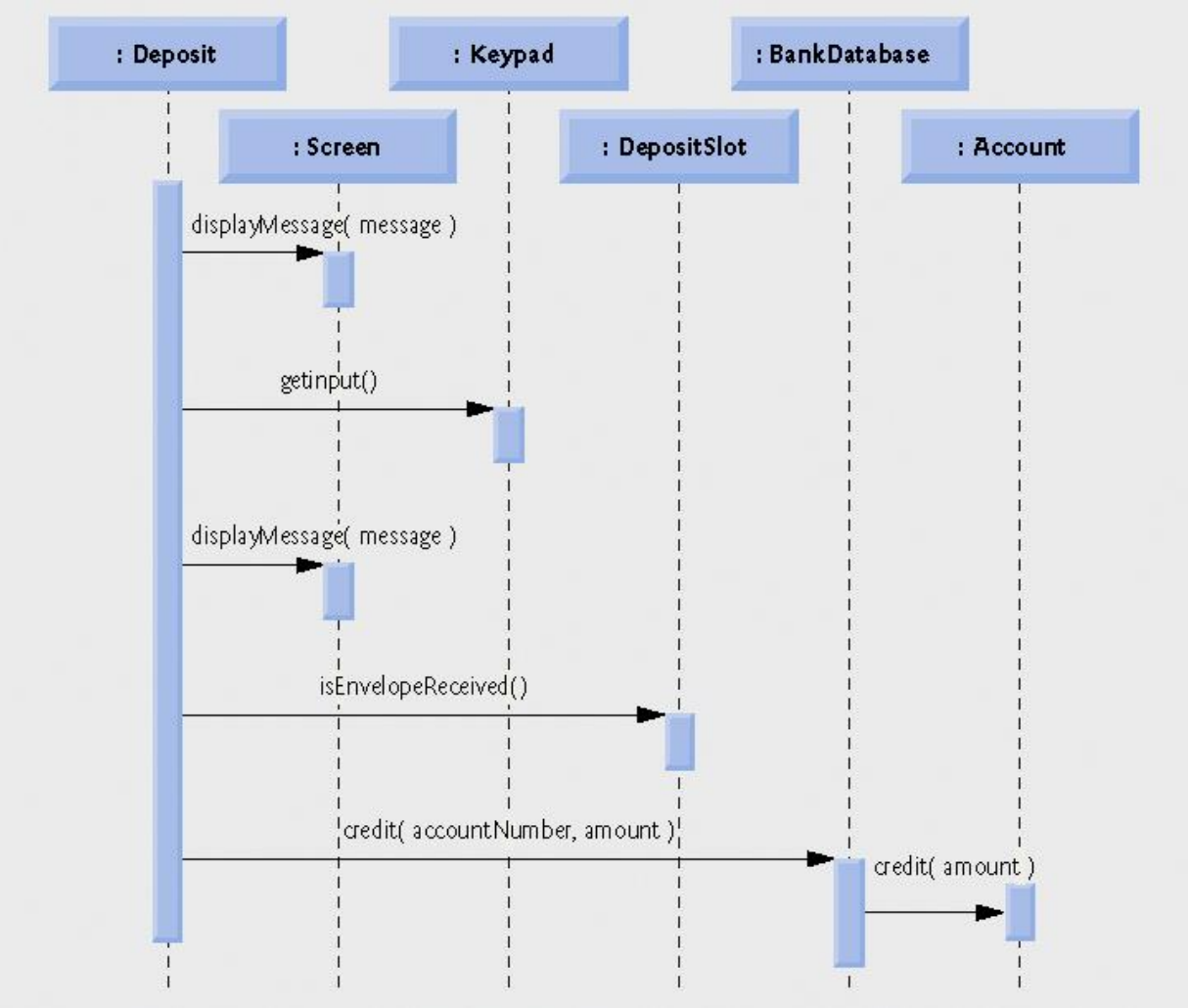


Fig. 7.29 | Sequence diagram that models a `Deposit` executing.

© 1992-2007 Pearson Education, Inc. All rights reserved.

3

Introduction to Classes and Objects



© 1992-2007 Pearson Education, Inc. All rights reserved.

You will see something new. Two things. And I call them Thing One and Thing Two.

— Dr. Theodor Seuss Geisel

Nothing can have value without being an object of utility.

— Karl Marx

Your public servants serve you right.

— Adlai E. Stevenson

Knowing how to answer one who speaks,

To reply to one who sends a message.

— Amenemope



© 1992-2007 Pearson Education, Inc. All rights reserved.

OBJECTIVES

In this chapter you will learn:

- What classes, objects, methods and instance variables are.
- How to declare a class and use it to create an object.
- How to declare methods in a class to implement the class's behaviors.
- How to declare instance variables in a class to implement the class's attributes.
- How to call an object's method to make that method perform its task.
- The differences between instance variables of a class and local variables of a method.
- How to use a constructor to ensure that an object's data is initialized when the object is created.
- The differences between primitive and reference types.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

3.1 Introduction

3.2 Classes, Objects, Methods and Instance Variables

3.3 Declaring a Class with a Method and Instantiating an Object of a Class

3.4 Declaring a Method with a Parameter

3.5 Instance Variables, *set* Methods and *get* Methods

3.6 Primitive Types vs. Reference Types

3.7 Initializing Objects with Constructors

3.8 Floating-Point Numbers and Type `double`

3.9 (Optional) GUI and Graphics Case Study: Using Dialog Boxes

3.10 (Optional) Software Engineering Case Study: Identifying the Classes in a Requirements Document

3.11 Wrap-Up



© 1992-2007 Pearson Education, Inc. All rights reserved.

What is OOP?

- OOP stands for **Object-Oriented Programming**.

Object-oriented programming has several advantages :

1. OOP is faster and easier to execute
2. OOP provides a clear structure for the programs
3. OOP helps to keep the Java code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
4. OOP makes it possible to create full reusable applications with less code and shorter development time



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.1 Introduction

- **Classes**
- **Floating-Point numbers**



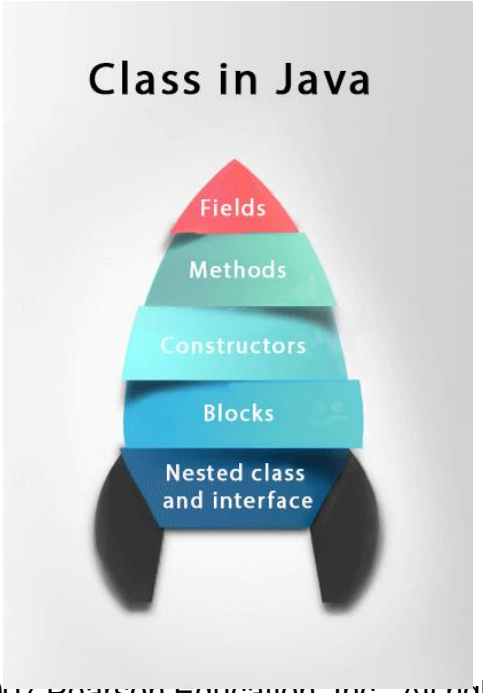
© 1992-2007 Pearson Education, Inc. All rights reserved.

3.1 What is a class

- A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

A class in Java can contain:

- **Fields**
- **Methods**
- **Constructors**
- **Blocks**
- **Nested class and interfac**



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.1 Introduction

```
class <class_name>{
    field;
    method;
}
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.1 Create an Object

In Java, an object is created from a class. We have already created the class named Main,

so now we can use this to create objects.

To create an object of Main, specify the class name, followed by the object name, and use the keyword new:

```
ClassName  myObj = new ClassName();
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.2 Classes, Objects, Methods and Instance Variables

- **Class provides one or more methods**
- **Method represents task in a program**
  - **Describes the mechanisms that actually perform its tasks**
  - **Hides from its user the complex tasks that it performs**
  - **Method call tells method to perform its task**



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.2 Classes, Objects, Methods and Instance Variables (Cont.)

- **Classes contain one or more attributes**
  - **Specified by instance variables**
  - **Carried with the object as it is used**



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.3 Declaring a Class with a Method and Instantiating an Object of a Class

- **Each class declaration that begins with keyword `public` must be stored in a file that has the same name as the class and ends with the `.java` file-name extension.**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Class GradeBook

- keyword `public` is an access modifier
- Class declarations include:
  - Access modifier
  - Keyword `class`
  - Pair of left and right braces



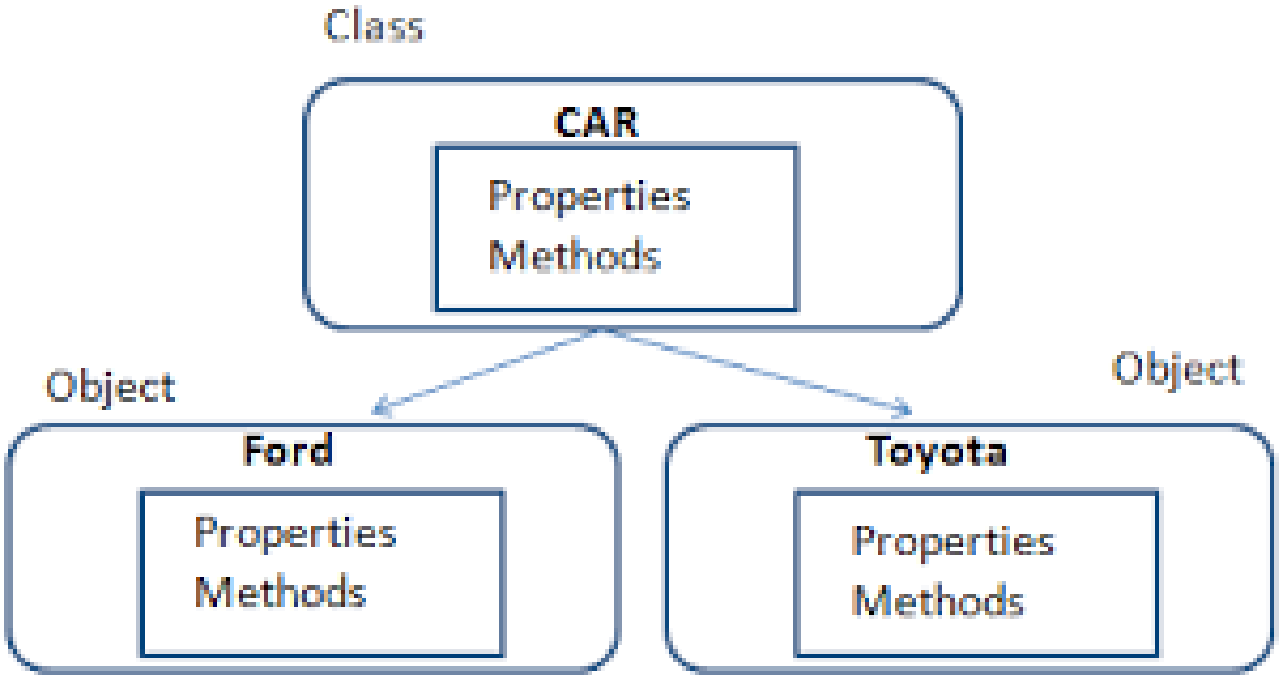
© 1992-2007 Pearson Education, Inc. All rights reserved.

Class GradeBook

- **Method declarations**
  - **Keyword `public` indicates method is available to public**
  - **Keyword `void` indicates no return type**
  - **Access modifier, return type, name of method and parentheses comprise method header**

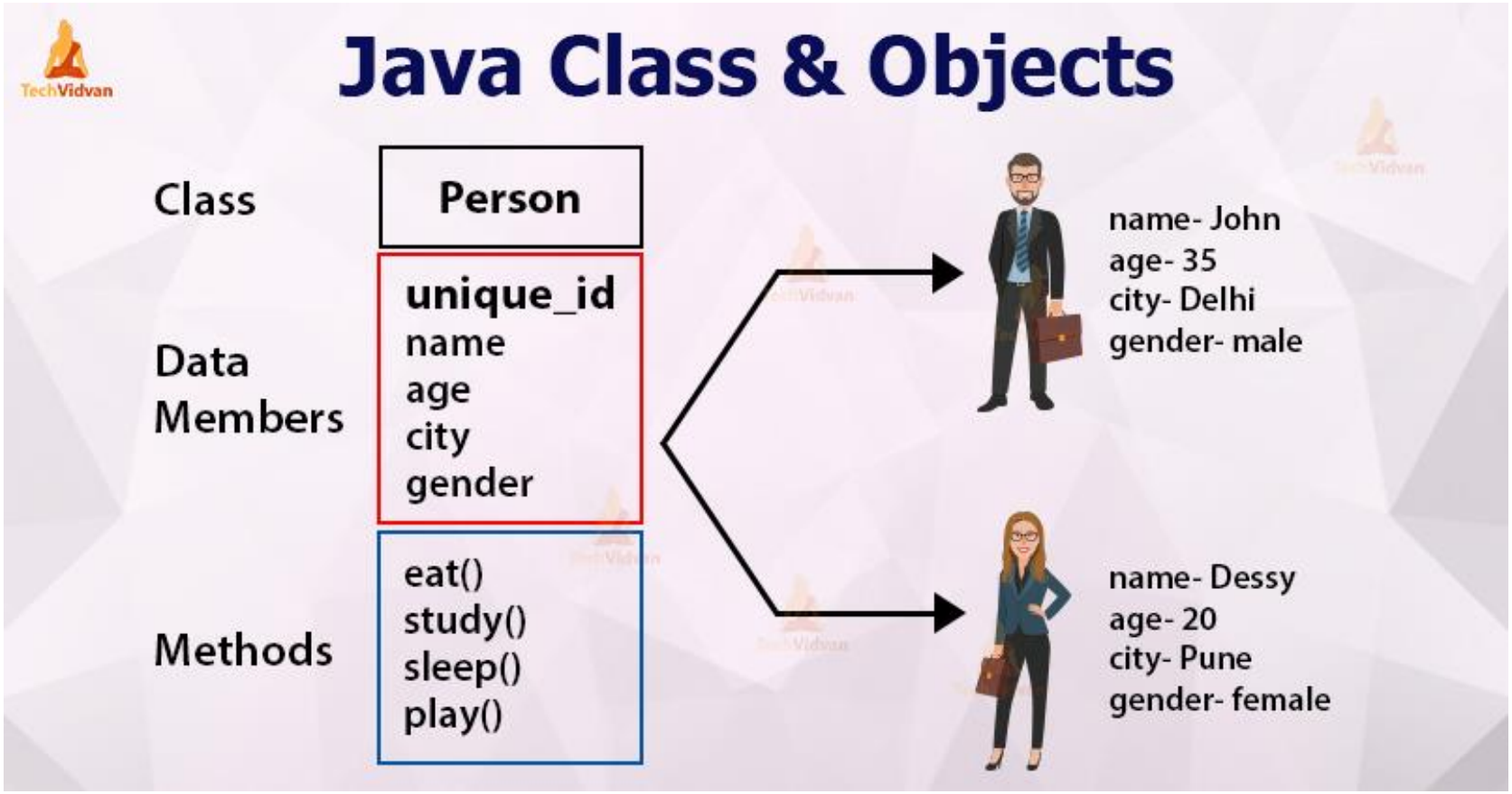


© 1992-2007 Pearson Education, Inc. All rights reserved.





© 1992-2007 Pearson Education, Inc. All rights reserved.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Class VS Object

	Class	Object
Definition	Class is mechanism of binding data members and associated methods in a single unit.	Instance of class or variable of class.
Existence	It is logical existence	It is physical existence
Memory Allocation	Memory space is not allocated , when it is created.	Memory space is allocated, when it is created.
Declaration/definition	Definition is created once.	it is created many time as you require.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

```
1  // Fig. 3.1: GradeBook.java
2  // Class declaration with one method.
3
4  public class GradeBook
5  {
6     // display a welcome message to the GradeBook
7     public void displayMessage()
8     {
9        System.out.println( "Welcome to the Grade Book!" );
10    } // end method displayMessage
11
12 } // end class GradeBook
```

Print line of text to output



© 1992-2007 Pearson Education, Inc. All rights reserved.

Class GradeBookTest

- **Java is extensible**
  - **Programmers can create new classes**
- **Class instance creation expression**
  - **Keyword** `new`
  - **Then name of class to create and parentheses**
- **Calling a method**
  - **Object name, then dot separator (.)**
  - **Then method name and parentheses**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBookTest.java

```
1  // Fig. 3.2: GradeBookTest.java
2  // Create a GradeBook object and call its displayMessage method.
3
4  public class GradeBookTest
5  {
6     // main method begins program execution
7     public static void main( String args[] )
8     {
9        // create a GradeBook object and assign it
10       GradeBook myGradeBook = new GradeBook();
11
12       // call myGradeBook's displayMessage metho
13       myGradeBook.displayMessage();
14    } // end main
15
16 } // end class GradeBookTest
```

Use class instance creation expression to create object of class `GradeBook`

Call method `displayMessage` using `GradeBook` object

```
Welcome to the Grade Book!
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Compiling an Application with Multiple Classes

- **Compiling multiple classes**
  - **List each `.java` file in the compilation command and separate them with spaces**
  - **Compile with `*.java` to compile all `.java` files in that directory**



© 1992-2007 Pearson Education, Inc. All rights reserved.

UML Class Diagram for Class GradeBook

- **UML class diagrams**
  - **Top compartment contains name of the class**
  - **Middle compartment contains class's attributes or instance variables**
  - **Bottom compartment contains class's operations or methods**
    - **Plus sign indicates `public` methods**



© 1992-2007 Pearson Education, Inc. All rights reserved.

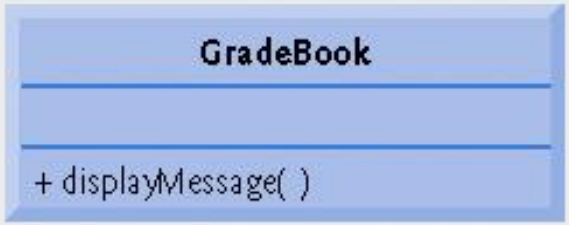


Fig. 3.3 | UML class diagram indicating that class `GradeBook` has a `public` `displayMessage` operation.



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.4 Declaring a Method with a Parameter

- **Method parameters**
  - **Additional information passed to a method**
  - **Supplied in the method call with arguments**



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.4 Declaring a Method with a Parameter

- `Scanner` **methods**
  - `nextLine` **reads next line of input**
  - `next` **reads next word of input**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

```
1  // Fig. 3.4: GradeBook.java
2  // Class declaration with a method that has a parameter.
3
4  public class GradeBook
5  {
6     // display a welcome message to the GradeBook user
7     public void displayMessage( String courseName )
8     {
9        System.out.printf( "Welcome to the grade book for\n%s!\n",
10          courseName );
11    } // end method displayMessage
12
13 } // end class GradeBook
```

Call `printf` method with `courseName` argument



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBookTest.java

```
1  // Fig. 3.5: GradeBookTest.java
2  // Create GradeBook object and pass a String to
3  // its displayMessage method.
4  import java.util.Scanner; // program uses Scanner
5
6  public class GradeBookTest
7  {
8     // main method begins program execution
9     public static void main( String args[] )
10    {
11       // create Scanner to obtain input from command window
12       Scanner input = new Scanner( System.in );
13
14       // create a GradeBook object and assign it
15       GradeBook myGradeBook = new GradeBook();
16
17       // prompt for and input course name
18       System.out.println( "Please enter the course name:" );
19       String nameOfCourse = input.nextLine(); //
20       System.out.println(); // outputs a blank l
21
22       // call myGradeBook's displayMessage method
23       // and pass nameOfCourse as an argument
24       myGradeBook.displayMessage( nameOfCourse );
25    } // end main
26
27 } // end class GradeBookTest
```

Call `nextLine` method to read a line of input

Call `displayMessage` with an argument

```
Please enter the course name:
CS101 Introduction to Java Programming

Welcome to the grade book for
CS101 Introduction to Java Programming!
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 3.1

Normally, objects are created with `new`. One exception is a string literal that is contained in quotes, such as `"hello"`. String literals are references to `String` objects that are implicitly created by Java.



© 1992-2007 Pearson Education, Inc. All rights reserved.

More on Arguments and Parameters

- **Parameters specified in method's parameter list**
  - **Part of method header**
  - **Uses a comma-separated list**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 3.2

A compilation error occurs if the number of arguments in a method call does not match the number of parameters in the method declaration.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 3.3

A compilation error occurs if the types of the arguments in a method call are not consistent with the types of the corresponding parameters in the method declaration.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Updated UML Class Diagram for Class GradeBook

- **UML class diagram**
  - **Parameters specified by parameter name followed by a colon and parameter type**



© 1992-2007 Pearson Education, Inc. All rights reserved.

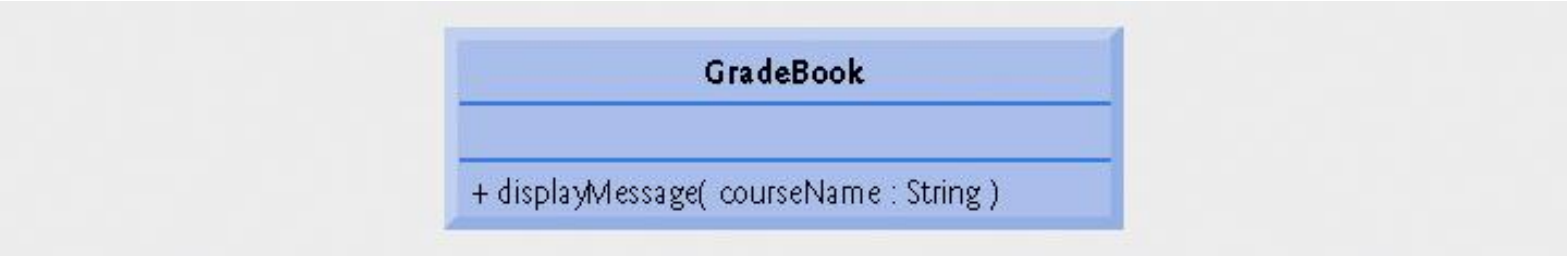


Fig. 3.6 | UML class diagram indicating that class GradeBook has a displayMessage operation with a courseName parameter of UML type String.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Notes on `Import` Declarations

- `java.lang` **is implicitly imported into every program**
- **Default package**
  - **Contains classes compiled in the same directory**
  - **Implicitly imported into source code of other files in directory**
- **Imports unnecessary if fully-qualified names are used**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 3.2

The Java compiler does not require `import` declarations in a Java source code file if the fully qualified class name is specified every time a class name is used in the source code. But most Java programmers consider using fully qualified names to be cumbersome, and instead prefer to use `import` declarations.



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.5 Instance Variables, *set* Methods and *get* Methods

- **Variables declared in the body of method**
  - **Called local variables**
  - **Can only be used within that method**
- **Variables declared in a class declaration**
  - **Called fields or instance variables**
  - **Each object of the class has a separate instance of the variable**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 3.7: GradeBook.java
2  // GradeBook class that contains a courseName instance variable
3  // and methods to set and get its value.
4
5  public class GradeBook
6  {
7     private String courseName; // course name for this GradeBook
8
9     // method to set the course name
10    public void setCourseName( String name )
11    {
12       courseName = name; // store the course name
13    } // end method setCourseName
14
15    // method to retrieve the course name
16    public String getCourseName()
17    {
18       return courseName;
19    } // end method getCourseName
20
21    // display a welcome message to the GradeBook user
22    public void displayMessage()
23    {
24       // this statement calls getCourseName to get the
25       // name of the course this GradeBook represents
26       System.out.printf( "Welcome to the grade book for\n%s!\n",
27          getCourseName() );
28    } // end method displayMessage
29
30 } // end class GradeBook
```

Instance variable `courseName`

set method for `courseName`

get method for `courseName`

Call *get* method

GradeBook.java



© 1992-2007 Pearson Education, Inc. All rights reserved.

Access Modifiers `public` and `private`

- **private keyword**
  - **Used for most instance variables**
  - **`private` variables and methods are accessible only to methods of the class in which they are declared**
  - **Declaring instance variables `private` is known as data hiding**
- **Return type**
  - **Indicates item returned by method**
  - **Declared in method header**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 3.3

Precede every field and method declaration with an access modifier. As a rule of thumb, instance variables should be declared `private` and methods should be declared `public`. (We will see that it is appropriate to declare certain methods `private`, if they will be accessed only by other methods of the class.)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Good Programming Practice 3.1

We prefer to list the fields of a class first, so that, as you read the code, you see the names and types of the variables before you see them used in the methods of the class. It is possible to list the class's fields anywhere in the class outside its method declarations, but scattering them tends to lead to hard-to-read code.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Good Programming Practice 3.2

Place a blank line between method declarations to separate the methods and enhance program readability.



© 1992-2007 Pearson Education, Inc. All rights reserved.

`GradeBookTest` Class That Demonstrates Class `GradeBook`

- **Default initial value**
  - **Provided for all fields not initialized**
  - **Equal to `null` for `String`s**



© 1992-2007 Pearson Education, Inc. All rights reserved.

set and *get* methods

- `private` **instance variables**
  - **Cannot be accessed directly by clients of the object**
  - **Use *set* methods to alter the value**
  - **Use *get* methods to retrieve the value**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBookTest.java

(1 of 2)

```
1  // Fig. 3.8: GradeBookTest.java
2  // Create and manipulate a GradeBook object.
3  import java.util.Scanner; // program uses Scanner
4
5  public class GradeBookTest
6  {
7     // main method begins program execution
8     public static void main( String args[] )
9     {
10       // create Scanner to obtain input from command window
11       Scanner input = new Scanner( System.in );
12
13       // create a GradeBook object and assign it to myGradeBook
14       GradeBook myGradeBook = new GradeBook();
15
16       // display initial value of courseName
17       System.out.printf( "Initial course name is: %s\n\n",
18          myGradeBook.getCourseName() );
19
```

Call *get* method for `courseName`



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
20       // prompt for and read course name
21       System.out.println( "Please enter the course name:" );
22       String theName = input.nextLine(); // read a line of text
23       myGradeBook.setCourseName( theName ); // s
24       System.out.println(); // outputs a blank line
25
26       // display welcome message after specifying course name
27       myGradeBook.displayMessage();
28    } // end main
29
30 } // end class GradeBookTest
```

Call *set* method for `courseName`

Call `displayMessage`

GradeBookTest.java

(2 of 2)

```
Initial course name is: null

Please enter the course name:
CS101 Introduction to Java Programming

Welcome to the grade book for
CS101 Introduction to Java Programming!
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

GradeBook's UML Class Diagram with an Instance Variable and *set* and *get* Methods

- **Attributes**
  - **Listed in middle compartment**
  - **Attribute name followed by colon followed by attribute type**
- **Return type of a method**
  - **Indicated with a colon and return type after the parentheses after the operation name**



© 1992-2007 Pearson Education, Inc. All rights reserved.

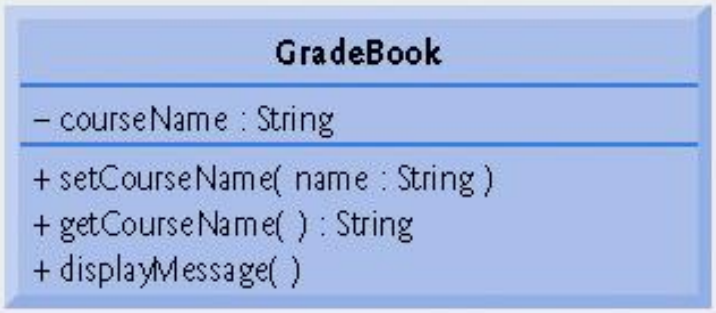


Fig. 3.9 | UML class diagram indicating that class `GradeBook` has a `courseName` attribute of UML type `String` and three operations—`setCourseName` (with a `name` parameter of UML type `String`), `getCourseName` (returns UML type `String`) and `displayMessage`.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Primitive Types vs. Reference Types

- **Types in Java**
  - **Primitive**
    - `boolean`, `byte`, `char`, `short`, `int`, `long`, `float`, `double`
  - **Reference (sometimes called nonprimitive types)**
    - **Objects**
    - **Default value of `null`**
    - **Used to invoke an object's methods**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 3.4

A variable's declared type (e.g., `int`, `double` or `GradeBook`) indicates whether the variable is of a primitive or a reference type. If a variable's type is not one of the eight primitive types, then it is a reference type. For example, `Account account1` indicates that `account1` is a reference to an `Account` object).



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.7 Initializing Objects with Constructors

- **Constructors**
  - **Initialize an object of a class**
  - **Java requires a constructor for every class**
  - **Java will provide a default no-argument constructor if none is provided**
  - **Called when keyword `new` is followed by the class name and parentheses**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

(1 of 2)

```
1  // Fig. 3.10: GradeBook.java
2  // GradeBook class with a constructor to initialize the course name.
3
4  public class GradeBook
5  {
6     private String courseName; // course name for this GradeBook
7
8     // constructor initializes courseName with String supplied as argument
9     public GradeBook( String name )
10    {
11       courseName = name; // initializes courseNam
12    } // end constructor
13
14    // method to set the course name
15    public void setCourseName( String name )
16    {
17       courseName = name; // store the course name
18    } // end method setCourseName
19
20    // method to retrieve the course name
21    public String getCourseName()
22    {
23       return courseName;
24    } // end method getCourseName
```

Constructor to initialize `courseName` variable



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBook.java

(2 of 2)

```
25
26    // display a welcome message to the GradeBook user
27    public void displayMessage()
28    {
29       // this statement calls getCourseName to get the
30       // name of the course this GradeBook represents
31       System.out.printf( "Welcome to the grade book for\n%s!\n",
32          getCourseName() );
33    } // end method displayMessage
34
35 } // end class GradeBook
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

GradeBookTest.java

```
1  // Fig. 3.11: GradeBookTest.java
2  // GradeBook constructor used to specify the course name at the
3  // time each GradeBook object is created.
4
5  public class GradeBookTest
6  {
7     // main method begins program execution
8     public static void main( String args[] )
9     {
10       // create GradeBook object
11       GradeBook gradeBook1 = new GradeBook(
12          "CS101 Introduction to Java Programming" );
13       GradeBook gradeBook2 = new GradeBook(
14          "CS102 Data Structures in Java" );
15
16       // display initial value of courseName for each GradeBook
17       System.out.printf( "gradeBook1 course name is: %s\n",
18          gradeBook1.getCourseName() );
19       System.out.printf( "gradeBook2 course name is: %s\n",
20          gradeBook2.getCourseName() );
21    } // end main
22
23 } // end class GradeBookTest
```

Call constructor to create first grade book object

Create second grade book object

```
gradeBook1 course name is: CS101 Introduction to Java Programming
gradeBook2 course name is: CS102 Data Structures in Java
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Error-Prevention Tip 3.1

Unless default initialization of your class's instance variables is acceptable, provide a constructor to ensure that your class's instance variables are properly initialized with meaningful values when each new object of your class is created.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Adding the Constructor to Class `GradeBookTest`'s UML Class Diagram

- **UML class diagram**
  - **Constructors go in third compartment**
  - **Place "<<constructor>>" before constructor name**
  - **By convention, place constructors first in their compartment**



© 1992-2007 Pearson Education, Inc. All rights reserved.

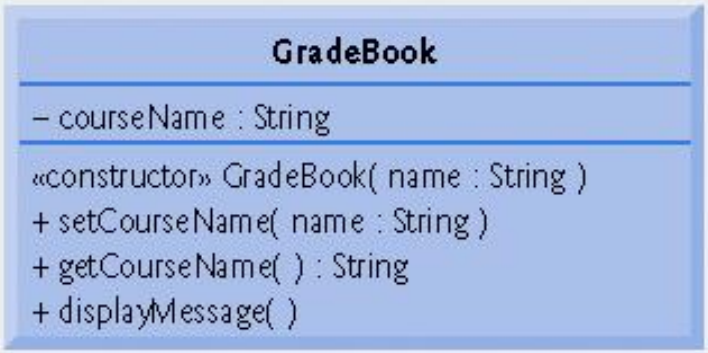


Fig. 3.12 | UML class diagram indicating that class `GradeBook` has a constructor that has a `name` parameter of UML type `String`.



© 1992-2007 Pearson Education, Inc. All rights reserved.

3.8 Floating-Point Numbers and Type `double`

- **Floating-point numbers**
  - `float`
  - `double`
    - **Stores numbers with greater magnitude and precision than `float`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Floating-Point Number Precision and Memory Requirements

- **float**
  - **Single-precision floating-point numbers**
  - **Seven significant digits**
- **double**
  - **Double-precision floating-point numbers**
  - **Fifteen significant digits**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 3.4

Using floating-point numbers in a manner that assumes they are represented precisely can lead to logic errors.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Account.java

```
1  // Fig. 3.13: Account.java
2  // Account class with a constructor to
3  // initialize instance variable balance.
4
5  public class Account
6  {
7     private double balance; // instance variable that stores the balance
8
9     // constructor
10    public Account( double initialBalance )
11    {
12       // validate that initialBalance is greater than 0.0;
13       // if it is not, balance is initialized to the default value 0.0
14       if ( initialBalance > 0.0 )
15          balance = initialBalance;
16    } // end Account constructor
17
18    // credit (add) an amount to the account
19    public void credit( double amount )
20    {
21       balance = balance + amount; // add amount to balance
22    } // end method credit
23
24    // return the account balance
25    public double getBalance()
26    {
27       return balance; // gives the value of balance to the calling method
28    } // end method getBalance
29
30 } // end class Account
```

double variable balance



© 1992-2007 Pearson Education, Inc. All rights reserved.

`AccountTest` Class to use Class `Account`

- **Format specifier `%f`**
  - **Used to output floating-point numbers**
  - **Place a decimal and a number between the percent sign and the `f` to specify a precision**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

AccountTest.java

(1 of 3)

```
1  // Fig. 3.14: AccountTest.java
2  // Create and manipulate an Account object.
3  import java.util.Scanner;
4
5  public class AccountTest
6  {
7     // main method begins execution of Java application
8     public static void main( String args[] )
9     {
10       Account account1 = new Account( 50.00 ); // create Account object
11       Account account2 = new Account( -7.53 ); // create Account object
12
13       // display initial balance of each object
14       System.out.printf( "account1 balance: $%.2f\n",
15          account1.getBalance() );
16       System.out.printf( "account2 balance: $%.2f\n\n",
17          account2.getBalance() );
18
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

AccountTest.java

```
19      // create Scanner to obtain input from command window
20      Scanner input = new Scanner( System.in );
21      double depositAmount; // deposit amount read from user
22
23      System.out.print( "Enter deposit amount for account1: " ); // prompt
24      depositAmount = input.nextDouble(); // obtain user input
25      System.out.printf( "\nadding %.2f to accoun
26         depositAmount );
27      account1.credit( depositAmount ); // add to account1 balance
28
29      // display balances
30      System.out.printf( "account1 balance: $%.2f\n",
31         account1.getBalance() );
32      System.out.printf( "account2 balance: $%.2f\n\n",
33         account2.getBalance() );
34
35      System.out.print( "Enter deposit amount for account2: " ); // prompt
36      depositAmount = input.nextDouble(); // obtain user input
37      System.out.printf( "\nadding %.2f to account2 balance\n\n",
38         depositAmount );
39      account2.credit( depositAmount ); // add to account2 balance
40
```

3)

Input a `double` value

Input a `double` value



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
41       // display balances
42       System.out.printf( "account1 balance: $%.2f\n",
43          account1.getBalance() );
44       System.out.printf( "account2 balance: $%.2f\n",
45          account2.getBalance() );
46    } // end main
47
48 } // end class AccountTest
```

AccountTest.java

Output a `double` value

```
account1 balance: $50.00
account2 balance: $0.00

Enter deposit amount for account1: 25.53

adding 25.53 to account1 balance

account1 balance: $75.53
account2 balance: $0.00

Enter deposit amount for account2: 123.45

adding 123.45 to account2 balance

account1 balance: $75.53
account2 balance: $123.45
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

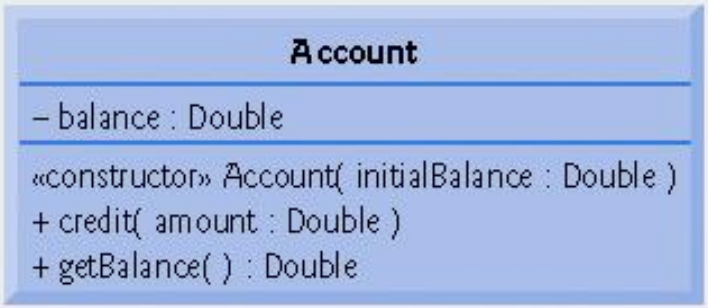


Fig. 3.15 | UML class diagram indicating that class `Account` has a `private balance` attribute of UML type `Double`, a constructor (with a parameter of UML type `Double`) and two `public` operations—`credit` (with an `amount` parameter of UML type `Double`) and `getBalance` (returns UML type `Double`).



© 1992-2007 Pearson Education, Inc. All rights reserved.

Location	Title—Exercise(s)
Section 3.9	Using Dialog Boxes—Basic input and output with dialog boxes
Section 4.14	Creating Simple Drawings—Displaying and drawing lines on the screen
Section 5.10	Drawing Rectangles and Ovals—Using shapes to represent data
Section 6.13	Colors and Filled Shapes—Drawing a bull's-eye and random graphics
Section 7.13	Drawing Arcs—Drawing spirals with arcs
Section 8.18	Using Objects with Graphics—Storing shapes as objects
Section 9.8	Displaying Text and Images Using Labels—Providing status information
Section 10.8	Drawing with Polymorphism—Identifying the similarities between shapes
Exercise 11.18	Expanding the Interface—Using GUI components and event handling
Exercise 12.12	Adding Java 2D—Using the Java 2D API to enhance drawings

Fig. 3.16 | Summary of the GUI and Graphics Case Study in each chapter.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Displaying Text in a Dialog Box

- **Windows and dialog boxes**
  - **Many Java applications use these to display output**
  - **`JOptionPane` provides prepackaged dialog boxes called message dialogs**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Dialog1.java

```
1  // Fig. 3.17: Dialog1.java
2  // Printing multiple lines in dialog box.
3  import javax.swing.JOptionPane; // import class JOptionPane
4
5  public class Dialog1
6  {
7     public static void main( String args[] )
8     {
9        // display a dialog with the message
10       JOptionPane.showMessageDialog( null, "Welcome\nto\nJava" );
11    } // end main
12 } // end class Dialog1
```

Import class `JOptionPane`

Show a message dialog with text



© 1992-2007 Pearson Education, Inc. All rights reserved.

Displaying Text in a Dialog Box

- **Package `javax.swing`**
  - **Contains classes to help create graphical user interfaces (GUIs)**
  - **Contains class `JOptionPane`**
    - **Declares `static` method `showMessageDialog` for displaying a message dialog**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Entering Text in a Dialog Box

- **Input dialog**
  - **Allows user to input information**
  - **Created using method `showInputDialog` from class `JOptionPane`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

NameDialog.java

```
1  // Fig. 3.18: NameDialog.java
2  // Basic input with a dialog box.
3  import javax.swing.JOptionPane;
4
5  public class NameDialog
6  {
7     public static void main( String args[] )
8     {
9        // prompt user to enter name
10       String name =
11          JOptionPane.showInputDialog( "What is your name?" );
12
13       // create the message
14       String message =
15          String.format( "Welcome, %s, to Java Programming!", name );
16
17       // display the message to welcome the user by name
18       JOptionPane.showMessageDialog( null, message );
19    } // end main
20 } // end class NameDialog
```

Show input dialog

Format a `String` to output to user







© 1992-2007 Pearson Education, Inc. All rights reserved.

3.10 (Optional) Software Engineering Case Study: Identifying the Classes in a Requirements Document

- **Begin designing the ATM system**
  - **Analyze the nouns and noun phrases**
  - **Introduce UML class diagrams**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Identifying the Classes in a System

- **Key nouns and noun phrases in requirements document**
  - **Some are attributes of other classes**
  - **Some do not correspond to parts of the system**
  - **Some are classes**
    - **To be represented by UML class diagrams**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Nouns and noun phrases in the requirements document		
bank	money / funds	account number
ATM	screen	PIN
user	keypad	bank database
customer	cash dispenser	balance inquiry
transaction	$20 bill / cash	withdrawal
account	deposit slot	deposit
balance	deposit envelope	

Fig. 3.19 | Nouns and noun phrases in the requirements document.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Modeling Classes

- **UML class diagrams**
  - **Top compartment contains name of the class**
  - **Middle compartment contains class's attributes or instance variables**
  - **Bottom compartment contains class's operations or methods**



© 1992-2007 Pearson Education, Inc. All rights reserved.



Fig. 3.20 | Representing a class in the UML using a class diagram.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Modeling Classes

- **UML class diagrams**
  - **Allows suppression of class attributes and operations**
    - **Called an elided diagram**
  - **Solid line that connects two classes represents an association**
    - **numbers near end of each line are multiplicity values**



© 1992-2007 Pearson Education, Inc. All rights reserved.

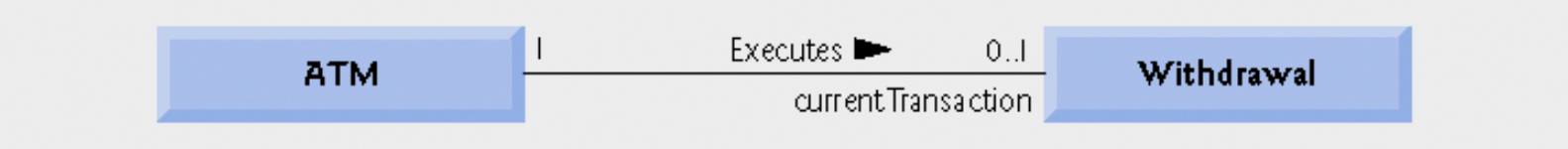


Fig. 3.21 | Class diagram showing an association among classes.

© 1992-2007 Pearson Education, Inc. All rights reserved.

Symbol	Meaning
0	None
1	One
m	An integer value
0..1	Zero or one
m, *n*	*m* or *n*
m..*n*	At least *m*, but not more than *n*
*	Any non-negative integer (zero or more)
0..*	Zero or more (identical to *)
1..*	One or more

Fig. 3.22 | Multiplicity types.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Modeling Classes

- **UML class diagrams**
  - **Solid diamonds attached to association lines indicate a composition relationship**
  - **Hollow diamonds indicate aggregation – a weaker form of composition**



© 1992-2007 Pearson Education, Inc. All rights reserved.

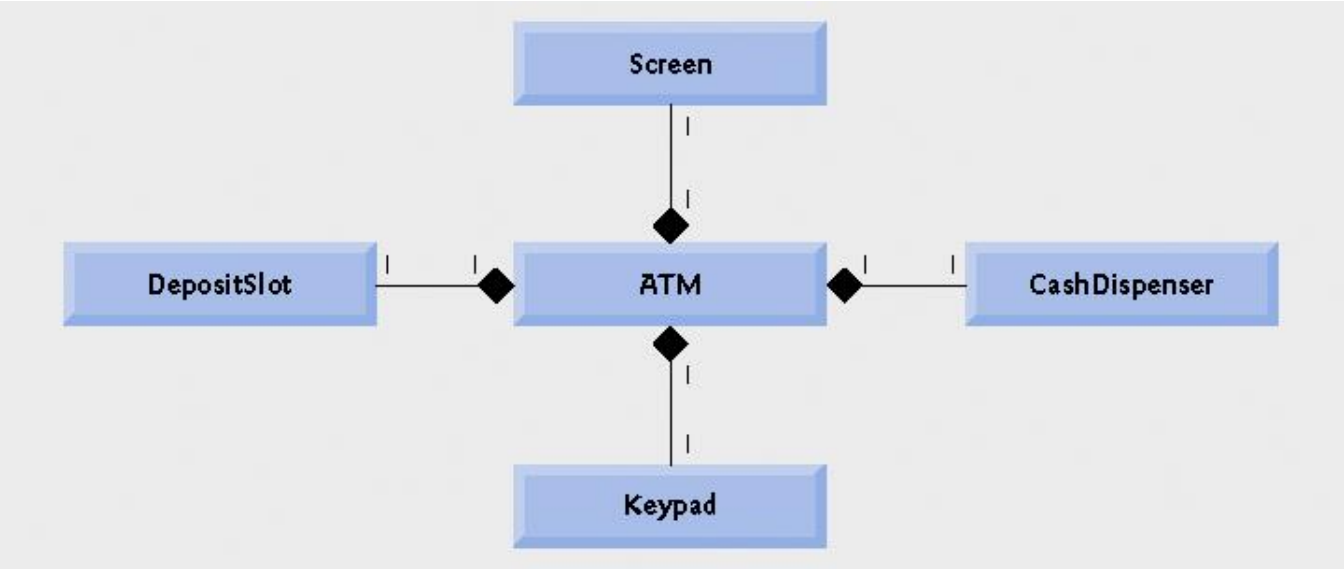


Fig. 3.23 | Class diagram showing composition relationships.



© 1992-2007 Pearson Education, Inc. All rights reserved.

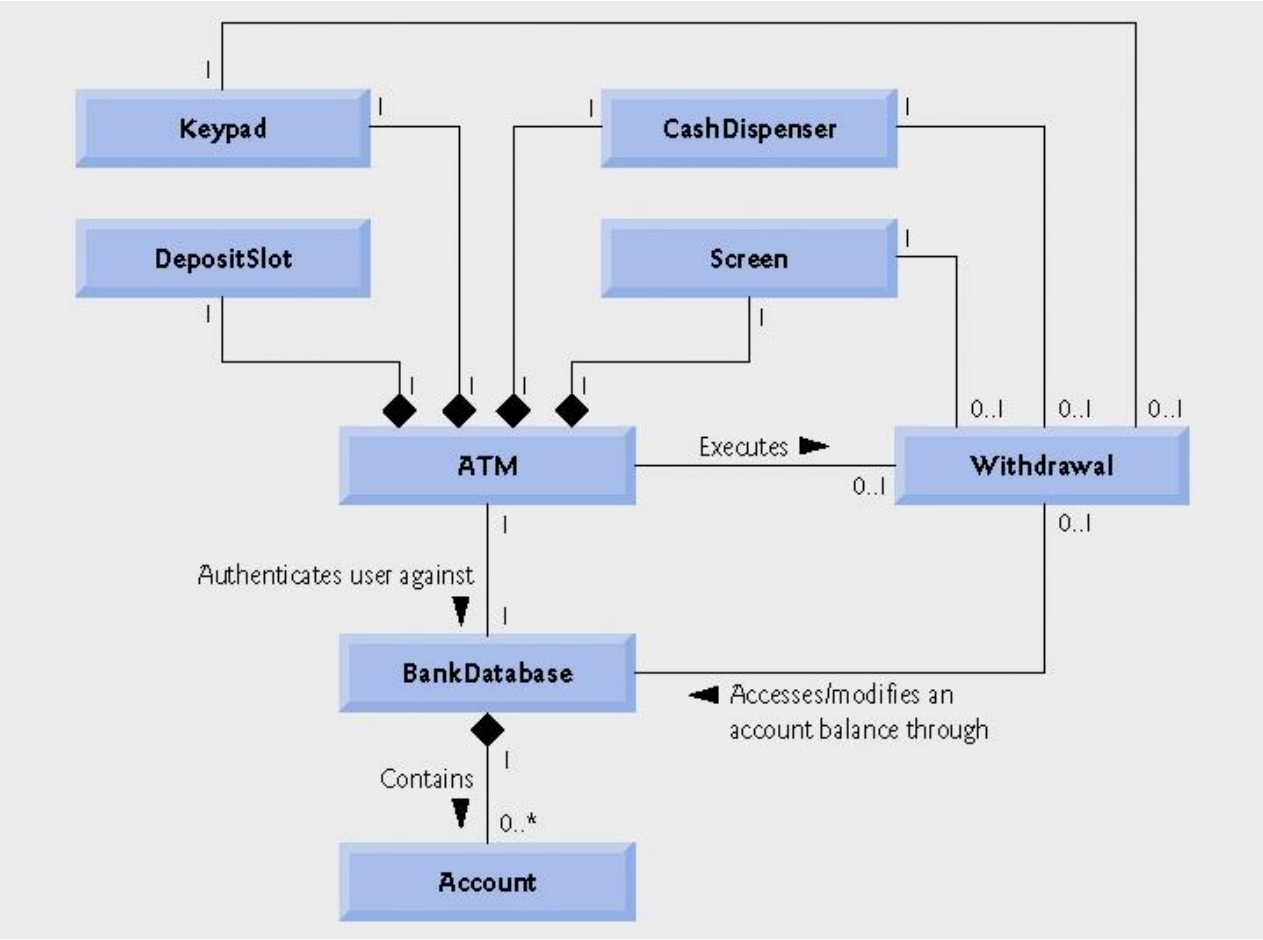


Fig. 3.24 | Class diagram for the ATM system model.

© 1992-2007 Pearson Education, Inc. All rights reserved.

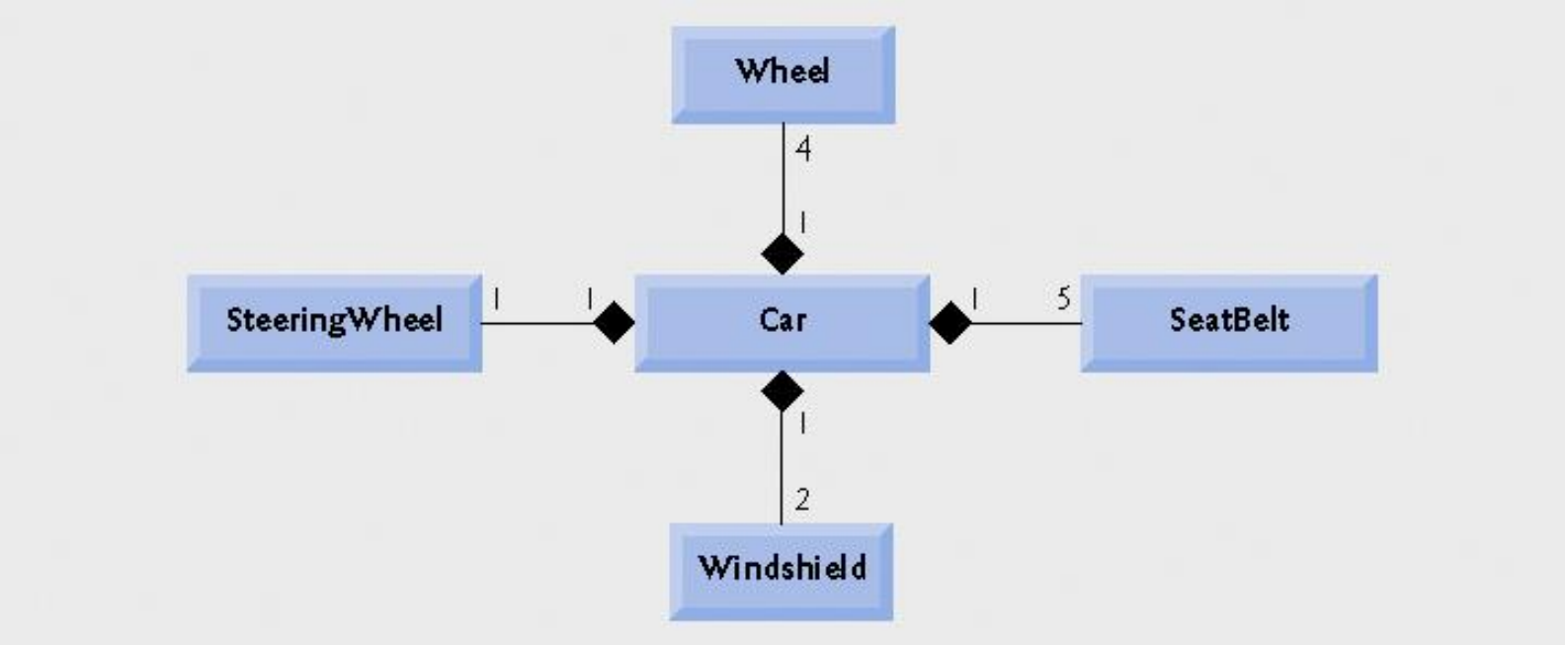


Fig. 3.25 | Class diagram showing composition relationships of a class `Car`.



© 1992-2007 Pearson Education, Inc. All rights reserved.

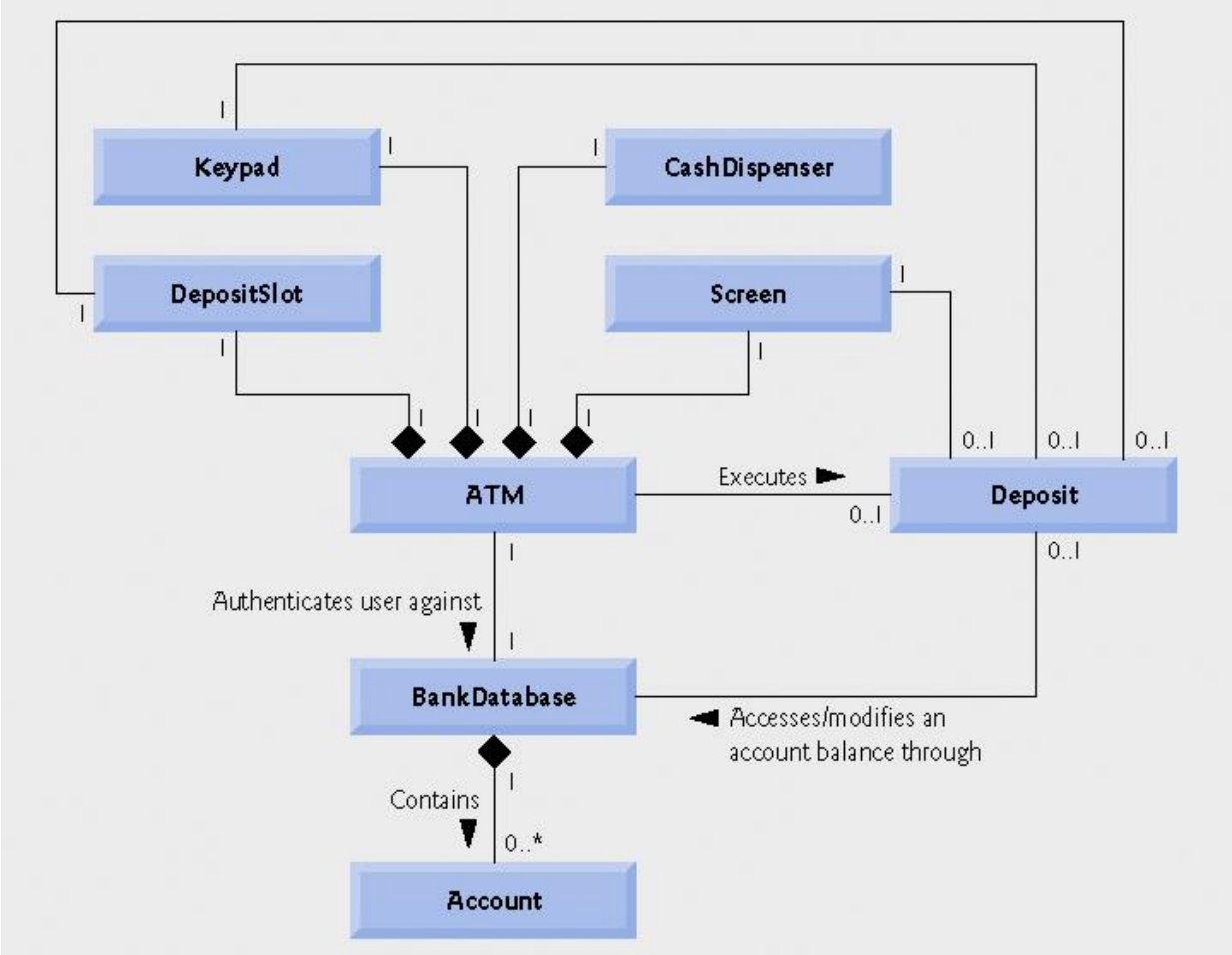


Fig. 3.26 | Class diagram for the ATM system model including class `Deposit`.

© 1992-2007 Pearson Education, Inc. All rights reserved.

8

Classes and Objects: A Deeper Look -

modified by Eileen Kraemer



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.2 Time Class Case Study

- **`public` services (or `public` interface)**
  - **`public` methods available for a client to use**
- **If a class does not define a constructor the compiler will provide a default constructor**
- **Instance variables**
  - **Can be initialized when they are declared or in a constructor**
  - **Should maintain consistent (valid) values**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Time1.java

(1 of 2)

```
1  // Fig. 8.1: Time1.java
2  // Time1 class declaration maintains the time in 24-hour format.
3
4  public class Time1
5  {
6     private int hour;   // 0 - 23
7     private int minute; // 0 - 59
8     private int second; // 0 - 59
9
10    // set a new time value using universal time; ensure that
11    // the data remains consistent by setting invalid values to zero
12    public void setTime( int h, int m, int s )
13
14       hour = ( ( h >= 0 && h < 24 ) ? h : 0 );   // validate hour
15       minute = ( ( m >= 0 && m < 60 ) ? m : 0 ); // validate minute
16       second = ( ( s >= 0 && s < 60 ) ? s : 0 ); // validate second
17    } // end method setTime
18
```

private instance variables

Declare **public** method **setTime**

Validate parameter values before setting instance variables



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Time1.java

(2 of 2)

```
19    // convert to String in universal-time format (HH:MM:SS)
20    public String toUniversalString()
21    {
22       return String.format( "%02d:%02d:%02d", hour, minute, second );
23    } // end method toUniversalString
24
25    // convert to String in standard-time format (H:MM:SS AM or PM)
26    public String toString()
27    {
28       return String.format( "%d:%02d:%02d %s",
29          ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 ),
30          minute, second, ( hour < 12 ? "AM" : "PM" ) );
31    } // end method toString
32 } // end class Time1
```

format strings



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.2 Time Class Case Study (Cont.)

- `String` method `format`
  - Similar to `printf` except it returns a formatted string instead of displaying it in a command window
- `new` implicitly invokes `Time1`'s default constructor since `Time1` does not declare any constructors



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Time1Test.java

(1 of 2)

```
1  // Fig. 8.2: Time1Test.java
2  // Time1 object used in an application.
3
4  public class Time1Test
5  {
6     public static void main( String args[] )
7     {
8        // create and initialize a Time1 object
9        Time1 time = new Time1(); // invokes Time1 constructor
10
11       // output string representations of the time
12       System.out.print( "The initial universal time is: " );
13       System.out.println( time.toUniversalString() );
14       System.out.print( "The initial standard time is: " );
15       System.out.println( time.toString() );
16       System.out.println(); // output a blank line
17
```

Create a **Time1** object

Call **toUniversalString** method

Call **toString** method



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Time1Test.java

```
18       // change time and output updated time
19       time.setTime( 13, 27, 6 );
20       System.out.print( "Universal time after setTime is: " );
21       System.out.println( time.toUniversalString() );
22       System.out.print( "Standard time after setTime is: " );
23       System.out.println( time.toString() );
24       System.out.println(); // output a blank line
25
26       // set time with invalid values; output updated time
27       time.setTime( 99, 99, 99 );
28       System.out.println( "After attempting invalid settings:" );
29       System.out.print( "Universal time: " );
30       System.out.println( time.toUniversalString() );
31       System.out.print( "Standard time: " );
32       System.out.println( time.toString() );
33    } // end main
34 } // end class Time1Test
```

Call **setTime** method

Call **setTime** method with invalid values

```
The initial universal time is: 00:00:00
The initial standard time is: 12:00:00 AM

Universal time after setTime is: 13:27:06
Standard time after setTime is: 1:27:06 PM

After attempting invalid settings:
Universal time: 00:00:00
Standard time: 12:00:00 AM
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.3 Controlling Access to Members

- **A class's public interface**
  - **`public` methods a view of the services the class provides to the class's clients**
- **A class's implementation details**
  - **`private` variables and `private` methods are not accessible to the class's clients**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.1

An attempt by a method that is not a member of a class to access a `private` member of that class is a compilation error.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

MemberAccessTest

.java

```
1  // Fig. 8.3: MemberAccessTest.java
2  // Private members of class Time1 are not accessible.
3  public class MemberAccessTest
4  {
5     public static void main( String args[] )
6     {
7        Time1 time = new Time1(); // create and initialize Time1 object
8
9        time.hour = 7;     // error: hour has private access in Time1
10       time.minute = 15; // error: minute has private access in Time1
11       time.second = 30; // error: second has private access in Time1
12    } // end main
13 } // end class MemberAccessTest
```

Attempting to access **`private`** instance variables

```
MemberAccessTest.java:9: hour has private access in Time1
      time.hour = 7;    // error: hour has private access in Time1
           ^
MemberAccessTest.java:10: minute has private access in Time1
      time.minute = 15; // error: minute has private access in Time1
           ^
MemberAccessTest.java:11: second has private access in Time1
      time.second = 30; // error: second has private access in Time1
           ^
3 errors
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.4 Referring to the Current Object's Members with the `this` Reference

- **The `this` reference**
  - **Any object can access a reference to itself with keyword `this`**
  - **Non-`static` methods implicitly use `this` when referring to the object's instance variables and other methods**
  - **Can be used to access instance variables when they are shadowed by local variables or method parameters**
- **A `.java` file can contain more than one class**
  - **But only one class in each `.java` file can be `public`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

ThisTest.java

(1 of 2)

```
1  // Fig. 8.4: ThisTest.java
2  // this used implicitly and explicitly to refer to members of an object.
3
4  public class ThisTest
5  {
6     public static void main( String args[] )
7     {
8        SimpleTime time = new SimpleTime( 15, 30, 19 );
9        System.out.println( time.buildString() );
10    } // end main
11 } // end class ThisTest
12
13 // class SimpleTime demonstrates the "this" reference
14 class SimpleTime
15 {
16    private int hour;   // 0-23
17    private int minute; // 0-59
18    private int second; // 0-59
19
20    // if the constructor uses parameter names identical to
21    // instance variable names the "this" reference is
22    // required to distinguish between names
23    public SimpleTime( int hour, int minute, int second )
24    {
25       this.hour = hour;       // set "this" object's hour
26       this.minute = minute;   // set "this" object's minute
27       this.second = second;   // set "this" object's second
28    } // end SimpleTime constructor
29
```

Create new **SimpleTime** object

Declare instance variables

Method parameters shadow instance variables

Using `this` to access the object's instance variables



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
30    // use explicit and implicit "this" to call toUniversalString
31    public String buildString()
32    {
33       return String.format( "%24s: %s\n%24s: %s",
34          "this.toUniversalString()", this.toUniversalString(),
35          "toUniversalString()", toUniversalString() );
36    } // end method buildString
37
38    // convert to String in universal-time format (HH:MM:SS)
39    public String toUniversalString()
40    {
41       // "this" is not required here to access instance variables,
42       // because method does not have local variables with same
43       // names as instance variables
44       return String.format( "%02d:%02d:%02d",
45          this.hour, this.minute, this.second );
46    } // end method toUniversalString
47 } // end class SimpleTime
```

```
this.toUniversalString(): 15:30:19
     toUniversalString(): 15:30:19
```

ThisTest.java

Using **this** explicitly and implicitly to call **toUniversalString**

(2 of 2)

Use of **this** not necessary here



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.2

It is often a logic error when a method contains a parameter or local variable that has the same name as a field of the class. In this case, use reference `this` if you wish to access the field of the class—otherwise, the method parameter or local variable will be referenced.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Error-Prevention Tip 8.1

Avoid method parameter names or local variable names that conflict with field names. This helps prevent subtle, hard-to-locate bugs.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Performance Tip 8.1

Java conserves storage by maintaining only one copy of each method per class—this method is invoked by every object of the class. Each object, on the other hand, has its own copy of the class's instance variables (i.e., non-`static` fields). Each method of the class implicitly uses `this` to determine the specific object of the class to manipulate.



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.5 Time Class Case Study: Overloaded Constructors

- **Overloaded constructors**
  - **Provide multiple constructor definitions with different signatures**
- **No-argument constructor**
  - **A constructor invoked without arguments**
- **The `this` reference can be used to invoke another constructor**
  - **Allowed only as the first statement in a constructor's body**



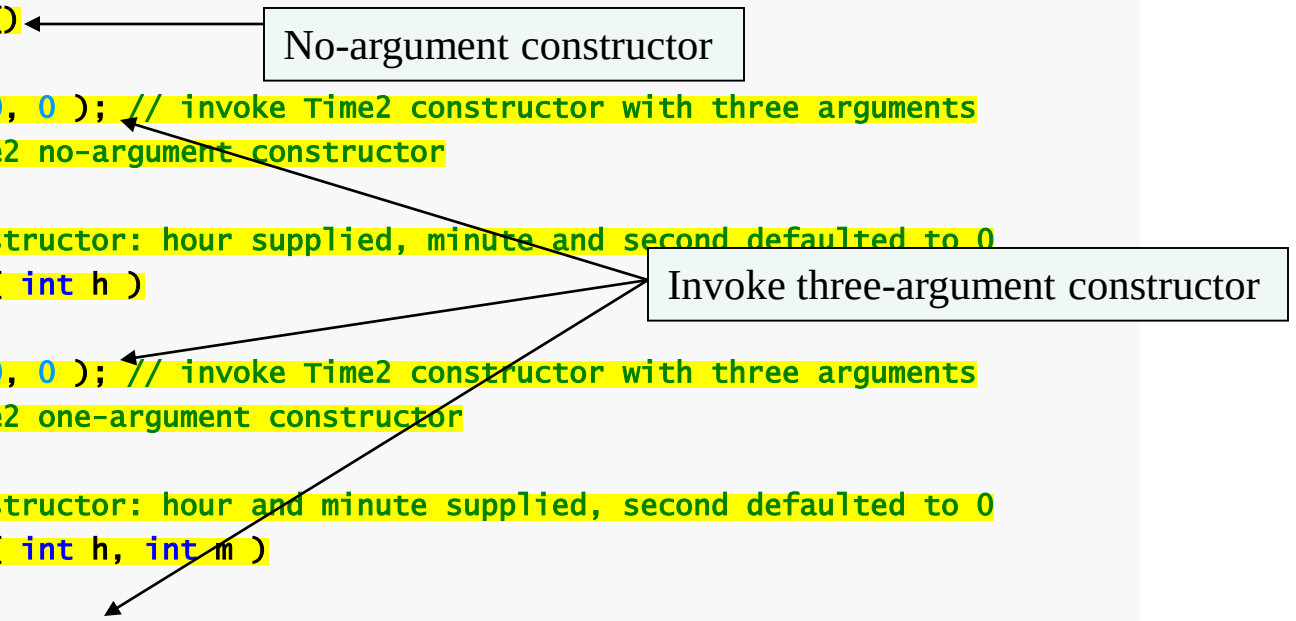
© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Time2.java

(1 of 4)

```
1  // Fig. 8.5: Time2.java
2  // Time2 class declaration with overloaded constructors.
3
4  public class Time2
5  {
6     private int hour;   // 0 - 23
7     private int minute; // 0 - 59
8     private int second; // 0 - 59
9
10    // Time2 no-argument constructor: initializes each instance variable
11    // to zero; ensures that Time2 objects start in a consistent state
12    public Time2()
13    {
14       this( 0, 0, 0 ); // invoke Time2 constructor with three arguments
15    } // end Time2 no-argument constructor
16
17    // Time2 constructor: hour supplied, minute and second defaulted to 0
18    public Time2( int h )
19    {
20       this( h, 0, 0 ); // invoke Time2 constructor with three arguments
21    } // end Time2 one-argument constructor
22
23    // Time2 constructor: hour and minute supplied, second defaulted to 0
24    public Time2( int h, int m )
25    {
26       this( h, m, 0 ); // invoke Time2 constructor with three arguments
27    } // end Time2 two-argument constructor
28
```





© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
29    // Time2 constructor: hour, minute and second supplied
30    public Time2( int h, int m, int s )
31    {
32       setTime( h, m, s ); // invoke setTime to validate time
33    } // end Time2 three-argument constructor
34
35    // Time2 constructor: another Time2 object supplied
36    public Time2( Time2 time )
37    {
38       // invoke Time2 three-argument c
39       this( time.getHour(), time.getMinute(), time.getSecond() );
40    } // end Time2 constructor with a Time2 object argument
41
42    // Set Methods
43    // set a new time value using universal time; ensure that
44    // the data remains consistent by setting invalid values to zero
45    public void setTime( int h, int m, int s )
46    {
47       setHour( h );   // set the hour
48       setMinute( m ); // set the minute
49       setSecond( s ); // set the second
50    } // end method setTime
51
```

Call **setTime** method

Time2.java

Constructor takes a reference to another **Time2** object as a parameter

(2 of 4)

Could have directly accessed instance variables of object **time** here



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Time2.java

(3 of 4)

```
52    // validate and set hour
53    public void setHour( int h )
54    {
55       hour = ( ( h >= 0 && h < 24 ) ? h : 0 );
56    } // end method setHour
57
58    // validate and set minute
59    public void setMinute( int m )
60    {
61       minute = ( ( m >= 0 && m < 60 ) ? m : 0 );
62    } // end method setMinute
63
64    // validate and set second
65    public void setSecond( int s )
66    {
67       second = ( ( s >= 0 && s < 60 ) ? s : 0 );
68    } // end method setSecond
69
70    // Get Methods
71    // get hour value
72    public int getHour()
73    {
74       return hour;
75    } // end method getHour
76
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Time2.java

(4 of 4)

```
77    // get minute value
78    public int getMinute()
79    {
80       return minute;
81    } // end method getMinute
82
83    // get second value
84    public int getSecond()
85    {
86       return second;
87    } // end method getSecond
88
89    // convert to String in universal-time format (HH:MM:SS)
90    public String toUniversalString()
91    {
92       return String.format(
93          "%02d:%02d:%02d", getHour(), getMinute(), getSecond() );
94    } // end method toUniversalString
95
96    // convert to String in standard-time format (H:MM:SS AM or PM)
97    public String toString()
98    {
99       return String.format( "%d:%02d:%02d %s",
100         ( (getHour() == 0 || getHour() == 12) ? 12 : getHour() % 12 ),
101         getMinute(), getSecond(), ( getHour() < 12 ? "AM" : "PM" ) );
102   } // end method toString
103} // end class Time2
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.3

It is a syntax error when `this` is used in a constructor's body to call another constructor of the same class if that call is not the first statement in the constructor. It is also a syntax error when a method attempts to invoke a constructor directly via `this`.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.4

A constructor can call methods of the class. Be aware that the instance variables might not yet be in a consistent state, because the constructor is in the process of initializing the object. Using instance variables before they have been initialized properly is a logic error.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 8.4

When one object of a class has a reference to another object of the same class, the first object can access all the second object's data and methods (including those that are `private`).



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.5 Time Class Case Study: Overloaded Constructors (Cont.)

- **Using *set* methods**
  - **Having constructors use *set* methods to modify instance variables instead of modifying them directly simplifies implementation changing**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Call overloaded constructors

Time2Test.java

(1 of 3)

```
1  // Fig. 8.6: Time2Test.java
2  // Overloaded constructors used to initialize Time2 objects.
3
4  public class Time2Test
5  {
6     public static void main( String args[] )
7     {
8        Time2 t1 = new Time2();           // 00:00:00
9        Time2 t2 = new Time2( 2 );        // 02:00:00
10       Time2 t3 = new Time2( 21, 34 );   // 21:34:00
11       Time2 t4 = new Time2( 12, 25, 42 ); // 12:25:42
12       Time2 t5 = new Time2( 27, 74, 99 ); // 00:00:00
13       Time2 t6 = new Time2( t4 );       // 12:25:42
14
15       System.out.println( "Constructed with:" );
16       System.out.println( "t1: all arguments defaulted" );
17       System.out.printf( "   %s\n", t1.toUniversalString() );
18       System.out.printf( "   %s\n", t1.toString() );
19
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
20      System.out.println(
21         "t2: hour specified; minute and second defaulted" );
22      System.out.printf( "   %s\n", t2.toUniversalString() );
23      System.out.printf( "   %s\n", t2.toString() );
24
25      System.out.println(
26         "t3: hour and minute specified; second defaulted" );
27      System.out.printf( "   %s\n", t3.toUniversalString() );
28      System.out.printf( "   %s\n", t3.toString() );
29
30      System.out.println( "t4: hour, minute and second specified" );
31      System.out.printf( "   %s\n", t4.toUniversalString() );
32      System.out.printf( "   %s\n", t4.toString() );
33
34      System.out.println( "t5: all invalid values specified" );
35      System.out.printf( "   %s\n", t5.toUniversalString() );
36      System.out.printf( "   %s\n", t5.toString() );
37
```

Outline

Time2Test.java

(2 of 3)



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
38       System.out.println( "t6: Time2 object t4 specified" );
39       System.out.printf( "   %s\n", t6.toUniversalString() );
40       System.out.printf( "   %s\n", t6.toString() );
41    } // end main
42 } // end class Time2Test
```

```
t1: all arguments defaulted
   00:00:00
   12:00:00 AM
t2: hour specified; minute and second defaulted
   02:00:00
   2:00:00 AM
t3: hour and minute specified; second defaulted
   21:34:00
   9:34:00 PM
t4: hour, minute and second specified
   12:25:42
   12:25:42 PM
t5: all invalid values specified
   00:00:00
   12:00:00 AM
t6: Time2 object t4 specified
   12:25:42
   12:25:42 PM
```

Outline

Time2Test.java

(3 of 3)



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.6 Default and No-Argument Constructors

- **Every class must have at least one constructor**
  - **If no constructors are declared, the compiler will create a default constructor**
    - **Takes no arguments and initializes instance variables to their initial values specified in their declaration or to their default values**
      - **Default values are zero for primitive numeric types, `false` for `boolean` values and `null` for references**
  - **If constructors are declared, the default initialization for objects of the class will be performed by a no-argument constructor (if one is declared)**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.5

If a class has constructors, but none of the `public` constructors are no-argument constructors, and a program attempts to call a no-argument constructor to initialize an object of the class, a compilation error occurs. A constructor can be called with no arguments only if the class does not have any constructors (in which case the default constructor is called) or if the class has a `public` no-argument constructor.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 8.6

Java allows other methods of the class besides its constructors to have the same name as the class and to specify return types. Such methods are not constructors and will not be called when an object of the class is instantiated. Java determines which methods are constructors by locating the methods that have the same name as the class and do not specify a return type.



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.9 Enumerations

- **`enum` types**
  - **Declared with an `enum` declaration**
    - **A comma-separated list of `enum` constants**
    - **Declares an `enum` class with the following restrictions:**
      - **`enum` types are implicitly `final`**
      - **`enum` constants are implicitly `static`**
      - **Attempting to create an object of an `enum` type with `new` is a compilation error**
  - **`enum` constants can be used anywhere constants can**
  - **`enum` constructor**
    - **Like class constructors, can specify parameters and be overloaded**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Book.java

(1 of 2)

```
1  // Fig. 8.10: Book.java
2  // Declaring an enum type with constructor and explicit instance fields
3  // and accessors for these field
4
5  public enum Book
6  {
7     // declare constants of enum type
8     JHTP6( "Java How to Program 6e", "2005" ),
9     CHTP4( "C How to Program 4e", "2004" ),
10    IW3HTP3( "Internet & World Wide Web How to Program 3e", "2004" ),
11    CPPHTP4( "C++ How to Program 4e", "2003" ),
12    VBHTP2( "Visual Basic .NET How to Program 2e", "2002" ),
13    CSHARPHTP( "C# How to Program", "2002" );
14
15    // instance fields
16    private final String title; // book title
17    private final String copyrightYear; // copyright year
18
19    // enum constructor
20    Book( String bookTitle, String year )
21    {
22       title = bookTitle;
23       copyrightYear = year;
24    } // end enum Book constructor
25
```

Declare six **enum** constants

Arguments to pass to the **enum** constructor

Declare instance variables

Declare **enum** constructor **Book**



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
26    // accessor for field title
27    public String getTitle()
28    {
29       return title;
30    } // end method getTitle
31
32    // accessor for field copyrightYear
33    public String getCopyrightYear()
34    {
35       return copyrightYear;
36    } // end method getCopyrightYear
37 } // end enum Book
```

Outline

Book.java

(2 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.9 Enumerations (Cont.)

- `static` method `values`
  - Generated by the compiler for every `enum`
  - Returns an array of the `enum`'s constants in the order in which they were declared
- `static` method `range` of class `EnumSet`
  - Takes two parameters, the first and last `enum` constants in the desired range
  - Returns an `EnumSet` containing the constants in that range, inclusive
  - An enhanced `for` statement can iterate over an `EnumSet` as it can over an array



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

EnumTest.java

```
1  // Fig. 8.11: EnumTest.java
2  // Testing enum type Book.
3  import java.util.EnumSet;
4
5  public class EnumTest
6  {
7     public static void main( String args[] )
8     {
9        System.out.println( "All books:\n" );
10
11       // print all books in enum Book
12       for ( Book book : Book.values() )
13          System.out.printf( "%-10s%-45s%s\n", book,
14             book.getTitle(), book.getCopyrightYear() );
15
16       System.out.println( "\nDisplay a range of enum constants:\n" );
17
18       // print first four books
19       for ( Book book : EnumSet.range( Book.JHTP6, Book.CPPHTP4 ) )
20          System.out.printf( "%-10s%-45s%s\n", book,
21             book.getTitle(), book.getCopyrightYear() );
22    } // end main
23 } // end class EnumTest
```

Enhanced **for** loop iterates for each **enum** constant in the array returned by method **value**

Enhanced **for** loop iterates for each **enum** constant in the **EnumSet** returned by method **range**



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
All books:

JHTP6     Java How to Program 6e                          2005
CHTP4     C How to Program 4e                             2004
IW3HTP3   Internet & World Wide Web How to Program 3e     2004
CPPHTP4   C++ How to Program 4e                           2003
VBHTP2    Visual Basic .NET How to Program 2e             2002
CSHARPHTP C# How to Program                               2002

Display a range of enum constants:

JHTP6     Java How to Program 6e                          2005
CHTP4     C How to Program 4e                             2004
IW3HTP3   Internet & World Wide Web How to Program 3e     2004
CPPHTP4   C++ How to Program 4e                           2003
```

Outline

EnumTest.java

(2 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.6

In an `enum` declaration, it is a syntax error to declare `enum` constants after the `enum` type's constructors, fields and methods in the `enum` declaration.



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.10 Garbage Collection and Method `finalize`

- **Garbage collection**
  - **JVM marks an object for garbage collection when there are no more references to that object**
  - **JVM's garbage collector will retrieve those objects memory so it can be used for other objects**
- **`finalize` method**
  - **All classes in Java have the `finalize` method**
    - **Inherited from the `Object` class**
  - **`finalize` is called by the garbage collector when it performs termination housekeeping**
  - **`finalize` takes no parameters and has return type `void`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 8.10

A class that uses system resources, such as files on disk, should provide a method to eventually release the resources. Many Java API classes provide `close` or `dispose` methods for this purpose. For example, class `Scanner` (`java.sun.com/javase/6/docs/api/java/util/Scanner.html`) has a `close` method.



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.11 `static` Class Members

- **`static` fields**
  - **Also known as class variables**
  - **Represents class-wide information**
  - **Used when:**
    - **all objects of the class should share the same copy of this instance variable or**
    - **this instance variable should be accessible even when no objects of the class exist**
  - **Can be accessed with the class name or an object name and a dot (.)**
  - **Must be initialized in their declarations, or else the compiler will initialize it with a default value (`0` for `int`s)**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 8.11

Use a `static` variable when all objects of a class must use the same copy of the variable.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 8.12

Static class variables and methods exist, and can be used, even if no objects of that class have been instantiated.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Employee.java

(1 of 2)

```
1  // Fig. 8.12: Employee.java
2  // Static variable used to maintain a count of the number of
3  // Employee objects in memory.
4
5  public class Employee
6  {
7     private String firstName;
8     private String lastName;
9     private static int count = 0; // number of objects in memory
10
11    // initialize employee, add 1 to static count and
12    // output String indicating that constructor was called
13    public Employee( String first, String last )
14    {
15       firstName = first;
16       lastName = last;
17
18       count++;  // increment static count of employees
19       System.out.printf( "Employee constructor: %s %s; count = %d\n",
20          firstName, lastName, count );
21    } // end Employee constructor
22
```

Declare a **`static`** field

Increment **`static`** field



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Employee.java

(2 of 2)

```
23    // subtract 1 from static count when garbage
24    // collector calls finalize to clean up object;
25    // confirm that finalize was called
26    protected void finalize()
27    {
28       count--; // decrement static count of employees
29       System.out.printf( "Employee finalizer: %s %s; count = %d\n",
30          firstName, lastName, count );
31    } // end method finalize
32
33    // get first name
34    public String getFirstName()
35    {
36       return firstName;
37    } // end method getFirstName
38
39    // get last name
40    public String getLastName()
41    {
42       return lastName;
43    } // end method getLastName
44
45    // static method to get static count value
46    public static int getCount()
47    {
48       return count;
49    } // end method getCount
50 } // end class Employee
```

Declare method **`finalize`**

Declare **`static`** method **`getCount`** to get **`static`** field **`count`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

EmployeeTest.java

(1 of 3)

```
1  // Fig. 8.13: EmployeeTest.java
2  // Static member demonstration.
3
4  public class EmployeeTest
5  {
6     public static void main( String args[] )
7     {
8        // show that count is 0 before creating Employees
9        System.out.printf( "Employees before instantiation: %d\n",
10          Employee.getCount() );
11
12       // create two Employees; count should be 2
13       Employee e1 = new Employee( "Susan", "Baker" );
14       Employee e2 = new Employee( "Bob", "Blue" );
15
```

Call **`static`** method **`getCount`** using class name **`Employee`**

Create new **`Employee`** objects



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

EmployeeTest.java

```
16      // show that count is 2 after creating two Employees
17      System.out.println( "\nEmployees after instantiation: " );
18      System.out.printf( "via e1.getCount(): %d\n", e1.getCount() );
19      System.out.printf( "via e2.getCount(): %d\n", e2.getCount() );
20      System.out.printf( "via Employee.getCount(): %d\n",
21         Employee.getCount() );
22
23      // get names of Employees
24      System.out.printf( "\nEmployee 1: %s %s\nEmployee 2: %s %s\n\n",
25         e1.getFirstName(), e1.getLastName(),
26         e2.getFirstName(), e2.getLastName() );
27
28      // in this example, there is only one reference to each Employee,
29      // so the following two statements cause the JVM to mark each
30      // Employee object for garbage collection
31      e1 = null;
32      e2 = null;
33
34      System.gc(); // ask for garbage collection to occur now
35
```

Call **static** method **getCount** using class name

Call **static** method **getCount** using variable name

Remove references to objects, JVM will mark them for garbage collection

Call **static** method **gc** of class **System** to indicate that garbage collection should be attempted

(2 of 3)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

EmployeeTest.java

(3 of 3)

```
36       // show Employee count after calling garbage collector; count
37       // displayed may be 0, 1 or 2 based on whether garbage collector
38       // executes immediately and number of Employee objects collected
39       System.out.printf( "\nEmployees after System.gc(): %d\n",
40          Employee.getCount() );
41    } // end main
42 } // end class EmployeeTest
```

Call **static** method **getCount**

```
Employees before instantiation: 0
Employee constructor: Susan Baker; count = 1
Employee constructor: Bob Blue; count = 2

Employees after instantiation:
via e1.getCount(): 2
via e2.getCount(): 2
via Employee.getCount(): 2

Employee 1: Susan Baker
Employee 2: Bob Blue

Employee finalizer: Bob Blue; count = 1
Employee finalizer: Susan Baker; count = 0

Employees after System.gc(): 0
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Good Programming Practice 8.1

Invoke every `static` method by using the class name and a dot (.) to emphasize that the method being called is a `static` method.



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.11 `static` Class Members (Cont.)

- **`String` objects are immutable**
  - **`String` concatenation operations actually result in the creation of a new `String` object**
- **`static` method `gc` of class `System`**
  - **Indicates that the garbage collector should make a best-effort attempt to reclaim objects eligible for garbage collection**
  - **It is possible that no objects or only a subset of eligible objects will be collected**
- **`static` methods cannot access non-`static` class members**
  - **Also cannot use the `this` reference**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.7

A compilation error occurs if a `static` method calls an instance (non-`static`) method in the same class by using only the method name. Similarly, a compilation error occurs if a `static` method attempts to access an instance variable in the same class by using only the variable name.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.8

Referring to `this` in a `static` method is a syntax error.



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.12 `static` Import

- **`static` import declarations**
  - **Enables programmers to refer to imported `static` members as if they were declared in the class that uses them**
  - **Single `static` import**
    - `import static packageName.ClassName.staticMemberName;`
  - **`static` import on demand**
    - `import static packageName.ClassName.*;`
    - **Imports all `static` members of the specified class**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

StaticImportTest.java

```
1  // Fig. 8.14: StaticImportTest.java
2  // Using static import to import static methods of class Math.
3  import static java.lang.Math.*;
4
5  public class StaticImportTest
6  {
7     public static void main( String args[] )
8     {
9        System.out.printf( "sqrt( 900.0 ) = %.1f\n", sqrt( 900.0 ) );
10       System.out.printf( "ceil( -9.8 ) = %.1f\n", ceil( -9.8 ) );
11       System.out.printf( "log( E ) = %.1f\n", log( E ) );
12       System.out.printf( "cos( 0.0 ) = %.1f\n", cos( 0.0 ) );
13    } // end main
14 } // end class StaticImportTest
```

static import on demand

Use **Math**'s **static** methods and instance variable without preceding them with **Math.**

```
sqrt( 900.0 ) = 30.0
ceil( -9.8 ) = -9.0
log( E ) = 1.0
cos( 0.0 ) = 1.0
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.9

A compilation error occurs if a program attempts to import `static` methods that have the same signature or `static` fields that have the same name from two or more classes.



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.13 `final` Instance Variables

- **Principle of least privilege**
  - **Code should have only the privilege and access it needs to accomplish its task, but no more**
- **`final` instance variables**
  - **Keyword `final`**
    - **Specifies that a variable is not modifiable (is a constant)**
  - **`final` instance variables can be initialized at their declaration**
    - **If they are not initialized in their declarations, they must be initialized in all constructors**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 8.13

Declaring an instance variable as `final` helps enforce the principle of least privilege. If an instance variable should not be modified, declare it to be `final` to prevent modification.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Increment.java

```
1  // Fig. 8.15: Increment.java
2  // final instance variable in a class.
3
4  public class Increment
5  {
6     private int total = 0; // total of all increments
7     private final int INCREMENT; // constant variable (uninitialized)
8
9     // constructor initializes final instance variable INCREMENT
10    public Increment( int incrementValue )
11    {
12       INCREMENT = incrementValue; // initialize constant variable (once)
13    } // end Increment constructor
14
15    // add INCREMENT to total
16    public void addIncrementToTotal()
17    {
18       total += INCREMENT;
19    } // end method addIncrementToTotal
20
21    // return String representation of an Increment object's data
22    public String toString()
23    {
24       return String.format( "total = %d", total );
25    } // end method toIncrementString
26 } // end class Increment
```

Declare **`final`** instance variable

Initialize **`final`** instance variable inside a constructor



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

IncrementTest.java

```
1  // Fig. 8.16: IncrementTest.java
2  // final variable initialized with a constructor argument.
3
4  public class IncrementTest
5  {
6     public static void main( String args[] )
7     {
8        Increment value = new Increment( 5 );
9
10       System.out.printf( "Before incrementing: %s\n\n", value );
11
12       for ( int i = 1; i <= 3; i++ )
13       {
14          value.addIncrementToTotal();
15          System.out.printf( "After increment %d: %s\n", i, value );
16       } // end for
17    } // end main
18 } // end class IncrementTest
```

Create an **Increment** object

Call method **addIncrementToTotal**

```
Before incrementing: total = 0

After increment 1: total = 5
After increment 2: total = 10
After increment 3: total = 15
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.10

Attempting to modify a `final` instance variable after it is initialized is a compilation error.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Error-Prevention Tip 8.2

Attempts to modify a `final` instance variable are caught at compilation time rather than causing execution-time errors. It is always preferable to get bugs out at compilation time, if possible, rather than allow them to slip through to execution time (where studies have found that the cost of repair is often many times more expensive).



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 8.14

A `final` field should also be declared `static` if it is initialized in its declaration. Once a `final` field is initialized in its declaration, its value can never change. Therefore, it is not necessary to have a separate copy of the field for every object of the class. Making the field `static` enables all objects of the class to share the `final` field.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.11

Not initializing a `final` instance variable in its declaration or in every constructor of the class yields a compilation error indicating that the variable might not have been initialized. The same error occurs if the class initializes the variable in some, but not all, of the class's constructors.



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
Increment.java:13: variable INCREMENT might not have been initialized
   } // end Increment constructor
   ^
1 error
```

Outline

Increment.java



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.16 Time Class Case Study: Creating Packages

- To declare a reusable class
  - Declare a `public` class
  - Add a `package` declaration to the source-code file
    - must be the first executable statement in the file
    - `package` name should consist of your Internet domain name in reverse order followed by other names for the package
      - example: `com.deitel.jhtp7.ch08`
      - `package` name is part of the fully qualified class name
        - Distinguishes between multiple classes with the same name belonging to different packages
        - Prevents name conflict (also called name collision)
      - Class name without `package` name is the simple name



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 8.18: Time1.java
2  // Time1 class declaration maintains the time in 24-hour format.
3  package com.deitel.jhtp7.ch08;
4
5  public class Time1
6  {
7     private int hour;   // 0 - 23
8     private int minute; // 0 - 59
9     private int second; // 0 - 59
10
11    // set a new time value using universal time; perform
12    // validity checks on the data; set invalid values to zero
13    public void setTime( int h, int m, int s )
14    {
15       hour = ( ( h >= 0 && h < 24 ) ? h : 0 );   // validate hour
16       minute = ( ( m >= 0 && m < 60 ) ? m : 0 ); // validate minute
17       second = ( ( s >= 0 && s < 60 ) ? s : 0 ); // validate second
18    } // end method setTime
19
```

`package` declaration

`Time1` is a **`public`** class so it can be used by importers of this package

Time1.java

(1 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
20    // convert to String in universal-time format (HH:MM:SS)
21    public String toUniversalString()
22    {
23       return String.format( "%02d:%02d:%02d", hour, minute, second );
24    } // end method toUniversalString
25
26    // convert to String in standard-time format (H:MM:SS AM or PM)
27    public String toString()
28    {
29       return String.format( "%d:%02d:%02d %s",
30          ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 ),
31          minute, second, ( hour < 12 ? "AM" : "PM" ) );
32    } // end method toString
33 } // end class Time1
```

Outline

Time1.java

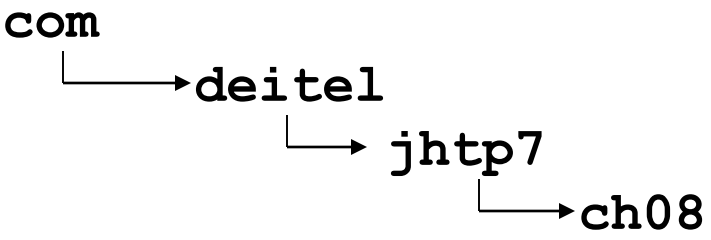
(2 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.16 Time Class Case Study: Creating Packages (Cont.)

- Compile the class so that it is placed in the appropriate package directory structure
  - Example: our package should be in the directory



  - `javac` command-line option `-d`
    - `javac` creates appropriate directories based on the class's `package` declaration
    - A period (`.`) after `-d` represents the current directory

© 1992-2007 Pearson Education, Inc. All rights reserved.

8.16 Time Class Case Study: Creating Packages (Cont.)

- Import the reusable class into a program
  - Single-type-import declaration
    - Imports a single class
    - Example: `import java.util.Random;`
  - Type-import-on-demand declaration
    - Imports all classes in a package
    - Example: `import java.util.*;`



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.12

Using the `import` declaration `import java.*;` causes a compilation error. You must specify the exact name of the package from which you want to import classes.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 8.19: Time1PackageTest.java
2  // Time1 object used in an application.
3  import com.deitel.jhtp6.ch08.Time1; // import class Time1
4
5  public class Time1PackageTest
6  {
7     public static void main( String args[] )
8     {
9        // create and initialize a Time1 object
10       Time1 time = new Time1(); // calls Time1 constructor
11
12       // output string representations of the time
13       System.out.print( "The initial universal time is: " );
14       System.out.println( time.toUniversalString() );
15       System.out.print( "The initial standard time is: " );
16       System.out.println( time.toString() );
17       System.out.println(); // output a blank line
18
```

Single-type **`import`** declaration

Refer to the **`Time1`** class by its simple name

`Time1PackageTest`

`.java`

(1 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Time1PackageTest.java

(2 of 2)

```
19       // change time and output updated time
20       time.setTime( 13, 27, 6 );
21       System.out.print( "Universal time after setTime is: " );
22       System.out.println( time.toUniversalString() );
23       System.out.print( "Standard time after setTime is: " );
24       System.out.println( time.toString() );
25       System.out.println(); // output a blank line
26
27       // set time with invalid values; output updated time
28       time.setTime( 99, 99, 99 );
29       System.out.println( "After attempting invalid settings:" );
30       System.out.print( "Universal time: " );
31       System.out.println( time.toUniversalString() );
32       System.out.print( "Standard time: " );
33       System.out.println( time.toString() );
34    } // end main
35 } // end class Time1PackageTest
```

```
The initial universal time is: 00:00:00
The initial standard time is: 12:00:00 AM

Universal time after setTime is: 13:27:06
Standard time after setTime is: 1:27:06 PM

After attempting invalid settings:
Universal time: 00:00:00
Standard time: 12:00:00 AM
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.16 Time Class Case Study: Creating Packages (Cont.)

- **Class loader**
  - **Locates classes that the compiler needs**
    - **First searches standard Java classes bundled with the JDK**
    - **Then searches for optional packages**
      - **These are enabled by Java's extension mechanism**
    - **Finally searches the classpath**
      - **List of directories or archive files separated by directory separators**
        - **These files normally end with `.jar` or `.zip`**
        - **Standard classes are in the archive file `rt.jar`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.16 Time Class Case Study: Creating Packages (Cont.)

- To use a classpath other than the current directory
  - `-classpath` option for the `javac` compiler
  - Set the `CLASSPATH` environment variable
- The JVM must locate classes just as the compiler does
  - The `java` command can use other classpathes by using the same techniques that the `javac` command uses



© 1992-2007 Pearson Education, Inc. All rights reserved.

Common Programming Error 8.13

Specifying an explicit classpath eliminates the current directory from the classpath. This prevents classes in the current directory (including packages in the current directory) from loading properly. If classes must be loaded from the current directory, include a dot (.) in the classpath to specify the current directory.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Software Engineering Observation 8.16

In general, it is a better practice to use the `-classpath` option of the compiler, rather than the `CLASSPATH` environment variable, to specify the classpath for a program. This enables each application to have its own classpath.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Error-Prevention Tip 8.3

Specifying the classpath with the CLASSPATH environment variable can cause subtle and difficult-to-locate errors in programs that use different versions of the same package.



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.17 Package Access

- **Package access**
  - **Methods and variables declared without any access modifier are given package access**
  - **This has no effect if the program consists of one class**
  - **This does have an effect if the program contains multiple classes from the same package**
    - **Package-access members can be directly accessed through the appropriate references to objects in other classes belonging to the same package**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

PackageDataTest.java

(1 of 2)

```
1  // Fig. 8.20: PackageDataTest.java
2  // Package-access members of a class are accessible by other classes
3  // in the same package.
4
5  public class PackageDataTest
6  {
7     public static void main( String args[] )
8     {
9        PackageData packageData = new PackageData();
10
11       // output String representation of packageData
12       System.out.printf( "After instantiation:\n%s\n", packageData );
13
14       // change package access data in packageData object
15       packageData.number = 77;
16       packageData.string = "Goodbye";
17
18       // output String representation of packageData
19       System.out.printf( "\nAfter changing values:\n%s\n", packageData );
20    } // end main
21 } // end class PackageDataTest
22
```

Can directly access package-access members



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

PackageDataTest.java

(2 of 2)

```
23 // class with package access instance variables
24 class PackageData
25 {
26    int number; // package-access instance variable
27    String string; // package-access instance variable
28
29    // constructor
30    public PackageData()
31    {
32       number = 0;
33       string = "Hello";
34    } // end PackageData constructor
35
36    // return PackageData object String representation
37    public String toString()
38    {
39       return String.format( "number: %d; string: %s", number, string );
40    } // end method toString
41 } // end class PackageData
```

Package-access instance variables

```
After instantiation:
number: 0; string: Hello

After changing values:
number: 77; string: Goodbye
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

8.18 (Optional) GUI and Graphics Case Study: Using Objects with Graphics

- **To create a consistent drawing that remains the same each time it is drawn**
  - **Store information about the displayed shapes so that they can be reproduced exactly the same way each time `paintComponent` is called**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Myline.java

```
1  // Fig. 8.21: MyLine.java
2  // Declaration of class MyLine.
3  import java.awt.Color;
4  import java.awt.Graphics;
5
6  public class MyLine
7  {
8     private int x1; // x coordinate of first endpoint
9     private int y1; // y coordinate of first endpoint
10    private int x2; // x coordinate of second endpoint
11    private int y2; // y coordinate of second endpoint
12    private Color myColor; // color of this shape
13
14    // constructor with input values
15    public MyLine( int x1, int y1, int x2, int y2, Color color )
16    {
17       this.x1 = x1; // set x coordinate of first endpoint
18       this.y1 = y1; // set y coordinate of first endpoint
19       this.x2 = x2; // set x coordinate of second endpoint
20       this.y2 = y2; // set y coordinate of second endpoint
21       myColor = color; // set the color
22    } // end MyLine constructor
23
24    // Draw the line in the specified color
25    public void draw( Graphics g )
26    {
27       g.setColor( myColor );
28       g.drawLine( x1, y1, x2, y2 );
29    } // end method draw
30 } // end class MyLine
```

Instance variables to store coordinates and color for a line

Initialize instance variables

Draw a line in the proper color at the proper coordinates



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

DrawPanel.java

(1 of 2)

```
1  // Fig. 8.22: DrawPanel.java
2  // Program that uses class MyLine
3  // to draw random lines.
4  import java.awt.Color;
5  import java.awt.Graphics;
6  import java.util.Random;
7  import javax.swing.JPanel;
8
9  public class DrawPanel extends JPanel
10 {
11    private Random randomNumbers = new Random();
12    private MyLine lines[]; // array of lines
13
14    // constructor, creates a panel with random shapes
15    public DrawPanel()
16    {
17       setBackground( Color.WHITE );
18
19       lines = new MyLine[ 5 + randomNumbers.nextInt( 5 ) ];
20
```

Declare a **MyLine** array

Create the **MyLine** array



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

DrawPanel.java

(2 of 2)

```
21       // create lines
22       for ( int count = 0; count < lines.length; count++ )
23       {
24          // generate random coordinates
25          int x1 = randomNumbers.nextInt( 300 );
26          int y1 = randomNumbers.nextInt( 300 );
27          int x2 = randomNumbers.nextInt( 300 );
28          int y2 = randomNumbers.nextInt( 300 );
29
30          // generate a random color
31          Color color = new Color( randomNumbers.nextInt( 256 ),
32             randomNumbers.nextInt( 256 ), randomNumbers.nextInt( 256 ) );
33
34          // add the line to the list of lines to be displayed
35          lines[ count ] = new MyLine( x1, y1, x2, y2, color );
36       } // end for
37    } // end DrawPanel constructor
38
39    // for each shape array, draw the individual shapes
40    public void paintComponent( Graphics g )
41    {
42       super.paintComponent( g );
43
44       // draw the lines
45       for ( MyLine line : lines )
46          line.draw( g );
47    } // end method paintComponent
48 } // end class DrawPanel
```

Generate coordinates for this line

Generate a color for this line

Create the new **MyLine** object with the generated attributes

Draw each **MyLine**

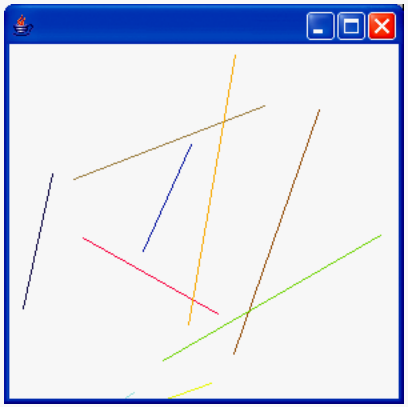


© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

TestDraw.java

```
1  // Fig. 8.23: TestDraw.java
2  // Test application to display a DrawPanel.
3  import javax.swing.JFrame;
4
5  public class TestDraw
6  {
7     public static void main( String args[] )
8     {
9        DrawPanel panel = new DrawPanel();
10       JFrame application = new JFrame();
11
12       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
13       application.add( panel );
14       application.setSize( 300, 300 );
15       application.setVisible( true );
16    } // end main
17 } // end class TestDraw
```





© 1992-2007 Pearson Education, Inc. All rights reserved.

9

Object-Oriented Programming: Inheritance



© 1992-2007 Pearson Education, Inc. All rights reserved.

Say not you know another entirely, till you have divided an inheritance with him.

— Johann Kasper Lavater

This method is to define as the number of a class the class of all classes similar to the given class.

— Bertrand Russell

Good as it is to inherit a library, it is better to collect one.

— Augustine Birrell

Save base authority from others' books.

— William Shakespeare



© 1992-2007 Pearson Education Inc. All rights reserved.

OBJECTIVES

In this chapter you will learn:

- How inheritance promotes software reusability.
- The notions of superclasses and subclasses.
- To use keyword `extends` to create a class that inherits attributes and behaviors from another class.
- To use access modifier `protected` to give subclass methods access to superclass members.
- To access superclass members with `super`.
- How constructors are used in inheritance hierarchies.
- The methods of class `Object`, the direct or indirect superclass of all classes in Java.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

9.1 Introduction

9.2 Superclasses and Subclasses

9.3 `protected` Members

9.4 Relationship between Superclasses and Subclasses

- 9.4.1 Creating and Using a `CommissionEmployee` Class
- 9.4.2 Creating a `BasePlusCommissionEmployee` Class without Using Inheritance
- 9.4.3 Creating a `CommissionEmployee`–`BasePlusCommissionEmployee` Inheritance Hierarchy
- 9.4.4 `CommissionEmployee`–`BasePlusCommissionEmployee` Inheritance Hierarchy Using `protected` Instance Variables
- 9.4.5 `CommissionEmployee`–`BasePlusCommissionEmployee` Inheritance Hierarchy Using `private` Instance Variables



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

9.5 Constructors in Subclasses

9.6 Software Engineering with Inheritance

9.7 `Object` Class

9.8 (Optional) GUI and Graphics Case Study: Displaying Text and Images Using Labels

9.9 Wrap-Up



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.1 Introduction

- **Inheritance**
  - **Software reusability**
  - **Create new class from existing class**
    - **Absorb existing class's data and behaviors**
    - **Enhance with new capabilities**
  - **Subclass extends superclass**
    - **Subclass**
      - **More specialized group of objects**
      - **Behaviors inherited from superclass**
        - **Can customize**
      - **Additional behaviors**



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.1 Introduction (Cont.)

- **Class hierarchy**
  - **Direct superclass**
    - **Inherited explicitly (one level up hierarchy)**
  - **Indirect superclass**
    - **Inherited two or more levels up hierarchy**
  - **Single inheritance**
    - **Inherits from one superclass**
  - **Multiple inheritance**
    - **Inherits from multiple superclasses**
      - **Java does not support multiple inheritance**



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.2 Superclasses and subclasses

- **Superclasses and subclasses**
  - **Object of one class "is an" object of another class**
    - **Example: Rectangle is quadrilateral.**
      - **Class `Rectangle` inherits from class `Quadrilateral`**
      - **`Quadrilateral`: superclass**
      - **`Rectangle`: subclass**
  - **Superclass typically represents larger set of objects than subclasses**
    - **Example:**
      - **superclass: `Vehicle`**
        - **Cars, trucks, boats, bicycles, …**
      - **subclass: `Car`**
        - **Smaller, more-specific subset of vehicles**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Superclass	Subclasses
`Student`	`GraduateStudent`, `UndergraduateStudent`
`Shape`	`Circle`, `Triangle`, `Rectangle`
`Loan`	`CarLoan`, `HomeImprovementLoan`, `MortgageLoan`
`Employee`	`Faculty`, `Staff`
`BankAccount`	`CheckingAccount`, `SavingsAccount`

Fig. 9.1 | Inheritance examples.



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.2 Superclasses and subclasses (Cont.)

- **Inheritance hierarchy**
  - **Inheritance relationships: tree-like hierarchy structure**
  - **Each class becomes**
    - **superclass**
      - **Supply members to other classes**

    OR
    - **subclass**
      - **Inherit members from other classes**



© 1992-2007 Pearson Education, Inc. All rights reserved.

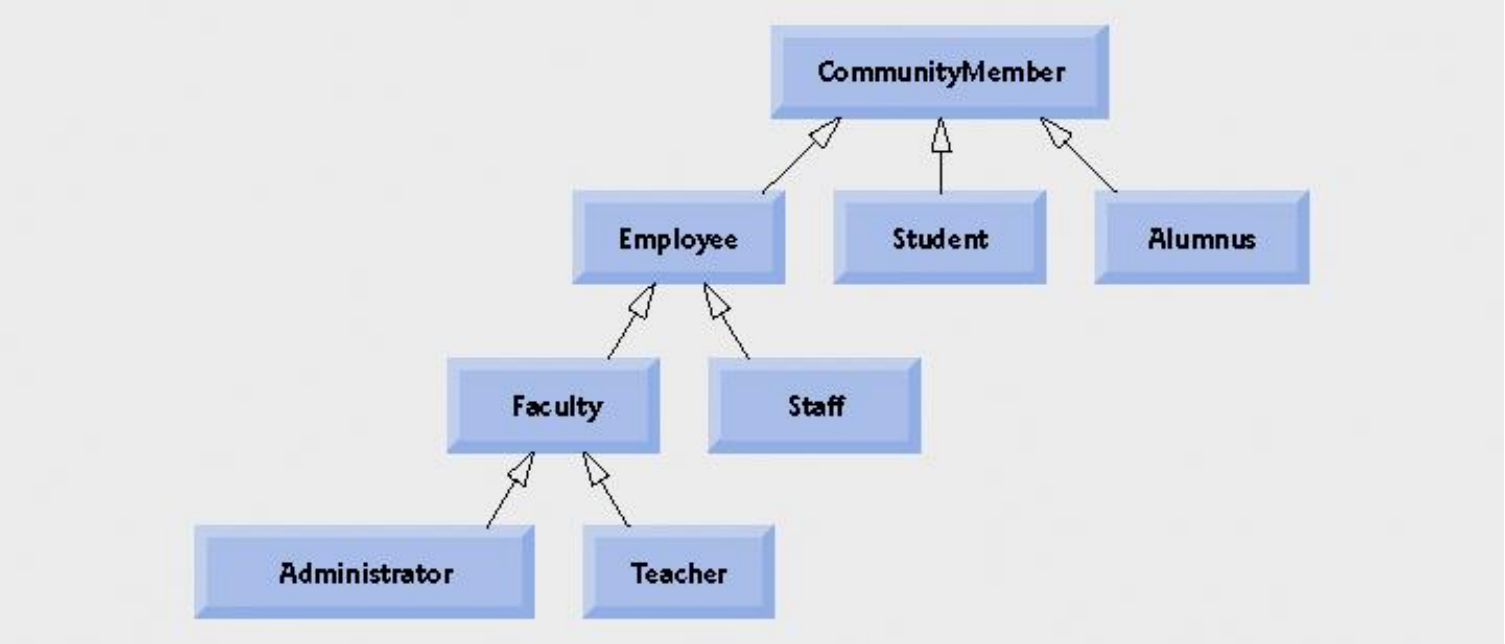


Fig. 9.2 | Inheritance hierarchy for university `CommunityMembers`

© 1992-2007 Pearson Education, Inc. All rights reserved.

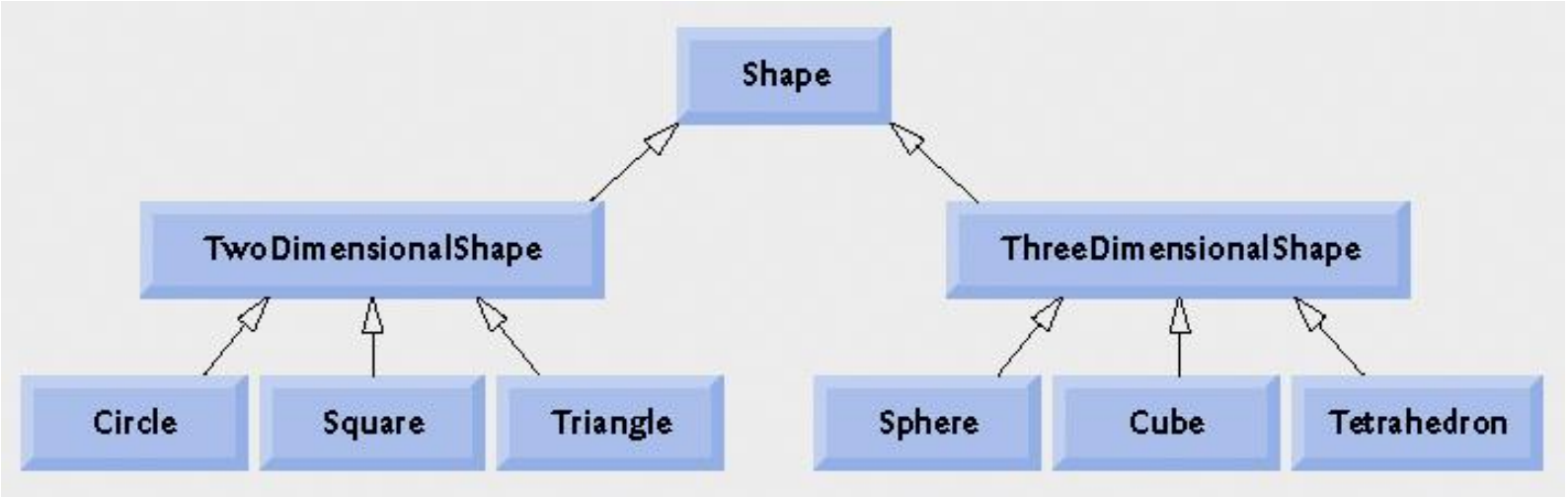


Fig. 9.3 | Inheritance hierarchy for Shapes.

© 1992-2007 Pearson Education, Inc. All rights reserved.

9.3 protected Members

- protected access
  - Intermediate level of protection between public and private
  - protected members accessible by
    - superclass members
    - subclass members
    - Class members in the same package
  - Subclass access to superclass member
    - Keyword super and a dot (.)



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.4 Relationship between Superclasses and Subclasses

- **Superclass and subclass relationship**
  - **Example: `CommissionEmployee/BasePlusCommissionEmployee` inheritance hierarchy**
    - `CommissionEmployee`
      - **First name, last name, SSN, commission rate, gross sale amount**
    - `BasePlusCommissionEmployee`
      - **First name, last name, SSN, commission rate, gross sale amount**
      - **Base salary**



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.4.1 Creating and Using a `CommissionEmployee` Class

- Class `CommissionEmployee`
  - Extends class `Object`
    - Keyword `extends`
    - Every class in Java extends an existing class
      - Except `Object`
    - Every class inherits `Object`'s methods
    - New class implicitly extends Object
      - If it does not extend another class



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 9.4: CommissionEmployee.java
2  // CommissionEmployee class represents a commiss
3
4  public class CommissionEmployee extends Obje
5  {
6     private String firstName;
7     private String lastName;
8     private String socialSecurityNumber;
9     private double grossSales; // gross weekly s
10    private double commissionRate; // commissio
11
12    // five-argument constructor
13    public CommissionEmployee( String first, String last, String ssn,
14       double sales, double rate )
15    {
16       // implicit call to Object constructor occurs here
17       firstName = first;
18       lastName = last;
19       socialSecurityNumber = ssn;
20       setGrossSales( sales ); // validate and store gross sales
21       setCommissionRate( rate ); // validate and store commission rate
22    } // end five-argument CommissionEmployee constructor
23
24    // set first name
25    public void setFirstName( String first )
26    {
27       firstName = first;
28    } // end method setFirstName
```

nEmployee

Declare `private` instance variables

Class `CommissionEmployee` `extends` class `Object`

Implicit call to `Object` constructor

Initialize instance variables

Invoke methods `setGrossSales` and `setCommissionRate` to validate data



ghts reserved.

```
30    // return first name
31    public String getFirstName()
32    {
33       return firstName;
34    } // end method getFirstName
35
36    // set last name
37    public void setLastName( String last )
38    {
39       lastName = last;
40    } // end method setLastName
41
42    // return last name
43    public String getLastName()
44    {
45       return lastName;
46    } // end method getLastName
47
48    // set social security number
49    public void setSocialSecurityNumber( String ssn )
50    {
51       socialSecurityNumber = ssn; // should validate
52    } // end method setSocialSecurityNumber
53
54    // return social security number
55    public String getSocialSecurityNumber()
56    {
57       return socialSecurityNumber;
58    } // end method getSocialSecurityNumber
59
```

ionEmployee



rights reserved.

Outline

CommissionEmployee.java

(3 of 4)

Lines 85-88

```
60    // set gross sales amount
61    public void setGrossSales( double sales )
62    {
63       grossSales = ( sales < 0.0 ) ? 0.0 : sales;
64    } // end method setGrossSales
65
66    // return gross sales amount
67    public double getGrossSales()
68    {
69       return grossSales;
70    } // end method getGrossSales
71
72    // set commission rate
73    public void setCommissionRate( double rate )
74    {
75       commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
76    } // end method setCommissionRate
77
78    // return commission rate
79    public double getCommissionRate()
80    {
81       return commissionRate;
82    } // end method getCommissionRate
83
84    // calculate earnings
85    public double earnings()
86    {
87       return commissionRate * grossSales;
88    } // end method earnings
89
```

Calculate earnings



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Override method `toString` of class `Object`

```
90    // return String representation of Comm
91    public String toString()
92    {
93       return String.format( "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
94          "commission employee", firstName, lastName,
95          "social security number", socialSecurityNumber,
96          "gross sales", grossSales,
97          "commission rate", commissionRate );
98    } // end method toString
99 } // end class CommissionEmployee
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 9.5: CommissionEmployeeTest.java
2  // Testing class CommissionEmployee.
3
4  public class CommissionEmployeeTest
5  {
6     public static void main( String args[] )
7     {
8        // instantiate CommissionEmployee object
9        CommissionEmployee employee = new CommissionEmployee(
10          "Sue", "Jones", "222-22-2222", 10000, .06 );
11
12       // get commission employee data
13       System.out.println(
14          "Employee information obtained by get methods: \n" );
15       System.out.printf( "%s %s\n", "
16          employee.getFirstName() );
17       System.out.printf( "%s %s\n", "
18          employee.getLastName() );
19       System.out.printf( "%s %s\n", "Social security number is",
20          employee.getSocialSecurityNumber() );
21       System.out.printf( "%s %.2f\n", "Gross sales is"
22          employee.getGrossSales() );
23       System.out.printf( "%s %.2f\n", "Commiss
24          employee.getCommissionRate() );
25
26       employee.setGrossSales( 500 ); // set gross sales
27       employee.setCommissionRate( .1 ); // set commission rate
28
```

Instantiate `CommissionEmployee` object

Use `CommissionEmployee`'s *get* methods to retrieve the object's instance variable values

Use `CommissionEmployee`'s *set* methods to change the object's instance variable values

CommissionEmployeeTest.java

(1 of 2)

Lines 9-10

Lines 15-25

Lines 26-27



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
29       System.out.printf( "\n%s:\n\n%s\n",
30          "Updated employee information obtained by toString", employee );
31    } // end main
32 } // end class CommissionEmployeeTest
```

Implicitly call object's `toString` method

```
Employee information obtained by get methods:

First name is Sue
Last name is Jones
Social security number is 222-22-2222
Gross sales is 10000.00
Commission rate is 0.06

Updated employee information obtained by toString:

commission employee: Sue Jones
social security number: 222-22-2222
gross sales: 500.00
commission rate: 0.10
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.4.2 Creating a `BasePlusCommissionEmployee` Class without Using Inheritance

- Class `BasePlusCommissionEmployee`
  - Implicitly extends `Object`
  - Much of the code is similar to `CommissionEmployee`
    - `private` instance variables
    - `public` methods
    - constructor
  - Additions
    - `private` instance variable `baseSalary`
    - Methods `setBaseSalary` and `getBaseSalary`



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 9.6: BasePlusCommissionEmployee.java
2  // BasePlusCommissionEmployee class represents an employee that receives
3  // a base salary in addition to commission.
4
5  public class BasePlusCommissionEmployee
6  {
7     private String firstName;
8     private String lastName;
9     private String socialSecurityNumber;
10    private double grossSales; // gross weekly sales
11    private double commissionRate; // commission percentage
12    private double baseSalary; // base salary per week
13
14    // six-argument constructor
15    public BasePlusCommissionEmployee( String first, String last,
16       String ssn, double sales, double rate, double salary )
17    {
18       // implicit call to Object constructor c
19       firstName = first;
20       lastName = last;
21       socialSecurityNumber = ssn;
22       setGrossSales( sales ); // validate and store gross sales
23       setCommissionRate( rate ); // validate and store commission rate
24       setBaseSalary( salary ); // validate and store base salary
25    } // end six-argument BasePlusCommissionEmployee constructor
26
```

Add instance variable `baseSalary`

Use method `setBaseSalary` to validate data



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
27    // set first name
28    public void setFirstName( String first )
29    {
30       firstName = first;
31    } // end method setFirstName
32
33    // return first name
34    public String getFirstName()
35    {
36       return firstName;
37    } // end method getFirstName
38
39    // set last name
40    public void setLastName( String last )
41    {
42       lastName = last;
43    } // end method setLastName
44
45    // return last name
46    public String getLastName()
47    {
48       return lastName;
49    } // end method getLastName
50
51    // set social security number
52    public void setSocialSecurityNumber( String ssn )
53    {
54       socialSecurityNumber = ssn; // should validate
55    } // end method setSocialSecurityNumber
56
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
57    // return social security number
58    public String getSocialSecurityNumber()
59    {
60       return socialSecurityNumber;
61    } // end method getSocialSecurityNumber
62
63    // set gross sales amount
64    public void setGrossSales( double sales )
65    {
66       grossSales = ( sales < 0.0 ) ? 0.0 : sales;
67    } // end method setGrossSales
68
69    // return gross sales amount
70    public double getGrossSales()
71    {
72       return grossSales;
73    } // end method getGrossSales
74
75    // set commission rate
76    public void setCommissionRate( double rate )
77    {
78       commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
79    } // end method setCommissionRate
80
81    // return commission rate
82    public double getCommissionRate()
83    {
84       return commissionRate;
85    } // end method getCommissionRate
86
```

ommission
java



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
87    // set base salary
88    public void setBaseSalary( double salary )
89    {
90       baseSalary = ( salary < 0.0 ) ? 0.0 : salary;
91    } // end method setBaseSalary
92
93    // return base salary
94    public double getBaseSalary()
95    {
96       return baseSalary;
97    } // end method getBaseSalary
98
99    // calculate e
100      public double earnings()
101      {
102         return baseSalary + ( commissionRate * grossSales );
103      } // end method earnings
104
105      // return String representation of Ba
106      public String toString()
107      {
108         return String.format(
109            "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f\n%s: %.2f",
110            "base-salaried commission employee", fir
111            "social security number", socialSecurity
112            "gross sales", grossSales, "commission r
113            "base salary", baseSalary );
114      } // end method toString
115   } // end class BasePlusCommissionEmployee
```

Method `setBaseSalary` validates data and sets instance variable `baseSalary`

Method `getBaseSalary` returns the value of instance variable `baseSalary`

Update method `earnings` to calculate the earnings of a base-salaried commission employee

Update method `toString` to display base salary



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 9.7: BasePlusCommissionEmployeeTest.java
2  // Testing class BasePlusCommissionEmployee.
3
4  public class BasePlusCommissionEmployeeTest
5  {
6     public static void m
7     {
8        // instantiate BasePlusCommissionEmployee object
9        BasePlusCommissionEmployee employee =
10          new BasePlusCommissionEmployee(
11          "Bob", "Lewis", "333-33-3333", 5000, .04, 300 );
12
13       // get base-salaried commission employee data
14       System.out.println(
15          "Employee information o
16       System.out.printf( "%s %s\
17          employee.getFirstName() );
18       System.out.printf( "%s %s\n", "Last name is",
19          employee.getLastName() );
20       System.out.printf( "%s %s\n", "Social security number is",
21          employee.getSocialSecurityNumber() );
22       System.out.printf( "%s %.2f\n", "Gross sales is",
23          employee.getGrossSales() );
24       System.out.printf( "%s %.2f\n", "Commission rate is",
25          employee.getCommissionRate() );
26       System.out.printf( "%s %.2f\n", "Base salary is",
27          employee.getBaseSalary() );
28
```

Instantiate `BasePlusCommissionEmployee` object

Use `BasePluCommissionEmployee`'s *get* methods to retrieve the object's instance variable values



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Use `BasePlusCommissionEmployee`'s `setBaseSalary` methods to set base salary

Explicitly call object's `toString` method

```
29       employee.setBaseSalary( 1000
30
31       System.out.printf( "\n%s
32          "Updated employee inf
33           employee.toString() );
34    } // end main
35 } // end class BasePlusCommissionEmployeeTest
```

ommission
est.java

```
Employee information obtained by get methods:

First name is Bob
Last name is Lewis
Social security number is 333-33-3333
Gross sales is 5000.00
Commission rate is 0.04
Base salary is 300.00

Updated employee information obtained by toString:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 1000.00
```

tput



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.4.3 Creating a CommissionEmployee-BasePlusCommiionEmployee Inheritance Hierarchy

- Class `BasePlusCommissionEmployee2`
  - Extends class `CommissionEmployee`
  - Is a `CommissionEmployee`
  - Has instance variable `baseSalary`
  - Inherits `public` and `protected` members
  - Constructor not inherited



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

BasePlusCommission
ee2.java

Line 4

Line 13

```
1  // Fig. 9.8: BasePlusCommissionEmployee2.java
2  // BasePlusCommissionEmployee2 inherits from class CommissionEmployee.
3
4  public class BasePlusCommissionEmployee2 extends CommissionEmployee
5  {
6     private double baseSalary; // base salary per week
7
8     // six-argument constructor
9     public BasePlusCommissionEmployee2( String first, String last,
10       String ssn, double sales, double rate, double amount)
11    {
12       // explicit call to superclass CommissionEmployee constructor
13       super( first, last, ssn, sales, rate );
14
15       setBaseSalary( amount ); // validate and store base salary
16    } // end six-argument BasePlusCommissionEmployee2 constructor
17
18    // set base salary
19    public void setBaseSalary( double salary )
20    {
21       baseSalary = ( salary < 0.0 ) ? 0.0 : salary;
22    } // end method setBaseSalary
23
```

Class `BasePluCommissionEmployee2` is a subclass of `CommissionEmployee`

Invoke the superclass constructor using the superclass constructor call syntax



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

BasePlusCommission

(2 of 3)

Line 34

Lines 41-46

```
24    // return base salary
25    public double getBaseSalary()
26    {
27       return baseSalary;
28    } // end method getBaseSalary
29
30    // calculate earnings
31    public double earnings()
32    {
33       // not allowed: commissionRate and grossSales private in superclass
34       return baseSalary + ( commissionRate * grossSales );
35    } // end method earnings
36
37    // return String representation
38    public String toString()
39    {
40       // not allowed: attempts to a
41       return String.format(
42          "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f\n%s: %.2f",
43          "base-salaried commission employee", firstName, lastName,
44          "social security number", socialSecurityNumber,
45          "gross sales", grossSales, "commission rate", commissionRate,
46          "base salary", baseSalary );
47    } // end method toString
48 } // end class BasePlusCommissionEmployee2
```

Compiler generates errors because superclass's instance variable `commissionRate` and `grossSales` are `private`

Compiler generates errors because superclass's instance variable `firstName`, `lastName`, `socialSecurityNumber`, `grossSales` and `commissionRate` are `private`



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
BasePlusCommissionEmployee2.java:34: commissionRate has private access in
CommissionEmployee
      return baseSalary + ( commissionRate * grossSales );
                            ^
BasePlusCommissionEmployee2.java:34: grossSales has private access in
CommissionEmployee
      return baseSalary + ( commissionRate * grossSales );
                                             ^
BasePlusCommissionEmployee2.java:43: firstName has private access in
CommissionEmployee
         "base-salaried commission employee", firstName, lastName,
                                              ^
BasePlusCommissionEmployee2.java:43: lastName has private access in
CommissionEmployee
         "base-salaried commission employee", firstName, lastName,
                                                         ^
BasePlusCommissionEmployee2.java:44: socialSecurityNumber has private access in
CommissionEmployee
         "social security number", socialSecurityNumber,
                                   ^
BasePlusCommissionEmployee2.java:45: grossSales has private access in
CommissionEmployee
         "gross sales", grossSales, "commission rate", commissionRate,
                        ^
BasePlusCommissionEmployee2.java:45: commissionRate has private access in
CommissionEmployee
         "gross sales", grossSales, "commission rate", commissionRate,
                                                      ^
7 errors
```

Outline

BasePlusCommissionEmployee2.java

(3 of 3)

Compiler generated errorss



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.4.4 CommissionEmployee–BasePlusCommissionEmployee Inheritance Hierarchy Using protected Instance Variables

- Use `protected` instance variables
  - Enable class `BasePlusCommissionEmployee` to directly access superclass instance variables
  - Superclass's `protected` members are inherited by all subclases of that superclass



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 9.9: CommissionEmployee2.java
2  // CommissionEmployee2 class represents a commission employee.
3
4  public class CommissionEmployee2
5  {
6     protected String firstName;
7     protected String lastName;
8     protected String socialSecurityNumber;
9     protected double grossSales; // gross weekly sales
10    protected double commissionRate; // commission percentage
11
12    // five-argument constructor
13    public CommissionEmployee2( String first, String last, String ssn,
14       double sales, double rate )
15    {
16       // implicit call to Object constructor occurs here
17       firstName = first;
18       lastName = last;
19       socialSecurityNumber = ssn;
20       setGrossSales( sales ); // validate and store gross sales
21       setCommissionRate( rate ); // validate and store commission rate
22    } // end five-argument CommissionEmployee2 constructor
23
24    // set first name
25    public void setFirstName( String first )
26    {
27       firstName = first;
28    } // end method setFirstName
29
```

Declare `protected` instance variables

Outline

CommissionEmployee2.java

(1 of 4)

Line 6-10



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
30    // return first name
31    public String getFirstName()
32    {
33       return firstName;
34    } // end method getFirstName
35
36    // set last name
37    public void setLastName( String last )
38    {
39       lastName = last;
40    } // end method setLastName
41
42    // return last name
43    public String getLastName()
44    {
45       return lastName;
46    } // end method getLastName
47
48    // set social security number
49    public void setSocialSecurityNumber( String ssn )
50    {
51       socialSecurityNumber = ssn; // should validate
52    } // end method setSocialSecurityNumber
53
54    // return social security number
55    public String getSocialSecurityNumber()
56    {
57       return socialSecurityNumber;
58    } // end method getSocialSecurityNumber
59
```

Outline

Commission

Employee2.java

(2 of 4)



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
60    // set gross sales amount
61    public void setGrossSales( double sales )
62    {
63       grossSales = ( sales < 0.0 ) ? 0.0 : sales;
64    } // end method setGrossSales
65
66    // return gross sales amount
67    public double getGrossSales()
68    {
69       return grossSales;
70    } // end method getGrossSales
71
72    // set commission rate
73    public void setCommissionRate( double rate )
74    {
75       commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
76    } // end method setCommissionRate
77
78    // return commission rate
79    public double getCommissionRate()
80    {
81       return commissionRate;
82    } // end method getCommissionRate
83
84    // calculate earnings
85    public double earnings()
86    {
87       return commissionRate * grossSales;
88    } // end method earnings
89
```

Outline

Commission

Employee2.java

(3 of 4)



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
90    // return String representation of CommissionEmployee2 object
91    public String toString()
92    {
93       return String.format( "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
94          "commission employee", firstName, lastName,
95          "social security number", socialSecurityNumber,
96          "gross sales", grossSales,
97          "commission rate", commissionRate );
98    } // end method toString
99 } // end class CommissionEmployee2
```

Outline

Commission Employee2.java

(4 of 4)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

BasePlusCommissionEmployee3.java

```
1  // Fig. 9.10: BasePlusCommissionEmployee3.java
2  // BasePlusCommissionEmployee3 inherits from CommissionEmployee2 and has
3  // access to CommissionEmployee2's protected members.
4
5  public class BasePlusCommissionEmployee3 extends CommissionEmployee2
6  {
7     private double baseSalary; // base salary per week
8
9     // six-argument constructor
10    public BasePlusCommissionEmployee3( String first, Strin
11       String ssn, double sales, double rate, double salary
12    {
13       super( first, last, ssn, sales, rate );
14       setBaseSalary( salary ); // validate and store base salary
15    } // end six-argument BasePlusCommissionEmployee3 constructor
16
17    // set base salary
18    public void setBaseSalary( double salary )
19    {
20       baseSalary = ( salary < 0.0 ) ? 0.0 : salary;
21    } // end method setBaseSalary
22
23    // return base salary
24    public double getBaseSalary()
25    {
26       return baseSalary;
27    } // end method getBaseSalary
28
```

Must call superclass's constructor

of 2)

Line 13



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
29    // calculate earnings
30    public double earnings()
31    {
32       return baseSalary + ( commissionRate * grossSales );
33    } // end method earnings
34
35    // return String representation of BasePlusCommissionEmployee3
36    public String toString()
37    {
38       return String.format(
39          "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f\n%s: %.2f",
40          "base-salaried commission employee", firstName, lastName,
41          "social security number", socialSecurityNumber,
42          "gross sales", grossSales, "commission rate", commissionRate,
43          "base salary", baseSalary );
44    } // end method toString
45 } // end class BasePlusCommissionEmployee3
```

Directly access superclass's `protected` instance variables

Outline

BasePlusCommissionEmployee3.java

Line 32

Lines 38-43



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

BasePlusCommissionEmployeeTest3.java

(1 of 2)

```
1  // Fig. 9.11: BasePlusCommissionEmployeeTest3.java
2  // Testing class BasePlusCommissionEmployee3.
3
4  public class BasePlusCommissionEmployeeTest3
5  {
6     public static void main( String args[] )
7     {
8        // instantiate BasePlusCommissionEmployee3 object
9        BasePlusCommissionEmployee3 employee =
10          new BasePlusCommissionEmployee3(
11          "Bob", "Lewis", "333-33-3333", 5000, .04, 300 );
12
13       // get base-salaried commission employee data
14       System.out.println(
15          "Employee information obtained by get methods: \n" );
16       System.out.printf( "%s %s\n", "First name is",
17          employee.getFirstName() );
18       System.out.printf( "%s %s\n", "Last name is",
19          employee.getLastName() );
20       System.out.printf( "%s %s\n", "Social security number is",
21          employee.getSocialSecurityNumber() );
22       System.out.printf( "%s %.2f\n", "Gross sales is",
23          employee.getGrossSales() );
24       System.out.printf( "%s %.2f\n", "Commission rate is",
25          employee.getCommissionRate() );
26       System.out.printf( "%s %.2f\n", "Base salary is",
27          employee.getBaseSalary() );
28
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
29       employee.setBaseSalary( 1000 ); // set base salary
30
31       System.out.printf( "\n%s:\n\n%s\n",
32          "Updated employee information obtained by toString",
33          employee.toString() );
34    } // end main
35 } // end class BasePlusCommissionEmployeeTest3
```

```
Employee information obtained by get methods:

First name is Bob
Last name is Lewis
Social security number is 333-33-3333
Gross sales is 5000.00
Commission rate is 0.04
Base salary is 300.00

Updated employee information obtained by toString:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 1000.00
```

Outline

BasePlusCommissionEmployeeTest3.java

(2 of 2)

Program output



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.4.4 CommissionEmployee-BasePlusCommissionEmployee Inheritance Hierarchy Using protected Instance Variables (Cont.)

- Using `protected` instance variables
  - Advantages
    - subclasses can modify values directly
    - Slight increase in performance
      - Avoid set/get method call overhead
  - Disadvantages
    - No validity checking
      - subclass can assign illegal value
    - Implementation dependent
      - subclass methods more likely dependent on superclass implementation
      - superclass implementation changes may result in subclass modifications
        - Fragile (brittle) software



© 1992-2007 Pearson Education, Inc. All rights reserved.

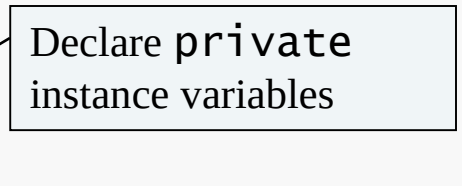
9.4.5 CommissionEmployee-BasePlusCommissionEmployee Inheritance Hierarchy Uing private Instance Variables

- **Reexamine hierarchy**
  - **Use the best software engineering practice**
    - **Declare instance variables as `private`**
    - **Provide public *get* and *set* methods**
    - **Use *get* method to obtain values of instance variables**



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 9.12: CommissionEmployee3.java
2  // CommissionEmployee3 class represents a commission employee.
3
4  public class CommissionEmployee3
5  {
6     private String firstName;
7     private String lastName;
8     private String socialSecurityNumber;
9     private double grossSales; // gross weekly sales
10    private double commissionRate; // commission percentage
11
12    // five-argument constructor
13    public CommissionEmployee3( String first, String last, String ssn,
14       double sales, double rate )
15    {
16       // implicit call to Object constructor occurs here
17       firstName = first;
18       lastName = last;
19       socialSecurityNumber = ssn;
20       setGrossSales( sales ); // validate and store gross sales
21       setCommissionRate( rate ); // validate and store commission rate
22    } // end five-argument CommissionEmployee3 constructor
23
24    // set first name
25    public void setFirstName( String first )
26    {
27       firstName = first;
28    } // end method setFirstName
29
```



Outline

Commission Employee3.java

(1 of 4)

Lines 6-10



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
30    // return first name
31    public String getFirstName()
32    {
33       return firstName;
34    } // end method getFirstName
35
36    // set last name
37    public void setLastName( String last )
38    {
39       lastName = last;
40    } // end method setLastName
41
42    // return last name
43    public String getLastName()
44    {
45       return lastName;
46    } // end method getLastName
47
48    // set social security number
49    public void setSocialSecurityNumber( String ssn )
50    {
51       socialSecurityNumber = ssn; // should validate
52    } // end method setSocialSecurityNumber
53
54    // return social security number
55    public String getSocialSecurityNumber()
56    {
57       return socialSecurityNumber;
58    } // end method getSocialSecurityNumber
59
```

Outline

Commission

Employee3.java

(2 of 4)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Commission

Employee3.java

(3 of 4)

```
60    // set gross sales amount
61    public void setGrossSales( double sales )
62    {
63       grossSales = ( sales < 0.0 ) ? 0.0 : sales;
64    } // end method setGrossSales
65
66    // return gross sales amount
67    public double getGrossSales()
68    {
69       return grossSales;
70    } // end method getGrossSales
71
72    // set commission rate
73    public void setCommissionRate( double rate )
74    {
75       commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
76    } // end method setCommissionRate
77
78    // return commission rate
79    public double getCommissionRate()
80    {
81       return commissionRate;
82    } // end method getCommissionRate
83
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
84    // calculate earnings
85    public double earnings()
86    {
87       return getCommissionRate() * getGrossSales();
88    } // end method earnings
89
90    // return String representation of CommissionEmployee
91    public String toString()
92    {
93       return String.format( "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
94          "commission employee", getFirstName(), getLastName(),
95          "social security number", getSocialSecurityNumber(),
96          "gross sales", getGrossSales(),
97          "commission rate", getCommissionRate() );
98    } // end method toString
99 } // end class CommissionEmployee3
```

Use *get* methods to obtain the values of instance variables

Outline

ission

Employee3.java

(4 of 4)

Line 87

Lines 94-97



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

BasePlusCommission
Employee4.java

```
1  // Fig. 9.13: BasePlusCommissionEmployee4.java
2  // BasePlusCommissionEmployee4 class inherits from CommissionEmployee3 and
3  // accesses CommissionEmployee3's private data via CommissionEmployee3's
4  // public methods.
5
6  public class BasePlusCommissionEmployee4 extends CommissionEmployee3
7  {
8     private double baseSalary; // base salary per week
9
10    // six-argument constructor
11    public BasePlusCommissionEmployee4( String first, String last,
12       String ssn, double sales, double rate, double salary )
13    {
14       super( first, last, ssn, sales, rate );
15       setBaseSalary( salary ); // validate and store base salary
16    } // end six-argument BasePlusCommissionEmployee4 constructor
17
18    // set base salary
19    public void setBaseSalary( double salary )
20    {
21       baseSalary = ( salary < 0.0 ) ? 0.0 : salary;
22    } // end method setBaseSalary
23
```

Inherits from
CommissionEmployee3



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
24    // return base salary
25    public double getBaseSalary()
26    {
27       return baseSalary;
28    } // end method getBaseSalary
29
30    // calculate earnings
31    public double earnings()
32    {
33       return getBaseSalary() + super.earnings();
34    } // end method earnings
35
36    // return String representation of BasePlusCommissionEmploye
37    public String toString()
38    {
39       return String.format( "%s %s\n%s: %.2f", "base-salaried",
40          super.toString(), "base salary", getBaseSalary() );
41    } // end method toString
42 } // end class BasePlusCommissionEmployee4
```

Invoke an overridden superclass method from a subclass

(2 of 2)

Use *get* methods to obtain the values of instance variables

Lines 40

Invoke an overridden superclass method from a subclass



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 9.14: BasePlusCommissionEmployeeTest4.java
2  // Testing class BasePlusCommissionEmployee4.
3
4  public class BasePlusCommissionEmployeeTest4
5  {
6     public static void main( String args[] )
7     {
8        // instantiate BasePlusCommissionEmployee4 object
9        BasePlusCommissionEmployee4 employee =
10          new BasePlusCommissionEmployee4(
11          "Bob", "Lewis", "333-33-3333", 5000, .04, 300 );
12
13       // get base-salaried commission employee data
14       System.out.println(
15          "Employee information obtained by get methods: \n" );
16       System.out.printf( "%s %s\n", "First name is",
17          employee.getFirstName() );
18       System.out.printf( "%s %s\n", "Last name is",
19          employee.getLastName() );
20       System.out.printf( "%s %s\n", "Social security number is",
21          employee.getSocialSecurityNumber() );
22       System.out.printf( "%s %.2f\n", "Gross sales is",
23          employee.getGrossSales() );
24       System.out.printf( "%s %.2f\n", "Commission rate is",
25          employee.getCommissionRate() );
26       System.out.printf( "%s %.2f\n", "Base salary
27          employee.getBaseSalary() );
28
```

Create `BasePlusCommissionEmployee4` object.

Lines 9-11

Lines 16-25

Use inherited *get* methods to access inherited `private` instance variables

Use `BasePlusCommissionEmployee4` *get* method to access `private` instance variable.



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
29       employee.setBaseSalary( 1000 ); // set base salary
30
31       System.out.printf( "\n%s:\n\n%s\n",
32          "Updated employee information obtaine
33          employee.toString() );
34    } // end main
35 } // end class BasePlusCommissionEmployeeTest4
```

Use `BasePlusCommissionEmployee4` *set* method to modify `private` instance variable `baseSalary`.

```
Employee information obtained by get methods:

First name is Bob
Last name is Lewis
Social security number is 333-33-3333
Gross sales is 5000.00
Commission rate is 0.04
Base salary is 300.00

Updated employee information obtained by toString:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 1000.00
```

Outline

on

EmployeeTest4.java

(2 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.5 Constructors in Subclasses

- **Instantiating subclass object**
  - **Chain of constructor calls**
    - **subclass constructor invokes superclass constructor**
      - **Implicitly or explicitly**
    - **Base of inheritance hierarchy**
      - **Last constructor called in chain is `Object`'s constructor**
      - **Original subclass constructor's body finishes executing last**
      - **Example: `CommissionEmployee3-BasePlusCommissionEmployee4` hierarchy**
        - **`CommissionEmployee3` constructor called second last (last is `Object` constructor)**
        - **`CommissionEmployee3` constructor's body finishes execution second (first is `Object` constructor's body)**



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 9.15: CommissionEmployee4.java
2  // CommissionEmployee4 class represents a commission employee.
3
4  public class CommissionEmployee4
5  {
6     private String firstName;
7     private String lastName;
8     private String socialSecurityNumber;
9     private double grossSales; // gross weekly sales
10    private double commissionRate; // commission percentage
11
12    // five-argument constructor
13    public CommissionEmployee4( String first, String last, String ssn,
14       double sales, double rate )
15    {
16       // implicit call to Object constructor occurs here
17       firstName = first;
18       lastName = last;
19       socialSecurityNumber = ssn;
20       setGrossSales( sales ); // validate and store gross sales
21       setCommissionRate( rate ); // validate and store commission rate
22
23       System.out.printf(
24          "\nCommissionEmployee4 constructor:\n%s\n", this );
25    } // end five-argument CommissionEmployee4 constructor
26
```

nEmployee

Constructor outputs message to demonstrate method call order.



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
27    // set first name
28    public void setFirstName( String first )
29    {
30       firstName = first;
31    } // end method setFirstName
32
33    // return first name
34    public String getFirstName()
35    {
36       return firstName;
37    } // end method getFirstName
38
39    // set last name
40    public void setLastName( String last )
41    {
42       lastName = last;
43    } // end method setLastName
44
45    // return last name
46    public String getLastName()
47    {
48       return lastName;
49    } // end method getLastName
50
51    // set social security number
52    public void setSocialSecurityNumber( String ssn )
53    {
54       socialSecurityNumber = ssn; // should validate
55    } // end method setSocialSecurityNumber
```

e

sionEmployee



All rights reserved.

```
57    // return social security number
58    public String getSocialSecurityNumber()
59    {
60       return socialSecurityNumber;
61    } // end method getSocialSecurityNumber
62
63    // set gross sales amount
64    public void setGrossSales( double sales )
65    {
66       grossSales = ( sales < 0.0 ) ? 0.0 : sales;
67    } // end method setGrossSales
68
69    // return gross sales amount
70    public double getGrossSales()
71    {
72       return grossSales;
73    } // end method getGrossSales
74
75    // set commission rate
76    public void setCommissionRate( double rate )
77    {
78       commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
79    } // end method setCommissionRate
80
```

© 1992-2007 Pearson Education, Inc. All rights reserved.

```
81    // return commission rate
82    public double getCommissionRate()
83    {
84       return commissionRate;
85    } // end method getCommissionRate
86
87    // calculate earnings
88    public double earnings()
89    {
90       return getCommissionRate() * getGrossSales();
91    } // end method earnings
92
93    // return String representation of CommissionEmployee4 object
94    public String toString()
95    {
96       return String.format( "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
97          "commission employee", getFirstName(), getLastName(),
98          "social security number", getSocialSecurityNumber(),
99          "gross sales", getGrossSales(),
100         "commission rate", getCommissionRate() );
101    } // end method toString
102 } // end class CommissionEmployee4
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 9.16: BasePlusCommissionEmployee5.java
2  // BasePlusCommissionEmployee5 class declaration.
3
4  public class BasePlusCommissionEmployee5 extends CommissionEmployee4
5  {
6     private double baseSalary; // base salary per week
7
8     // six-argument constructor
9     public BasePlusCommissionEmployee5( String first, String last,
10       String ssn, double sales, doub
11    {
12       super( first, last, ssn, sales, rate );
13       setBaseSalary( salary ); // validate and store base salary
14
15       System.out.printf(
16          "\nBasePlusCommissionEmployee5 constructor:\n%s\n", this );
17    } // end six-argument BasePlusCommissionEmployee5 constructor
18
19    // set base salary
20    public void setBaseSalary( double salary )
21    {
22       baseSalary = ( salary < 0.0 ) ? 0.0 : salary;
23    } // end method setBaseSalary
24
```

Constructor outputs message to demonstrate method call order.



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
25    // return base salary
26    public double getBaseSalary()
27    {
28       return baseSalary;
29    } // end method getBaseSalary
30
31    // calculate earnings
32    public double earnings()
33    {
34       return getBaseSalary() + super.earnings();
35    } // end method earnings
36
37    // return String representation of BasePlusCommissionEmployee5
38    public String toString()
39    {
40       return String.format( "%s %s\n%s: %.2f", "base-salaried",
41          super.toString(), "base salary", getBaseSalary() );
42    } // end method toString
43 } // end class BasePlusCommissionEmployee5
```

on



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 9.17: ConstructorTest.java
2  // Display order in which superclass and subclass constructors are called.
3
4  public class ConstructorTest
5  {
6     public static void main( String args[] )
7     {
8        CommissionEmployee4 employee1 = new CommissionEmployee4(
9           "Bob", "Lewis", "333-33-3333", 5000, .04 );
10
11       System.out.println();
12       BasePlusCommissionEmployee5 employee2 =
13          new BasePlusCommissionEmployee5(
14          "Lisa", "Jones", "555-55-5555", 2000, .06, 800 );
15
16       System.out.println();
17       BasePlusCommissionEmployee5 employee3 =
18          new BasePlusCommissionEmployee5(
19          "Mark", "Sands", "888-88-8888", 8000, .15, 2000 );
20    } // end main
21 } // end class ConstructorTest
```

Instantiate `CommissionEmployee4` object

Instantiate two `BasePlusCommissionEmployee5` objects to demonstrate order of subclass and superclass constructor method calls.



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
CommissionEmployee4 constructor:
commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04

CommissionEmployee4 constructor:
base-salaried commission employee: Lisa Jones
social security number: 555-55-5555
gross sales: 2000.00
commission rate: 0.06
base salary: 0.00

BasePlusCommissionEmployee5 constructor:
base-salaried commission employee: Lisa Jones
social security number: 555-55-5555
gross sales: 2000.00
commission rate: 0.06
base salary: 800.00


CommissionEmployee4 constructor:
base-salaried commission employee: Mark Sands
social security number: 888-88-8888
gross sales: 8000.00
commission rate: 0.15
base salary: 0.00

BasePlusCommissionEmployee5 constructor:
base-salaried commission employee: Mark Sands
social security number: 888-88-8888
gross sales: 8000.00
commission rate: 0.15
base salary: 2000.00
```

Subclass `BasePlusCommissionEmployee5` constructor body executes after superclass `CommissionEmployee4`'s constructor finishes execution.



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.6 Software Engineering with Inheritance

- **Customizing existing software**
  - **Inherit from existing classes**
    - **Include additional members**
    - **Redefine superclass members**
    - **No direct access to superclass's source code**
      - **Link to object code**
  - **Independent software vendors (ISVs)**
    - **Develop proprietary code for sale/license**
      - **Available in object-code format**
    - **Users derive new classes**
      - **Without accessing ISV proprietary source code**



© 1992-2007 Pearson Education, Inc. All rights reserved.

9.7 `Object` Class

- Class `Object` methods
  - `clone`
  - `equals`
  - `finalize`
  - `getClass`
  - `hashCode`
  - `notify`, `notifyAll`, `wait`
  - `toString`



© 1992-2007 Pearson Education, Inc. All rights reserved.

Method	Description
`clone`	This `protected` method, which takes no arguments and returns an `Object` reference, makes a copy of the object on which it is called. When cloning is required for objects of a class, the class should override method `clone` as a `public` method and should implement interface `Cloneable` (package `java.lang`). The default implementation of this method performs a so-called shallow copy—instance variable values in one object are copied into another object of the same type. For reference types, only the references are copied. A typical overridden `clone` method's implementation would perform a deep copy that creates a new object for each reference type instance variable. There are many subtleties to overriding method `clone`. You can learn more about cloning in the following article: `java.sun.com/developer/JDCTechTips/2001/tt0306.html`

Fig. 9.18 | Object methods that are inherited directly or indirectly by all classes. (Part 1 of 4)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Method	Description
`Equals`	This method compares two objects for equality and returns `true` if they are equal and `false` otherwise. The method takes any `Object` as an argument. When objects of a particular class must be compared for equality, the class should override method `equals` to compare the contents of the two objects. The method's implementation should meet the following requirements: • It should return `false` if the argument is `null`. • It should return `true` if an object is compared to itself, as in `object1.equals( object1 )`. • It should return `true` only if both `object1.equals( object2 )` and `object2.equals( object1 )` would return `true`. • For three objects, if `object1.equals( object2 )` returns `true` and `object2.equals( object3 )` returns `true`, then `object1.equals( object3 )` should also return `true`. • If `equals` is called multiple times with the two objects and the objects do not change, the method should consistently return `true` if the objects are equal and `false` otherwise. A class that overrides `equals` should also override `hashCode` to ensure that equal objects have identical hashcodes. The default `equals` implementation uses operator == to determine whether two references *refer to the same object* in memory. Section 29.3.3 demonstrates class `String`'s `equals` method and differentiates between comparing `String` objects with == and with `equals`.

Fig. 9.18 | `Object` methods that are inherited directly or indirectly by all classes. (Part 2 of 4)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Method	Description
`finalize`	This `protected` method (introduced in Section 8.10 and Section 8.11) is called by the garbage collector to perform termination housekeeping on an object just before the garbage collector reclaims the object's memory. It is not guaranteed that the garbage collector will reclaim an object, so it cannot be guaranteed that the object's `finalize` method will execute. The method must specify an empty parameter list and must return `void`. The default implementation of this method serves as a placeholder that does nothing.
`getClass`	Every object in Java knows its own type at execution time. Method `getClass` (used in Section 10.5 and Section 21.3) returns an object of class `Class` (package `java.lang`) that contains information about the object's type, such as its class name (returned by `Class` method `getName`). You can learn more about class `Class` in the online API documentation at `java.sun.com/j2se/5.0/docs/api/java/lang/Class.html`.

Fig. 9.18 | Object methods that are inherited directly or indirectly by all classes. (Part 3 of 4)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Method	Description
`hashCode`	A hashtable is a data structure (discussed in Section 19.10) that relates one object, called the key, to another object, called the value. When initially inserting a value into a hashtable, the key's `hashCode` method is called. The hashcode value returned is used by the hashtable to determine the location at which to insert the corresponding value. The key's hashcode is also used by the hashtable to locate the key's corresponding value.
`notify`, `notifyAll`, `wait`	Methods `notify`, `notifyAll` and the three overloaded versions of `wait` are related to multithreading, which is discussed in Chapter 23. In J2SE 5.0, the multithreading model has changed substantially, but these features continue to be supported.
`toString`	This method (introduced in Section 9.4.1) returns a `String` representation of an object. The default implementation of this method returns the package name and class name of the object's class followed by a hexadecimal representation of the value returned by the object's `hashCode` method.

Fig. 9.18 | Object methods that are inherited directly or indirectly by all classes. (Part 4 of 4)



© 1992-2007 Pearson Education, Inc. All rights reserved.

10

Object-Oriented Programming: Polymorphism



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.1 Introduction

- **Polymorphism**
  - **Enables “programming in the general”**
  - **The same invocation can produce “many forms” of results**
- **Interfaces**
  - **Implemented by classes to assign common functionality to possibly unrelated classes**



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.2 Polymorphism Examples

- **Polymorphism**
  - **When a program invokes a method through a superclass variable, the correct subclass version of the method is called, based on the type of the reference stored in the superclass variable**
  - **The same method name and signature can cause different actions to occur, depending on the type of object on which the method is invoked**
  - **Facilitates adding new classes to a system with minimal modifications to the system's code**



© 1992-2007 Pearson Education, Inc. All rights reserved.

- **What is polymorphism:**
- **Polymorphism in Java** is a concept by which we can perform a *single action in different ways*. Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms. So polymorphism means many forms.
- **Why polymorphism:**
  - Reusability.
  - Override.
  - in General



© 1992-2007 Pearson Education, Inc. All rights reserved.

- Types of polymorphism
  1. runtime polymorphism (overriding).
  2. compile-time polymorphism (overloading).



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.3 Demonstrating Polymorphic Behavior

- **A superclass reference can be aimed at a subclass object**
  - **This is possible because a subclass object *is a* superclass object as well**
  - **When invoking a method from that reference, the type of the actual referenced object, not the type of the reference, determines which method is called**
- **A subclass reference can be aimed at a superclass object only if the object is downcasted**



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 10.1: PolymorphismTest.java
2  // Assigning superclass and subclass references to superclass and
3  // subclass variables.
4
5  public class PolymorphismTest
6  {
7     public static void main( String args[] )
8     {
9        // assign superclass reference to superclass variable
10       CommissionEmployee3 commissionEmployee = new CommissionEmployee3(
11          "Sue", "Jones", "222-22-2222", 10000, .06 );
12
13       // assign subclass reference to subclass variable
14       BasePlusCommissionEmployee4 basePlusCommissionEmployee =
15          new BasePlusCommissionEmployee4(
16          "Bob", "Lewis", "333-33-3333", 5000, .04, 300 );
17
18       // invoke toString on superclass object using superclass variable
19       System.out.printf( "%s %s:\n\n%s\n\n",
20          "Call CommissionEmployee3's toString with superclass reference ",
21          "to superclass object", commissionEmployee.toString() );
22
23       // invoke toString on subclass object using subclass variable
24       System.out.printf( "%s %s:\n\n%s\n\n",
25          "Call BasePlusCommissionEmployee4's toString with subclass",
26          "reference to subclass object",
27          basePlusCommissionEmployee.toString() );
28
```

Typical reference assignments



All rights reserved.

```
29       // invoke toString on subclass object using super
30       CommissionEmployee3 commissionEmployee2 =
31          basePlusCommissionEmployee;
32       System.out.printf( "%s %s:\n\n%s\n",
33          "Call BasePlusCommissionEmployee4's toString with superclass",
34          "reference to subclass object", commissionEmployee2.toString() );
35    } // end main
36 } // end class PolymorphismTest
```

Assign a reference to a **basePlusCommissionEmployee** object to a **CommissionEmployee3** variable

PolymorphismTest.java

Polymorphically call **basePlusCommissionEmployee**'s **toString** method

```
Call CommissionEmployee3's toString with supercla
object:

commission employee: Sue Jones
social security number: 222-22-2222
gross sales: 10000.00
commission rate: 0.06

Call BasePlusCommissionEmployee4's toString with subclass reference to
subclass object:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 300.00

Call BasePlusCommissionEmployee4's toString with superclass reference to
subclass object:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 300.00
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.4 Abstract Classes and Methods

- **Abstract classes**
  - **Classes that are too general to create real objects**
  - **Used only as abstract superclasses for concrete subclasses and to declare reference variables**
  - **Many inheritance hierarchies have abstract superclasses occupying the top few levels**
  - **Keyword `abstract`**
    - **Use to declare a class `abstract`**
    - **Also use to declare a method `abstract`**
      - **Abstract classes normally contain one or more abstract methods**
      - **All concrete subclasses must override all inherited abstract methods**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Abstract Classes

- **Abstraction** is a process of hiding the implementation details and showing only functionality to the user.
- Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.



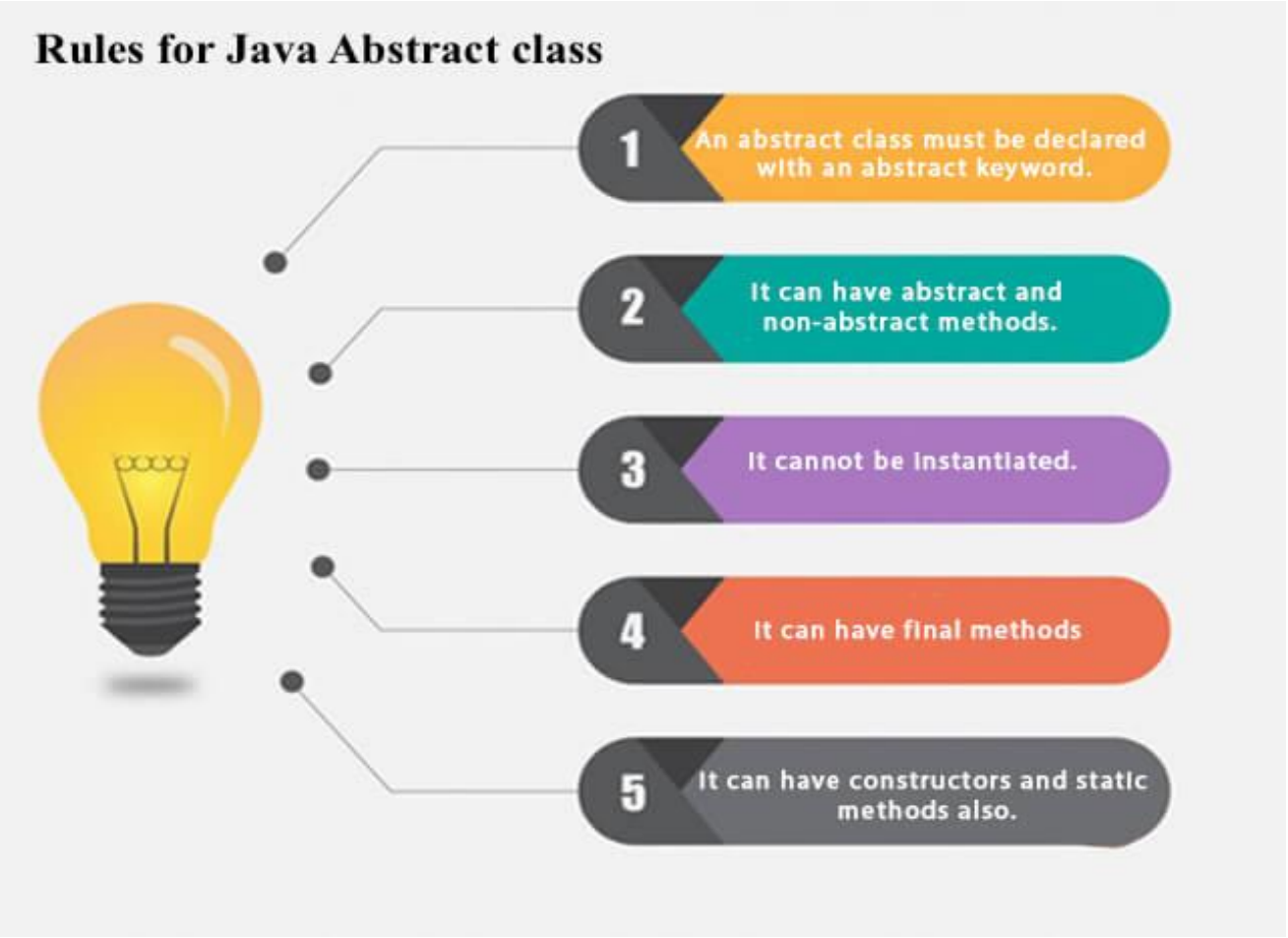
© 1992-2007 Pearson Education, Inc. All rights reserved.

Abstract class in Java

- A class which is declared as abstract is known as an **abstract class**. It can have abstract and non-abstract methods. It needs to be extended and its method implemented. It cannot be instantiated.
-



© 1992-2007 Pearson Education, Inc. All rights reserved.





© 1992-2007 Pearson Education, Inc. All rights reserved.

Abstract Method in Java

- **ABSTRACT METHOD** in Java, is a method that has just the method definition but does not contain implementation. A method without a body is known as an Abstract Method. It must be declared in an abstract class. The abstract method will never be final because the abstract class must implement all the abstract methods.
- **public abstract void MethodName();**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Interfaces

- An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.

- Why use Java interface?
  - It is used to achieve abstraction.
  - By interface, we can support the functionality of multiple inheritance.



© 1992-2007 Pearson Education, Inc. All rights reserved.

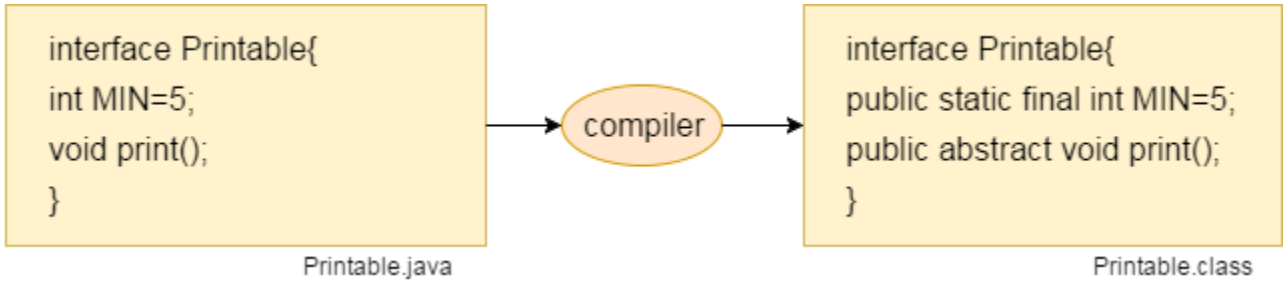
Interfaces

- How to declare an interface?
  - An interface is declared by using the interface keyword. It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default. A class that implements an interface must implement all the methods declared in the interface.
- Syntax:

```
interface <interface_name>{
    // declare constant fields
   // declare methods that abstract
}
```

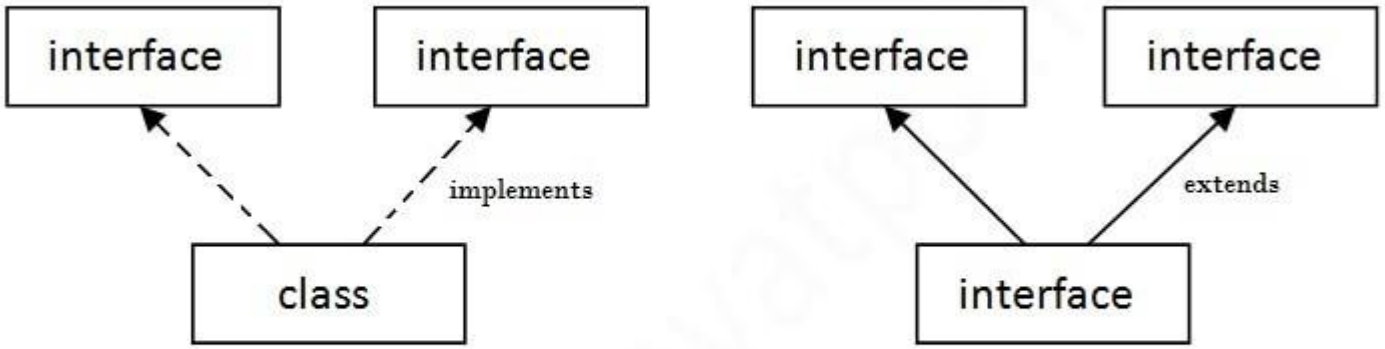


© 1992-2007 Pearson Education, Inc. All rights reserved.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Multiple inheritance in Java by interface



Multiple Inheritance in Java

© 1992-2007 Pearson Education, Inc. All rights reserved.

10.4 Abstract Classes and Methods (Cont.)

- `Iterator` class
  - Traverses all the objects in a collection, such as an array
  - Often used in polymorphic programming to traverse a collection that contains references to objects from various levels of a hierarchy



© 1992-2007 Pearson Education, Inc. All rights reserved.

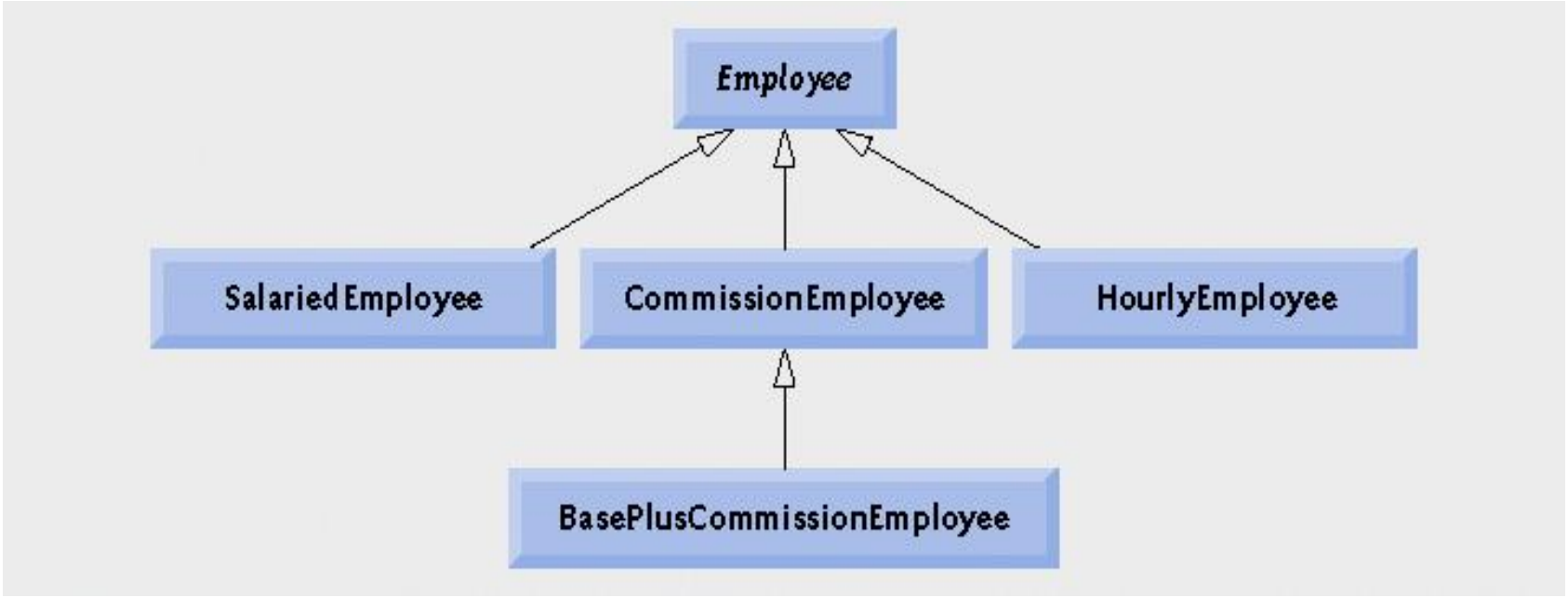


Fig. 10.2 | `Employee` hierarchy UML class diagram.



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.5.1 Creating Abstract Superclass `Employee`

- **`abstract` superclass `Employee`**
  - **`earnings` is declared `abstract`**
    - **No implementation can be given for `earnings` in the `Employee` `abstract` class**
  - **An array of `Employee` variables will store references to subclass objects**
    - **`earnings` method calls from these variables will call the appropriate version of the `earnings` method**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Attributes common to all employees

stract class Employee

```
1  // Fig. 10.4: Employee.java
2  // Employee abstract superclass.
3
4  public abstract class Employee
5  {
6     private String firstName;
7     private String lastName;
8     private String socialSecurityNumber;
9
10    // three-argument constructor
11    public Employee( String first, String last, String ssn )
12    {
13       firstName = first;
14       lastName = last;
15       socialSecurityNumber = ssn;
16    } // end three-argument Employee constructor
17
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
18    // set first name
19    public void setFirstName( String first )
20    {
21       firstName = first;
22    } // end method setFirstName
23
24    // return first name
25    public String getFirstName()
26    {
27       return firstName;
28    } // end method getFirstName
29
30    // set last name
31    public void setLastName( String last )
32    {
33       lastName = last;
34    } // end method setLastName
35
36    // return last name
37    public String getLastName()
38    {
39       return lastName;
40    } // end method getLastName
41
```

Outline

Employee.java

(2 of 3)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Employee.java

(3 of 3)

```
42    // set social security number
43    public void setSocialSecurityNumber( String ssn )
44    {
45       socialSecurityNumber = ssn; // should validate
46    } // end method setSocialSecurityNumber
47
48    // return social security number
49    public String getSocialSecurityNumber()
50    {
51       return socialSecurityNumber;
52    } // end method getSocialSecurityNumber
53
54    // return String representation of Employee object
55    public String toString()
56    {
57       return String.format( "%s %s\nsocial security number: %s",
58          getFirstName(), getLastName(), getSocialSecurityNumber() );
59    } // end method toString
60
61    // abstract method overridden by subclasses
62    public abstract double earnings(); // no implementation here
63 } // end abstract class Employee
```

`abstract` method **`earnings`** has no implementation



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Class **SalariedEmployee** extends class **Employee**

```
1  // Fig. 10.5: SalariedEmployee.java
2  // SalariedEmployee class extends Employee.
3
4  public class SalariedEmployee extends Employee
5  {
6     private double weeklySalary;
7
8     // four-argument constructor
9     public SalariedEmployee( String first, String las
10       double salary )
11    {
12       super( first, last, ssn ); // pass to Employee constructor
13       setWeeklySalary( salary ); // validate and s
14    } // end four-argument SalariedEmployee constructor
15
16    // set salary
17    public void setWeeklySalary( double salary )
18    {
19       weeklySalary = salary < 0.0 ? 0.0 : salary;
20    } // end method setWeeklySalary
21
```

Call superclass constructor

Call **setWeeklySalary** method

Validate and set weekly salary value



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

SalariedEmployee.java

```
22    // return salary
23    public double getWeeklySalary()
24    {
25       return weeklySalary;
26    } // end method getWeeklySalary
27
28    // calculate earnings; override abstract method earnings in Employee
29    public double earnings()
30    {
31       return getWeeklySalary();
32    } // end method earnings
33
34    // return String representation of SalariedEmployee object
35    public String toString()
36    {
37       return String.format( "salaried employee: %s\n%s: $%,.2f",
38          super.toString(), "weekly salary", getWeeklySalary() );
39    } // end method toString
40 } // end class SalariedEmployee
```

Override **earnings** method so **SalariedEmployee** can be concrete

(2 of 2)

Override **toString** method

Call superclass's version of **toString**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

HourlyEmployee

.java

(1 of 2)

```
1  // Fig. 10.6: HourlyEmployee.java
2  // HourlyEmployee class extends Employee.
3
4  public class HourlyEmployee extends Employee
5  {
6     private double wage; // wage per hour
7     private double hours; // hours worked for week
8
9     // five-argument constructor
10    public HourlyEmployee( String first, String last, String ssn,
11       double hourlyWage, double hoursWorked )
12    {
13       super( first, last, ssn );
14       setWage( hourlyWage ); // validate hourly wage
15       setHours( hoursWorked ); // validate hours worked
16    } // end five-argument HourlyEmployee constructor
17
18    // set wage
19    public void setWage( double hourlyWage )
20    {
21       wage = ( hourlyWage < 0.0 ) ? 0.0 : hourlyWage;
22    } // end method setWage
23
24    // return wage
25    public double getWage()
26    {
27       return wage;
28    } // end method getWage
29
```

Class **HourlyEmployee** extends class **Employee**

Call superclass constructor

Validate and set hourly wage value



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

HourlyEmployee .java

(2 of 2)

```
30    // set hours worked
31    public void setHours( double hoursWorked )
32    {
33       hours = ( ( hoursWorked >= 0.0 ) && ( hoursWorked <= 168.0 ) ) ?
34          hoursWorked : 0.0;
35    } // end method setHours
36
37    // return hours worked
38    public double getHours()
39    {
40       return hours;
41    } // end method getHours
42
43    // calculate earnings; override abstract method earnings in Employee
44    public double earnings()
45    {
46       if ( getHours() <= 40 ) // no overtime
47          return getWage() * getHours();
48       else
49          return 40 * getWage() + ( gethours() - 40 ) * getWage() * 1.5;
50    } // end method earnings
51
52    // return String representation of HourlyEmployee object
53    public String toString()
54    {
55       return String.format( "hourly employee: %s\n%s: $%,.2f; %s: %,.2f",
56          super.toString(), "hourly wage", getWage(),
57          "hours worked", getHours() );
58    } // end method toString
59 } // end class HourlyEmployee
```

Validate and set hours worked value

Override **earnings** method so **HourlyEmployee** can be concrete

Override **toString** method

Call superclass's **toString** method



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 10.7: CommissionEmployee.java
2  // CommissionEmployee class extends Employee.
3
4  public class CommissionEmployee extends Employee
5  {
6     private double grossSales; // gross weekly sales
7     private double commissionRate; // commission percentage
8
9     // five-argument constructor
10    public CommissionEmployee( String first, String last, String ssn,
11       double sales, double rate )
12    {
13       super( first, last, ssn );
14       setGrossSales( sales );
15       setCommissionRate( rate );
16    } // end five-argument CommissionEmployee constructor
17
18    // set commission rate
19    public void setCommissionRate( double rate )
20    {
21       commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
22    } // end method setCommissionRate
23
```

Class **CommissionEmployee** extends class **Employee**

CommissionEmployee.java

(1 of 3)

Call superclass constructor

Validate and set commission rate value



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

CommissionEmployee.java

(2 of 3)

```
24    // return commission rate
25    public double getCommissionRate()
26    {
27       return commissionRate;
28    } // end method getCommissionRate
29
30    // set gross sales amount
31    public void setGrossSales( double sales )
32    {
33       grossSales = ( sales < 0.0 ) ? 0.0 : sales;
34    } // end method setGrossSales
35
36    // return gross sales amount
37    public double getGrossSales()
38    {
39       return grossSales;
40    } // end method getGrossSales
41
```

Validate and set the gross sales value



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
42    // calculate earnings; override abstract method earnings in Employee
43    public double earnings()
44    {
45       return getCommissionRate() * getGrossSales();
46    } // end method earnings
47
48    // return String representation of CommissionEmployee object
49    public String toString()
50    {
51       return String.format( "%s: %s\n%s: $%,.2f; %s: %.2f",
52          "commission employee", super.toString(),
53          "gross sales", getGrossSales(),
54          "commission rate", getCommissionRate() );
55    } // end method toString
56 } // end class CommissionEmployee
```

Override **`earnings`** method so **`CommissionEmployee`** can be concrete

`CommissionEmployee.java`

Override **`toString`** method

(3 of 3)

Call superclass's **`toString`** method



© 1992-2007 Pearson Education, Inc. All rights reserved.

Class **BasePlusCommissionEmployee** extends class **CommissionEmployee**

Outline

BasePlusCommissionEmployee.java

```
1  // Fig. 10.8: BasePlusCommissionEmp
2  // BasePlusCommissionEmployee clas
3
4  public class BasePlusCommissionEmployee extends CommissionEmployee
5  {
6     private double baseSalary; // base salary per week
7
8     // six-argument constructor
9     public BasePlusCommissionEmployee( String first, String last,
10       String ssn, double sales, double rate, double salary )
11    {
12       super( first, last, ssn, sales, rate );
13       setBaseSalary( salary ); // validate and store base salary
14    } // end six-argument BasePlusCommissionEmployee constructor
15
16    // set base salary
17    public void setBaseSalary( double salary )
18    {
19       baseSalary = ( salary < 0.0 ) ? 0.0 : salary; // non-negative
20    } // end method setBaseSalary
21
```

Call superclass constructor

(1 of 2)

Validate and set base salary value



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

BasePlusCommissionEmployee.java

```
22    // return base salary
23    public double getBaseSalary()
24    {
25       return baseSalary;
26    } // end method getBaseSalary
27
28    // calculate earnings; override method earnings in CommissionEmployee
29    public double earnings()
30    {
31       return getBaseSalary() + super.earnings();
32    } // end method earnings
33
34    // return String representation of BasePlusCommissionEmployee object
35    public String toString()
36    {
37       return String.format( "%s %s; %s: $%,.2f",
38          "base-salaried", super.toString(),
39          "base salary", getBaseSalary() );
40    } // end method toString
41 } // end class BasePlusCommissionEmployee
```

Override **earnings** method

Call superclass's **earnings** method

(2 of 2)

Override **toString** method

Call superclass's **toString** method



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

PayrollSystemTest.java

(1 of 5)

```
1  // Fig. 10.9: PayrollSystemTest.java
2  // Employee hierarchy test program.
3
4  public class PayrollSystemTest
5  {
6     public static void main( String args[] )
7     {
8        // create subclass objects
9        SalariedEmployee salariedEmployee =
10          new SalariedEmployee( "John", "Smith", "111-11-1111", 800.00 );
11       HourlyEmployee hourlyEmployee =
12          new HourlyEmployee( "Karen", "Price", "222-22-2222", 16.75, 40 );
13       CommissionEmployee commissionEmployee =
14          new CommissionEmployee(
15          "Sue", "Jones", "333-33-3333", 10000, .06 );
16       BasePlusCommissionEmployee basePlusCommissionEmployee =
17          new BasePlusCommissionEmployee(
18          "Bob", "Lewis", "444-44-4444", 5000, .04, 300 );
19
20       System.out.println( "Employees processed individually:\n" );
21
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

PayrollSystemTest.java

(2 of 5)

```
22       System.out.printf( "%s\n%s: $%,.2f\n\n",
23          salariedEmployee, "earned", salariedEmployee.earnings() );
24       System.out.printf( "%s\n%s: $%,.2f\n\n",
25          hourlyEmployee, "earned", hourlyEmployee.earnings() );
26       System.out.printf( "%s\n%s: $%,.2f\n\n",
27          commissionEmployee, "earned", commissionEmployee.earnings() );
28       System.out.printf( "%s\n%s: $%,.2f\n\n",
29          basePlusCommissionEmployee,
30          "earned", basePlusCommissionEmployee.earnings() );
31
32       // create four-element Employee array
33       Employee employees[] = new Employee[ 4 ];
34
35       // initialize array with Employees
36       employees[ 0 ] = salariedEmployee;
37       employees[ 1 ] = hourlyEmployee;
38       employees[ 2 ] = commissionEmployee;
39       employees[ 3 ] = basePlusCommissionEmployee;
40
41       System.out.println( "Employees processed polymorphically:\n" );
42
43       // generically process each element in array employees
44       for ( Employee currentEmployee : employees )
45       {
46          System.out.println( currentEmployee ); // invokes toString
47
```

Assigning subclass objects to supercalss variables

Implicitly and polymorphically call **`toString`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
48          // determine whether element is a BasePlusCommissionEmployee
49          if ( currentEmployee instanceof BasePlusCommissionEmployee )
50          {
51             // downcast Employee reference to
52             // BasePlusCommissionEmployee reference
53             BasePlusCommissionEmployee employee =
54                ( BasePlusCommissionEmployee ) currentEmployee;
55
56             double oldBaseSalary = employee.getBaseSalary();
57             employee.setBaseSalary( 1.10 * oldBaseSalary );
58             System.out.printf(
59                "new base salary with 10%% increase is: $%,.2f\n",
60                employee.getBaseSalary() );
61          } // end if
62
63          System.out.printf(
64             "earned $%,.2f\n\n", currentEmployee.earnings() );
65       } // end for
66
67       // get type name of each object in employees array
68       for ( int j = 0; j < employees.length; j++ )
69          System.out.printf( "Employee %d is a %s\n", j,
70             employees[ j ].getClass().getName() );
71    } // end main
72 } // end class PayrollSystemTest
```

If the **currentEmployee** variable points to a **BasePlusCommissionEmployee** object

PayrollSystemTest

Downcast **currentEmployee** to a **BasePlusCommissionEmployee** reference

(3 of 5)

Give **BasePlusCommissionEmployee**s a 10% base salary bonus

Polymorphically call **earnings** method

Call **getClass** and **getName** methods to display each **Employee** subclass object's class name



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
Employees processed individually:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
earned: $800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: $16.75; hours worked: 40.00
earned: $670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: $10,000.00; commission rate: 0.06
earned: $600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: $5,000.00; commission rate: 0.04; base salary: $300.00
earned: $500.00
```

Outline

PayrollSystemTest.java

(4 of 5)



© 1992-2007 Pearson Education, Inc. All rights reserved.

```
Employees processed polymorphically:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
earned $800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: $16.75; hours worked: 40.00
earned $670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: $10,000.00; commission rate: 0.06
earned $600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: $5,000.00; commission rate: 0.04; base salary: $300.00
new base salary with 10% increase is: $330.00
earned $530.00

Employee 0 is a SalariedEmployee
Employee 1 is a HourlyEmployee
Employee 2 is a CommissionEmployee
Employee 3 is a BasePlusCommissionEmployee
```

Same results as when the employees were processed individually

Base salary is increased by 10%

Each employee's type is displayed

Outline

`PayrollSystemTest.java`

(5 of 5)



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.5.6 Demonstrating Polymorphic Processing, Operator `instanceof` and Downcasting

- **Dynamic binding**
  - **Also known as late binding**
  - **Calls to overridden methods are resolved at execution time, based on the type of object referenced**
- **`instanceof` operator**
  - **Determines whether an object is an instance of a certain type**



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.5.6 Demonstrating Polymorphic Processing, Operator `instanceof` and Downcasting (Cont.)

- **Downcasting**
  - **Convert a reference to a superclass to a reference to a subclass**
  - **Allowed only if the object has an *is-a* relationship with the subclass**
- **`getClass` method**
  - **Inherited from `Object`**
  - **Returns an object of type `Class`**
- **`getName` method of class `Class`**
  - **Returns the class's name**



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.5.7 Summary of the Allowed Assignments Between Superclass and Subclass Variables

- **Superclass and subclass assignment rules**
  - **Assigning a superclass reference to a superclass variable is straightforward**
  - **Assigning a subclass reference to a subclass variable is straightforward**
  - **Assigning a subclass reference to a superclass variable is safe because of the *is-a* relationship**
    - **Referring to subclass-only members through superclass variables is a compilation error**
  - **Assigning a superclass reference to a subclass variable is a compilation error**
    - **Downcasting can get around this error**



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.6 `final` Methods and Classes

- **`final` methods**
  - **Cannot be overridden in a subclass**
  - **`private` and `static` methods are implicitly `final`**
  - **`final` methods are resolved at compile time, this is known as static binding**
    - **Compilers can optimize by inlining the code**
- **`final` classes**
  - **Cannot be extended by a subclass**
  - **All methods in a `final` class are implicitly `final`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.6 `static` Method

- Static methods have access to class variables (static variables) without using the class's object (instance).

```
Access_modifier static void methodName() { // Method body. }
```

```
  className.methodName();
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.7 Case Study: Creating and Using Interfaces

- **Interfaces**
  - **Keyword `interface`**
  - **Contains only constants and `abstract` methods**
    - **All fields are implicitly `public`, `static` and `final`**
    - **All methods are implicitly `public` `abstract` methods**
  - **Classes can `implement` interfaces**
    - **The class must declare each method in the interface using the same signature or the class must be declared `abstract`**
  - **Typically used when disparate classes need to share common methods and constants**
  - **Normally declared in their own files with the same names as the interfaces and with the `.java` file-name extension**



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.7.1 Developing a `Payable` Hierarchy

- **`Payable` interface**
  - **Contains method `getPaymentAmount`**
  - **Is implemented by the `Invoice` and `Employee` classes**
- **UML representation of interfaces**
  - **Interfaces are distinguished from classes by placing the word "interface" in guillemets (« and ») above the interface name**
  - **The relationship between a class and an interface is known as realization**
    - **A class "realizes" the methods of an interface**



© 1992-2007 Pearson Education, Inc. All rights reserved.

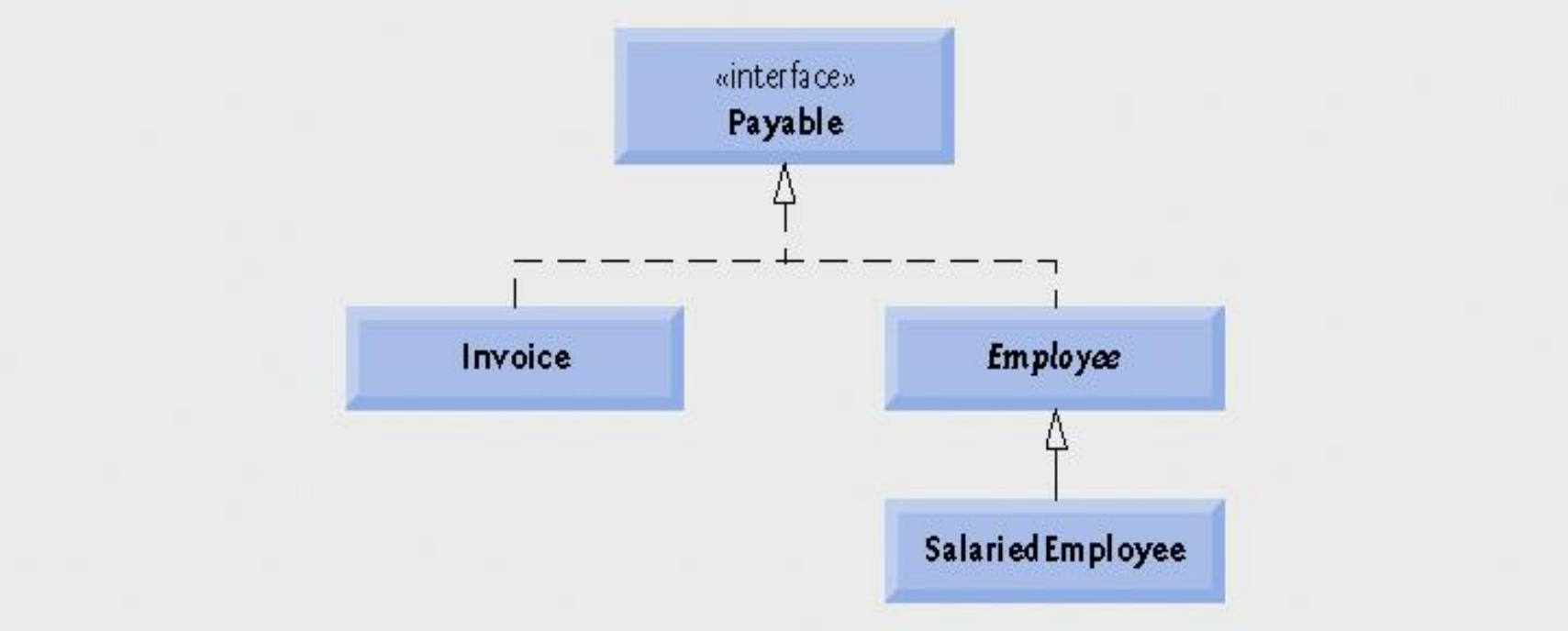


Fig. 10.10 | Payable interface hierarchy UML class diagram.



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Payable.java

```
1 // Fig. 10.11: Payable.java
2 // Payable interface declaration.
3
4 public interface Payable
5 {
6    double getPaymentAmount(); // calculate payment; no implementation
7 } // end interface Payable
```

Declare interface **Payable**

Declare **getPaymentAmount** method which is implicitly **public** and **abstract**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Invoice.java

(1 of 3)

```
1  // Fig. 10.12: Invoice.java
2  // Invoice class implements Payable.
3
4  public class Invoice implements Payable
5  {
6     private String partNumber;
7     private String partDescription;
8     private int quantity;
9     private double pricePerItem;
10
11    // four-argument constructor
12    public Invoice( String part, String description, int count,
13       double price )
14    {
15       partNumber = part;
16       partDescription = description;
17       setQuantity( count ); // validate and store quantity
18       setPricePerItem( price ); // validate and store price per item
19    } // end four-argument Invoice constructor
20
21    // set part number
22    public void setPartNumber( String part )
23    {
24       partNumber = part;
25    } // end method setPartNumber
26
```

Class **Invoice** implements interface **Payable**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Invoice.java

(2 of 3)

```
27    // get part number
28    public String getPartNumber()
29    {
30       return partNumber;
31    } // end method getPartNumber
32
33    // set description
34    public void setPartDescription( String description )
35    {
36       partDescription = description;
37    } // end method setPartDescription
38
39    // get description
40    public String getPartDescription()
41    {
42       return partDescription;
43    } // end method getPartDescription
44
45    // set quantity
46    public void setQuantity( int count )
47    {
48       quantity = ( count < 0 ) ? 0 : count; // quantity cannot be negative
49    } // end method setQuantity
50
51    // get quantity
52    public int getQuantity()
53    {
54       return quantity;
55    } // end method getQuantity
56
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Invoice.java

(3 of 3)

```
57    // set price per item
58    public void setPricePerItem( double price )
59    {
60       pricePerItem = ( price < 0.0 ) ? 0.0 : price; // validate price
61    } // end method setPricePerItem
62
63    // get price per item
64    public double getPricePerItem()
65    {
66       return pricePerItem;
67    } // end method getPricePerItem
68
69    // return String representation of Invoice object
70    public String toString()
71    {
72       return String.format( "%s: \n%s: %s (%s) \n%s: %d \n%s: $%,.2f",
73          "invoice", "part number", getPartNumber(), getPartDescription(),
74          "quantity", getQuantity(), "price per item", getPricePerItem() );
75    } // end method toString
76
77    // method required to carry out contract with interface Payable
78    public double getPaymentAmount()
79    {
80       return getQuantity() * getPricePerItem(); // calculate total cost
81    } // end method getPaymentAmount
82 } // end class Invoice
```

Declare **`getPaymentAmount`** to fulfill contract with interface **`Payable`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.7.3 Creating Class `Invoice`

- **A class can implement as many interfaces as it needs**
  - **Use a comma-separated list of interface names after keyword implements**
    - **Example: `public class` *ClassName* `extends` *SuperclassName* `implements` *FirstInterface*`,` *SecondInterface*`, ...`**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Employee.java

(1 of 3)

```
1  // Fig. 10.13: Employee.java
2  // Employee abstract superclass implements Payable.
3
4  public abstract class Employee implements Payable
5  {
6     private String firstName;
7     private String lastName;
8     private String socialSecurityNumber;
9
10    // three-argument constructor
11    public Employee( String first, String last, String ssn )
12    {
13       firstName = first;
14       lastName = last;
15       socialSecurityNumber = ssn;
16    } // end three-argument Employee constructor
17
```

Class **Employee** implements interface **Payable**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Employee.java

(2 of 3)

```
18    // set first name
19    public void setFirstName( String first )
20    {
21       firstName = first;
22    } // end method setFirstName
23
24    // return first name
25    public String getFirstName()
26    {
27       return firstName;
28    } // end method getFirstName
29
30    // set last name
31    public void setLastName( String last )
32    {
33       lastName = last;
34    } // end method setLastName
35
36    // return last name
37    public String getLastName()
38    {
39       return lastName;
40    } // end method getLastName
41
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

Employee.java

(3 of 3)

```
42    // set social security number
43    public void setSocialSecurityNumber( String ssn )
44    {
45       socialSecurityNumber = ssn; // should validate
46    } // end method setSocialSecurityNumber
47
48    // return social security number
49    public String getSocialSecurityNumber()
50    {
51       return socialSecurityNumber;
52    } // end method getSocialSecurityNumber
53
54    // return String representation of Employee object
55    public String toString()
56    {
57       return String.format( "%s %s\nsocial security number: %s",
58          getFirstName(), getLastName(), getSocialSecurityNumber() );
59    } // end method toString
60
61    // Note: We do not implement Payable method getPaymentAmount here so
62    // this class must be declared abstract to avoid a compilation error.
63 } // end abstract class Employee
```

`getPaymentAmount` method is not implemented here



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.7.5 Modifying Class `SalariedEmployee` for Use in the `Payable` Hierarchy

- **Objects of any subclasses of the class that implements the interface can also be thought of as objects of the interface**
  - **A reference to a subclass object can be assigned to an interface variable if the superclass implements that interface**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

```
1  // Fig. 10.14: SalariedEmployee.java
2  // SalariedEmployee class extends Employee, which implements Payable.
3
4  public class SalariedEmployee extends Employee
5  {
6     private double weeklySalary;
7
8     // four-argument constructor
9     public SalariedEmployee( String first, String last, String ssn,
10       double salary )
11    {
12       super( first, last, ssn ); // pass to Employee constructor
13       setWeeklySalary( salary ); // validate and store salary
14    } // end four-argument SalariedEmployee constructor
15
16    // set salary
17    public void setWeeklySalary( double salary )
18    {
19       weeklySalary = salary < 0.0 ? 0.0 : salary;
20    } // end method setWeeklySalary
21
```

Class **SalariedEmployee** extends class **Employee** (which implements interface **Payable**)

SalariedEmployee

.java

(1 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

SalariedEmployee.java

```
22    // return salary
23    public double getWeeklySalary()
24    {
25       return weeklySalary;
26    } // end method getWeeklySalary
27
28    // calculate earnings; implement interface Payable method that was
29    // abstract in superclass Employee
30    public double getPaymentAmount()
31    {
32       return getWeeklySalary();
33    } // end method getPaymentAmount
34
35    // return String representation of SalariedEmployee object
36    public String toString()
37    {
38       return String.format( "salaried employee: %s\n%s: $%,.2f",
39          super.toString(), "weekly salary", getWeeklySalary() );
40    } // end method toString
41 } // end class SalariedEmployee
```

Declare **`getPaymentAmount`** method instead of **`earnings`** method

(2 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

PayableInterfaceTest.java

```
1  // Fig. 10.15: PayableInterfaceTest.java
2  // Tests interface Payable.
3
4  public class PayableInterfaceTest
5  {
6     public static void main( String args[] )
7     {
8        // create four-element Payable array
9        Payable payableObjects[] = new Payable[ 4 ];
10
11       // populate array with objects that implement Payable
12       payableObjects[ 0 ] = new Invoice( "01234", "seat", 2, 375.00 );
13       payableObjects[ 1 ] = new Invoice( "56789", "tire", 4, 79.95 );
14       payableObjects[ 2 ] =
15          new SalariedEmployee( "John", "Smith", "111-11-1111", 800.00 );
16       payableObjects[ 3 ] =
17          new SalariedEmployee( "Lisa", "Barnes", "888-88-8888", 1200.00 );
18
19       System.out.println(
20          "Invoices and Employees processed polymorphically:\n" );
21
```

Declare array of **Payable** variables

Assigning references to **Invoice** objects to **Payable** variables

Assigning references to **SalariedEmployee** objects to **Payable** variables



© 1992-2007 Pearson Education, Inc. All rights reserved.

Outline

PayableInterface

Test.java

```
22       // generically process each element in array payableObjects
23       for ( Payable currentPayable : payableObjects )
24       {
25          // output currentPayable and its appropriate payment amount
26          System.out.printf( "%s \n%s: $%,.2f\n\n",
27             currentPayable.toString(),
28             "payment due", currentPayable.getPaymentAmount() );
29       } // end for
30    } // end main
31 } // end class PayableInterfaceTest
```

Call **`toString`** and **`getPaymentAmount`** methods polymorphically

(2 of 2)

```
Invoices and Employees processed polymorphically:

invoice:
part number: 01234 (seat)
quantity: 2
price per item: $375.00
payment due: $750.00

invoice:
part number: 56789 (tire)
quantity: 4
price per item: $79.95
payment due: $319.80

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
payment due: $800.00

salaried employee: Lisa Barnes
social security number: 888-88-8888
weekly salary: $1,200.00
payment due: $1,200.00
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

10.7.7 Declaring Constants with Interfaces

- **Interfaces can be used to declare constants used in many class declarations**
  - **These constants are implicitly `public`, `static` and `final`**
  - **Using a `static import` declaration allows clients to use these constants with just their names**



© 1992-2007 Pearson Education, Inc. All rights reserved.

Interface	Description
`Comparable`	As you learned in Chapter 2, Java contains several comparison operators (e.g., <, <=, >, >=, ==, !=) that allow you to compare primitive values. However, these operators cannot be used to compare the contents of objects. Interface `Comparable` is used to allow objects of a class that implements the interface to be compared to one another. The interface contains one method, `compareTo`, that compares the object that calls the method to the object passed as an argument to the method. Classes must implement `compareTo` such that it returns a value indicating whether the object on which it is invoked is less than (negative integer return value), equal to (`0` return value) or greater than (positive integer return value) the object passed as an argument, using any criteria specified by the programmer. For example, if class `Employee` implements `Comparable`, its `compareTo` method could compare `Employee` objects by their earnings amounts. Interface `Comparable` is commonly used for ordering objects in a collection such as an array. We use `Comparable` in Chapter 18, Generics, and Chapter 19, Collections.
`Serializable`	A tagging interface used only to identify classes whose objects can be written to (i.e., serialized) or read from (i.e., deserialized) some type of storage (e.g., file on disk, database field) or transmitted across a network. We use `Serializable` in Chapter 14, Files and Streams, and Chapter 24, Networking.

Fig. 10.16 | Common interfaces of the Java API. (Part 1 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

Interface	Description
`Runnable`	Implemented by any class for which objects of that class should be able to execute in parallel using a technique called multithreading (discussed in Chapter 23, Multithreading). The interface contains one method, `run`, which describes the behavior of an object when executed.
GUI event-listener interfaces	You work with Graphical User Interfaces (GUIs) every day. For example, in your Web browser, you might type in a text field the address of a Web site to visit, or you might click a button to return to the previous site you visited. When you type a Web site address or click a button in the Web browser, the browser must respond to your interaction and perform the desired task for you. Your interaction is known as an event, and the code that the browser uses to respond to an event is known as an event handler. In Chapter 11, GUI Components: Part 1, and Chapter 22, GUI Components: Part 2, you will learn how to build Java GUIs and how to build event handlers to respond to user interactions. The event handlers are declared in classes that implement an appropriate event-listener interface. Each event listener interface specifies one or more methods that must be implemented to respond to user interactions.
`SwingConstants`	Contains a set of constants used in GUI programming to position GUI elements on the screen. We explore GUI programming in Chapters 11 and 22.

Fig. 10.16 | Common interfaces of the Java API. (Part 2 of 2)



© 1992-2007 Pearson Education, Inc. All rights reserved.

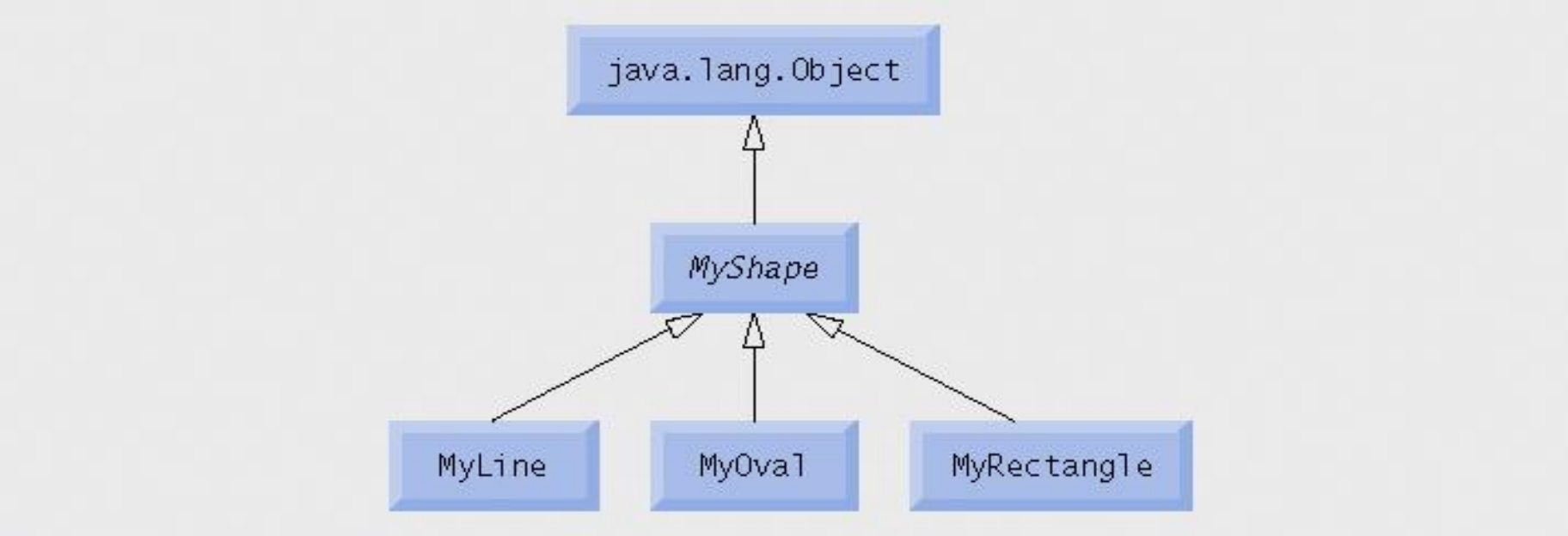


Fig. 10.17 | MyShape hierarchy.



© 1992-2007 Pearson Education, Inc. All rights reserved.

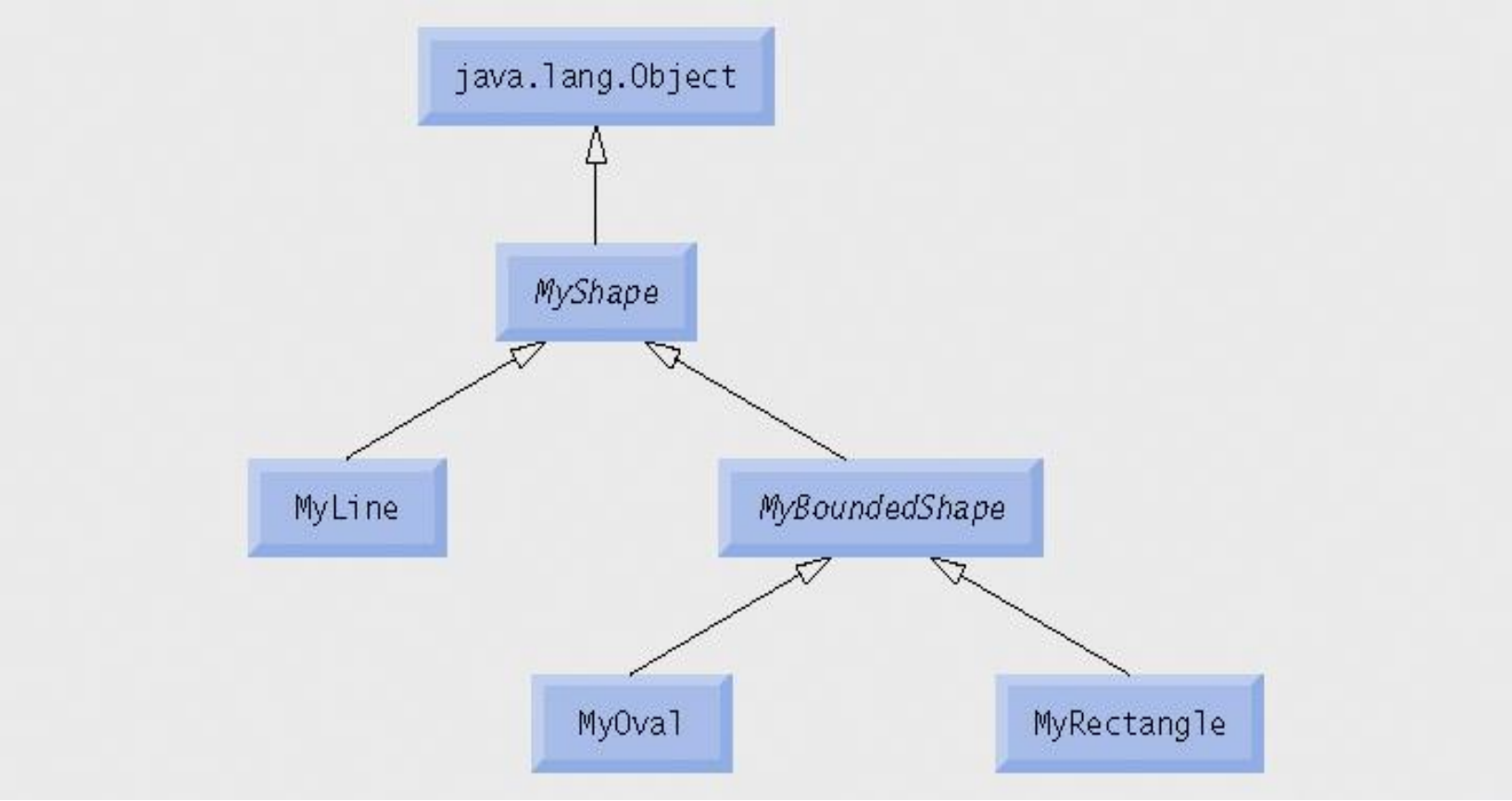


Fig. 10.18 | MyShape hierarchy with MyBoundedShape.

© 1992-2007 Pearson Education, Inc. All rights reserved.

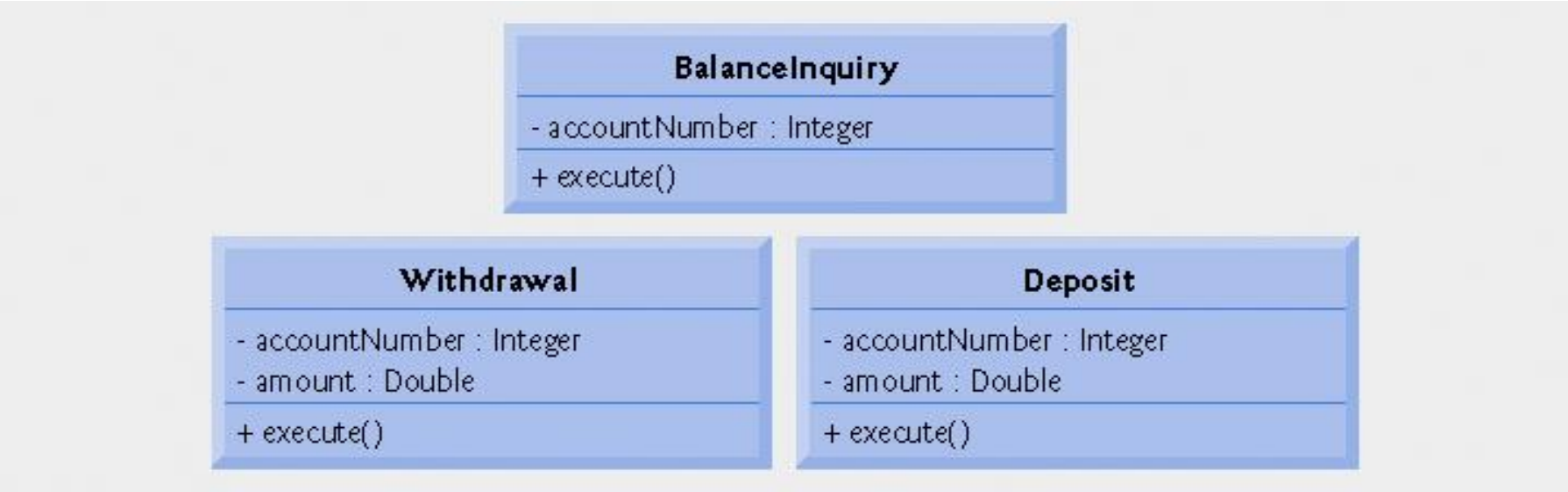


Fig. 10.19 | Attributes and operations of classes `BalanceInquiry`, `Withdrawal` and `Deposit`.

© 1992-2007 Pearson Education, Inc. All rights reserved.

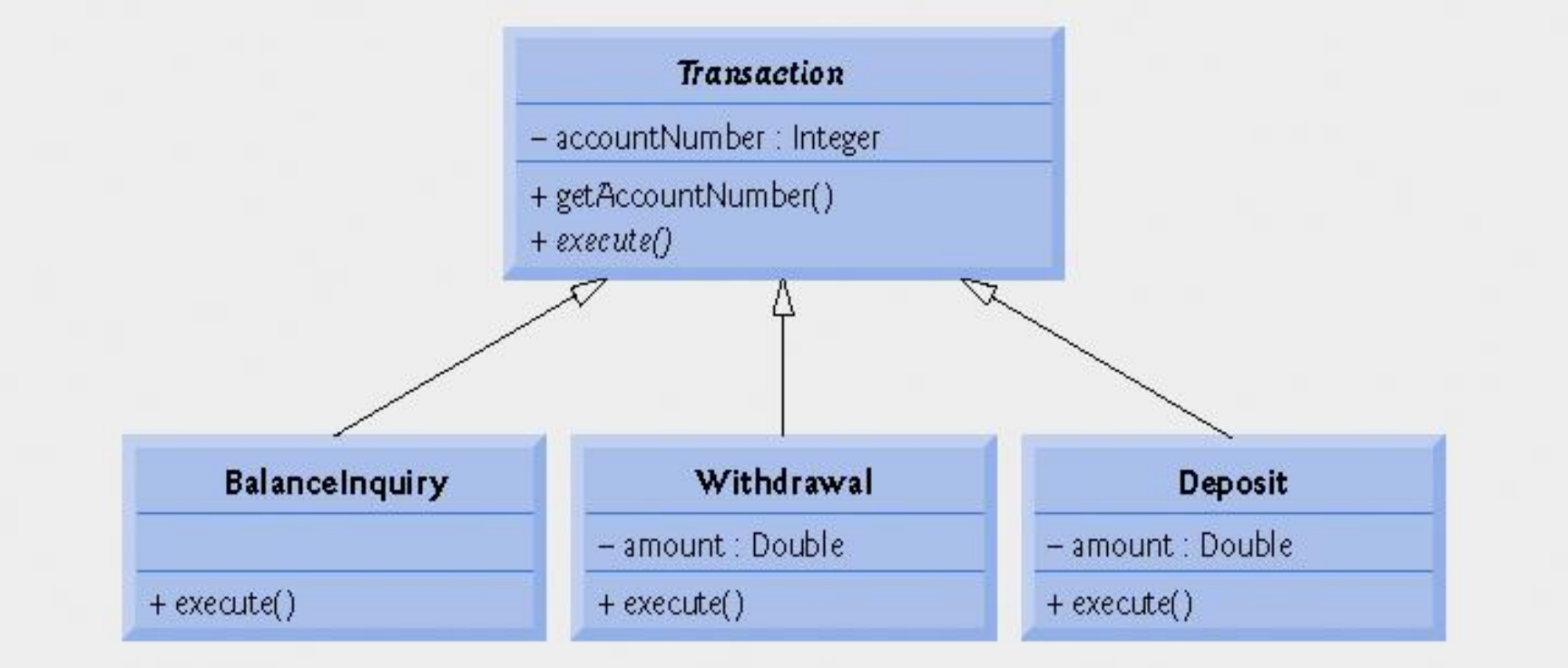


Fig. 10. 20 | Class diagram modeling generalization of superclass `Transaction` and subclasses `BalanceInquiry`, `Withdrawal` and `Deposit`. Note that abstract class names (e.g., `Transaction`) and method names (e.g., `execute` in class `Transaction`) appear in italics.

© 1992-2007 Pearson Education, Inc. All rights reserved.

Java String class methods

- The **java.lang.String** class provides many useful methods to perform operations on sequence of char values.
- Java String length()
  - The **Java String class length()** method finds the length of a string. The length of the Java string is the same as the Unicode code units of the string.
  - String s1="javatpoint";
  - System.out.println("string length is: "+s1.length());
-



© 1992-2007 Pearson Education, Inc. All rights reserved.

Java String class methods

- Java String charAt()
  - The **Java String class charAt()** method returns *a char value at the given index number*.

```
String name="javatpoint";
char ch=name.charAt(4);//returns the char value at the 4th index

System.out.println(ch);
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Java String class methods

- Java String concat
- The **Java String class concat()** method *combines specified string at the end of this string*. It returns a combined string. It is like appending another string.

```
String str1 = "Hello";
String str2 = "Javatpoint";
String str4 = str1.concat(str2);
System.out.println(str4);
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Java String class methods

- The **Java String class replace()** method returns a string replacing all the old char or CharSequence to new char or CharSequence.

```
String s1="javat  is a very good Language";
String replaceString=s1.replace('a','e');//replaces all occurrences of 'a' to 'e'
System.out.println(replaceString);
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Java String class methods

- Java String toLowerCase()
  - The **java string toLowerCase()** method returns the string in lowercase letter. In other words, it converts all characters of the string into lower case letter.

```
String s = "JAVA HELLO stRIng";
        String eng = s.toLowerCase();
        System.out.println(eng);
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Java String class methods

- Java String toUpperCase()
  - The **java string toUpperCase()** method returns the string in uppercase letter. In other words, it converts all characters of the string into upper case letter.

```
String s1="hello string";
String s1upper=s1.toUpperCase();
System.out.println(s1upper);
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Java String class methods

- Java String split()
  - The **java string split()** method splits this string against given regular expression and returns a char array.

```
String s1="java string split method by java";
String[] words=s1.split("\\s");//splits the string based on whitespace
for (int i = 0; i < words.length; i++)
{
  System.out.println(words[i]);
}
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Java String class methods

- Java String substring()
- The **Java String class substring()** method returns a part of the string.

```
String s = "welcome to java.com, best site for learning";
System.out.println(s.substring(11, 19));
System.out.println(s.substring(11));
```



© 1992-2007 Pearson Education, Inc. All rights reserved.

Create classes for banking first class Saving Account containing (name of holder, account number, branch number, balance, interest, withdraw limit) and second-class Current account containing (name of holder, account number, branch number, balance, overdraft)

- Apply Setter & Getter.
- Apply Inheritance of this classes.
- Create Constructor in the super class containing (name, account number);
- Create Constructor in the super class containing (name, account number, balance);
- Create Constructor in the saving Account class containing (name, account number, balance, interest);
- Create Report to print details of accounts using overring method.